

Software Delivery, Operations, and Evolution

The Complete Software Engineering Lifecycle — Volume II

Software Delivery,

Operations, and Evolution

The Complete Software Engineering Lifecycle — Volume II



Moody Amakobe

Global Data Science Institute

2026 Edition

Print ISBN: 979-8-2408-9370-4

Copyright © 2026 Moody Amakobe

2026 Edition

Global Data Science Institute

Print ISBN: 979-8-2408-9370-4

Ebook ISBN: TBD

This work is licensed under the Creative Commons Attribution 4.0 International License (CC BY 4.0).

Software engineering technologies, tools, frameworks, cloud platforms, APIs, and standards evolve continuously. Readers should consult current vendor and standards documentation before production implementation.

About the Series

The Complete Software Engineering Lifecycle is a two-volume series examining software engineering from requirements and design through production delivery, operations, security, and long-term evolution.

Volume I focuses on foundations, design, development practices, and quality. Volume II focuses on delivery, production systems, security, maintenance, and professional practice. Each volume is designed to stand on its own while remaining part of a coherent lifecycle.

Preface to Volume II

Software becomes valuable not merely when it has been designed and tested, but when it can be integrated, delivered, deployed, operated, secured, maintained, and evolved reliably. Volume II begins at that transition.

This volume follows software from continuous integration through data and API design, cloud deployment, security, maintenance, professional responsibility, and lifecycle synthesis. It assumes familiarity with basic software-development concepts, but its front matter and chapter progression make it usable as an independent textbook and reference.

Foundational topics such as requirements, modeling, architecture, user experience, Agile practice, version control, and testing receive detailed treatment in *Software Engineering Foundations and Design*, Volume I of this series. They are identified explicitly when a cross-volume reference is useful.

How to Use This Volume

Read the chapters in sequence to follow the path from integrated code to sustainable production systems, or use individual chapters as operational references. Learning objectives, examples, terminal sessions, summaries, key terms, review questions, and exercises support both independent study and structured courses.

The final chapter synthesizes the lifecycle into a professional project. Readers using only this volume can adapt its project guidance to an existing design or codebase; ownership of Volume I is not required.

Table of contents



About the Series	v
Preface to Volume II	vi
How to Use This Volume	vii
I. Delivery and Integration	1
1. Continuous Integration and Continuous Deployment	2
1.1. The Evolution of Software Delivery	3
1.2. Continuous Integration Fundamentals	7
1.3. Building CI Pipelines with GitHub Actions	13
1.4. Continuous Deployment Strategies	27
1.5. Environment Management	36
1.6. Infrastructure as Code	42
1.7. Deployment Automation	51
2. Data Management and APIs	75
2.1. The Role of Data in Software Systems	76
2.2. Relational Databases	78
2.3. NoSQL Databases	95
2.4. RESTful API Design	105
2.5. GraphQL	117

2.6.	API Documentation	131
2.7.	Data Validation	135

II. Production Systems 151

3. Cloud Services and Deployment 152

3.1.	The Cloud Computing Revolution	153
3.2.	Core Cloud Services	157
3.3.	Containerization with Docker	164
3.4.	Container Orchestration with Kubernetes	173
3.5.	Serverless Computing	183
3.6.	Infrastructure as Code	195
3.7.	Cloud Security Best Practices	204

4. Software Security 216

4.1.	The Imperative of Software Security	217
4.2.	OWASP Top 10 Web Application Vulnerabilities	221
4.3.	Input Validation and Sanitization	254
4.4.	Security Headers	258
4.5.	Security Testing	261
4.6.	Incident Response	265

III. Evolution and Professional Practice 274

5. Software Maintenance and Evolution 275

5.1.	The Reality of Software Maintenance	276
5.2.	Technical Debt	279
5.3.	Refactoring	284
5.4.	Working with Legacy Systems	296
5.5.	Documentation	303
5.6.	Requirements	308
5.7.	Project Structure	309

6. Professional Practice and Ethics 328

6.1.	The Ethical Dimension of Software Engineering	329
6.2.	Intellectual Property and Licensing	335

6.3.	Team Dynamics and Collaboration	340
6.4.	Career Development	345
6.5.	Legal and Regulatory Considerations	348
6.6.	Emerging Ethical Challenges	351
6.7.	Building an Ethical Practice	353
7.	Final Project Integration and Course Synthesis	361
7.1.	The Integration Challenge	362
7.2.	Project Completion Strategies	363
7.3.	Polish and Refinement	366
7.4.	Comprehensive Testing	370
7.5.	Preparing Effective Presentations	372
7.6.	Documentation for Posterity	375
7.7.	Project Structure	377

Part I.

Delivery and Integration

Continuous Integration and Continuous Deployment

Automating integration, delivery, deployment, and operational feedback.

LEARNING OBJECTIVES

- ✓ By the end of this chapter, you will be able to:
 - ✓ Explain the principles and benefits of continuous integration and continuous deployment
 - ✓ Distinguish between continuous integration, continuous delivery, and continuous deployment
 - ✓ Design and implement CI/CD pipelines using GitHub Actions
 - ✓ Configure automated builds, tests, and deployments
 - ✓ Implement deployment strategies including blue-green, canary, and rolling deployments
 - ✓ Manage environment configurations and secrets securely
 - ✓ Monitor deployments and implement rollback procedures
 - ✓ Apply infrastructure as code principles for reproducible environments
 - ✓ Troubleshoot common CI/CD pipeline issues

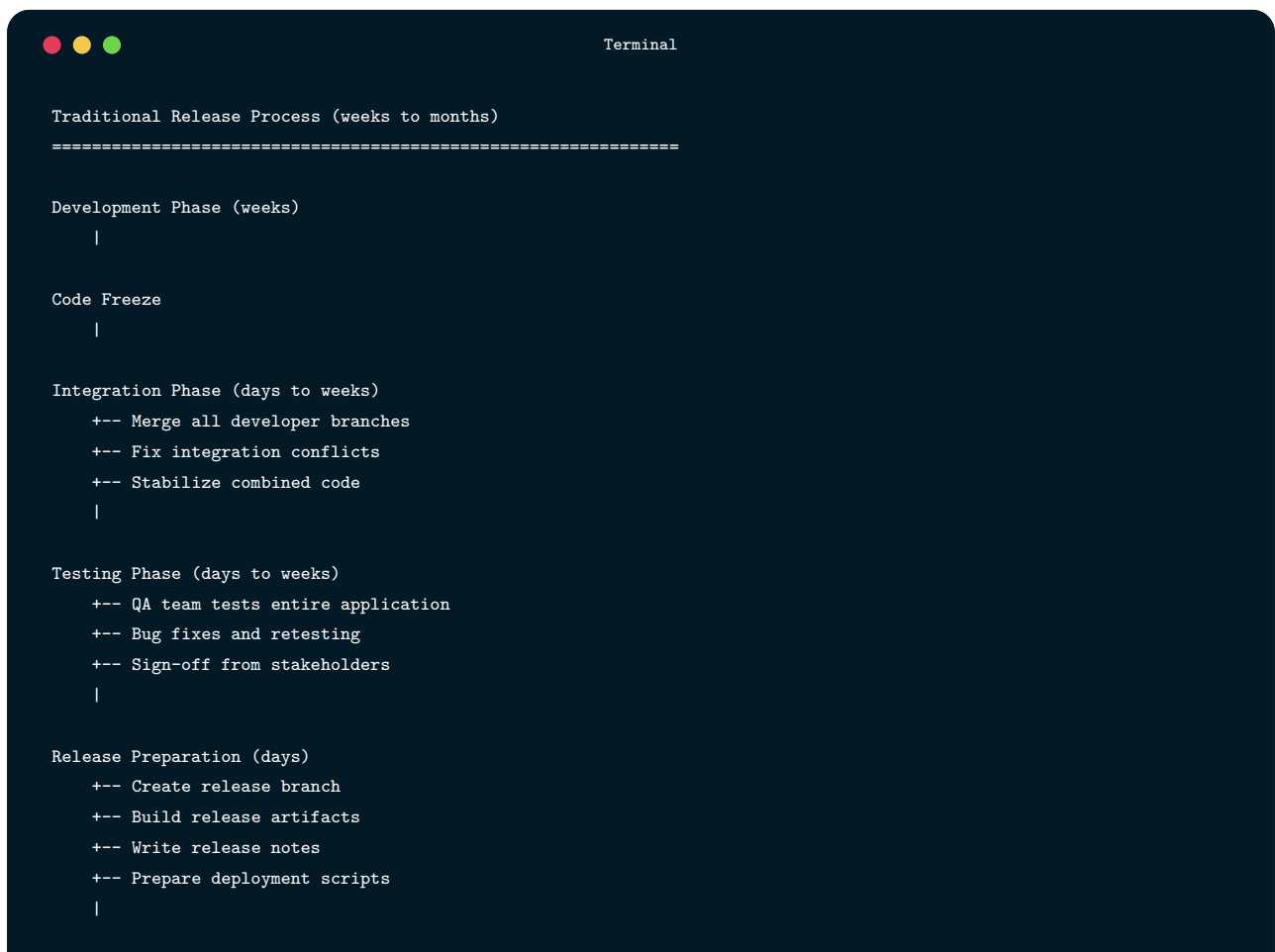
1.1

The Evolution of Software Delivery

Software delivery has transformed dramatically over the past decades. What once took months or years now happens in minutes. Understanding this evolution helps appreciate why CI/CD practices exist and why they matter.

1.1.1 The Old Way: Manual Releases

In traditional software development, releases were major events:



```
Terminal

Traditional Release Process (weeks to months)
=====

Development Phase (weeks)
|

Code Freeze
|

Integration Phase (days to weeks)
+-- Merge all developer branches
+-- Fix integration conflicts
+-- Stabilize combined code
|

Testing Phase (days to weeks)
+-- QA team tests entire application
+-- Bug fixes and retesting
+-- Sign-off from stakeholders
|

Release Preparation (days)
+-- Create release branch
+-- Build release artifacts
+-- Write release notes
+-- Prepare deployment scripts
|
```

Terminal -- continued

```

Deployment (hours to days)
+-- Schedule maintenance window
+-- Notify users of downtime
+-- Manual server updates
+-- Database migrations
+-- Smoke testing
+-- Prayer and hope
|

Post-Release (days)
+-- Monitor for issues
+-- Hotfix critical bugs
+-- Begin next development cycle

```

Problems with this approach:

- **Integration hell:** Merging weeks of isolated work caused massive conflicts
- **Long feedback loops:** Bugs weren't discovered until late in the cycle
- **Risky deployments:** Large changes meant large risks
- **Infrequent releases:** Customers waited months for features and fixes
- **Stressful releases:** "Release weekends" became dreaded events
- **Fear of change:** Teams avoided changes to avoid risk

1.1.2 The CI/CD Revolution

Modern practices flip this model:

Terminal

```

Modern CI/CD Process (minutes to hours)
=====

Developer commits code
|
| (seconds)
Automated pipeline triggers
|
| (minutes)
+-----+
| Build -> Lint -> Unit Tests -> Integration Tests -> Security |
+-----+

```


Terminal -- continued

```

|
| (minutes)
Deploy to staging environment
|
| (minutes)
Automated E2E tests on staging
|
| (automatic or one-click)
Deploy to production
|
| (continuous)
Monitoring and alerting

```

Benefits:

- **Fast feedback:** Know within minutes if changes break anything
- **Small changes:** Easier to review, test, and debug
- **Reduced risk:** Small, frequent deployments are safer than large, rare ones
- **Faster delivery:** Features reach users in hours, not months
- **Happier teams:** Routine deployments instead of stressful events
- **Higher quality:** Automated testing catches issues before users do

1.1.3 Key Terminology

Understanding the distinctions between related terms:

Terminal

```

+-----+
|              CI/CD TERMINOLOGY              |
+-----+
|
| CONTINUOUS INTEGRATION (CI)                  |
| -----|
| • Developers integrate code frequently (at least daily) |
| • Each integration triggers automated build and tests  |
| • Problems detected early, when they're easy to fix   |
| • Main branch stays stable and deployable             |
|
| CONTINUOUS DELIVERY (CD)                     |
| -----|
| • Code is always in a deployable state             |
|

```

Terminal -- continued

```
| • Automated pipeline prepares release artifacts |
| • Deployment to production requires manual approval |
| • "Push-button" releases whenever business decides |
| |
| CONTINUOUS DEPLOYMENT (CD) |
| ----- |
| • Every change that passes tests deploys automatically |
| • No manual intervention required |
| • Highest level of automation |
| • Requires mature testing and monitoring |
| |
| -----+-----
```

Visual comparison:

Terminal

	Continuous Integration	Continuous Delivery	Continuous Deployment
Code Commit			
Build	Automated	Automated	Automated
Test	Automated	Automated	Automated
Deploy to Staging	Optional	Automated	Automated
Deploy to Prod	Manual	Manual Trigger (One-click)	Automated

1.2

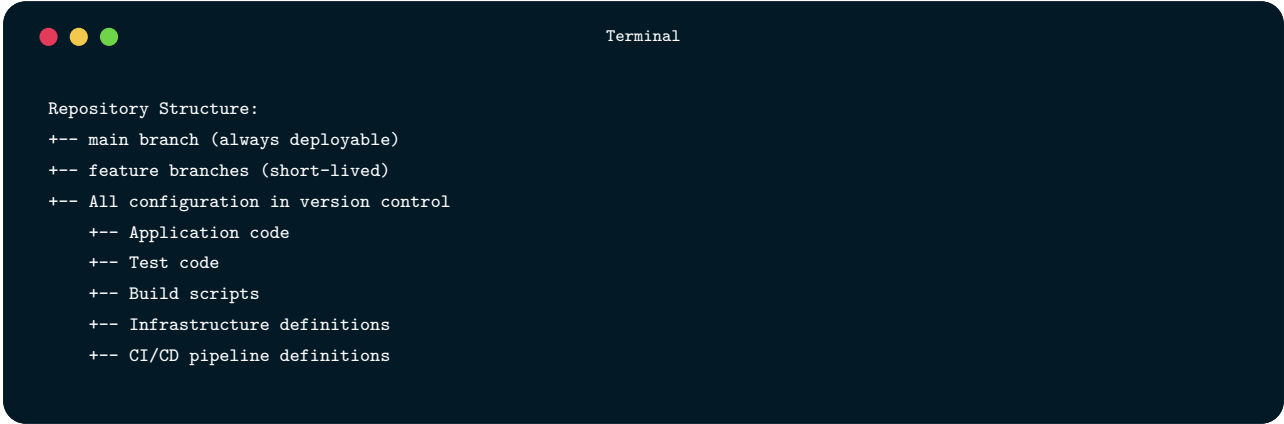
Continuous Integration Fundamentals

Continuous Integration (CI) is the practice of frequently integrating code changes into a shared repository, where each integration is verified by automated builds and tests.

1.2.1 Core CI Practices

1. Maintain a Single Source Repository

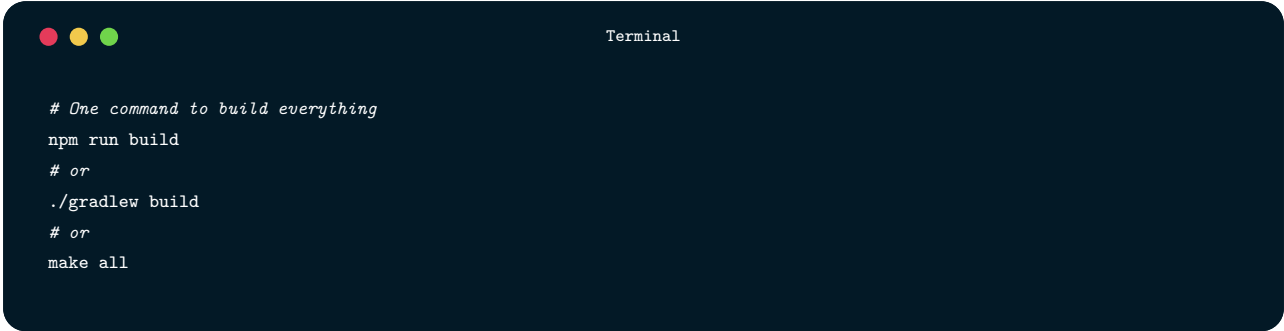
All code lives in version control. Everyone works from the same repository.

A terminal window with a dark blue background and white text. The title bar shows three colored circles (red, yellow, green) and the word "Terminal". The text inside lists the repository structure with comments.

```
Repository Structure:  
+-- main branch (always deployable)  
+-- feature branches (short-lived)  
+-- All configuration in version control  
    +-- Application code  
    +-- Test code  
    +-- Build scripts  
    +-- Infrastructure definitions  
    +-- CI/CD pipeline definitions
```

2. Automate the Build

Building software should require a single command:

A terminal window with a dark blue background and white text. The title bar shows three colored circles (red, yellow, green) and the word "Terminal". The text inside shows a list of build commands with comments.

```
# One command to build everything  
npm run build  
# or  
./gradlew build  
# or  
make all
```

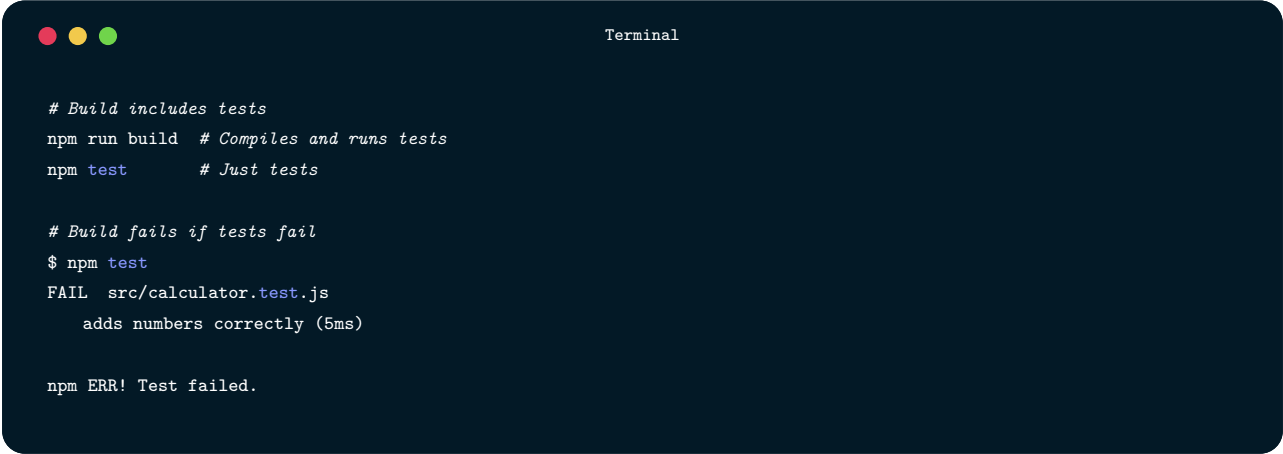
The build should:

- Compile all code

- Run static analysis
- Generate artifacts
- Be reproducible (same inputs → same outputs)

3. Make the Build Self-Testing

Every build runs automated tests:



```
Terminal

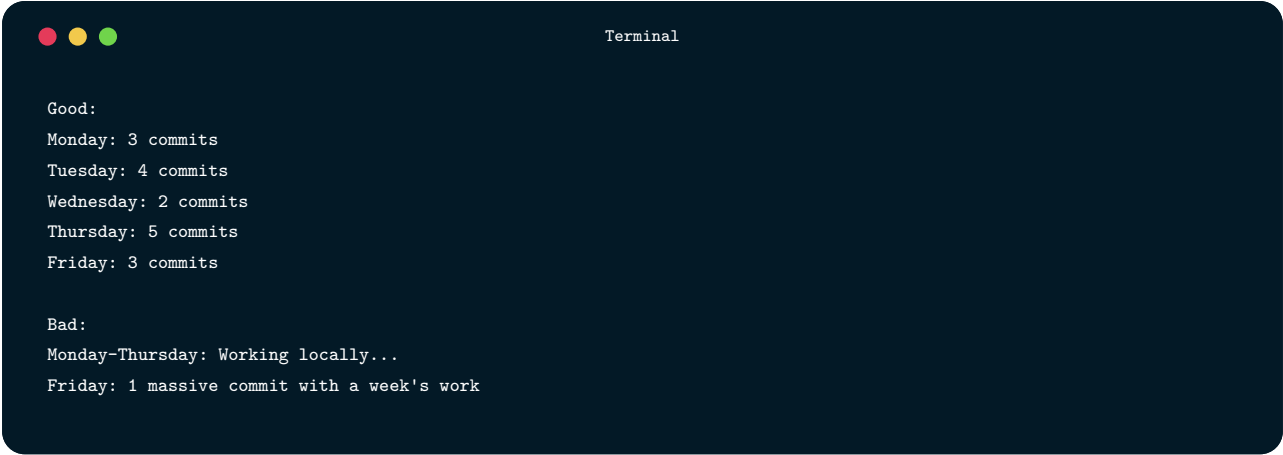
# Build includes tests
npm run build # Compiles and runs tests
npm test     # Just tests

# Build fails if tests fail
$ npm test
FAIL src/calculator.test.js
      adds numbers correctly (5ms)

npm ERR! Test failed.
```

4. Everyone Commits Frequently

Integrate at least daily—more often is better:



```
Terminal

Good:
Monday: 3 commits
Tuesday: 4 commits
Wednesday: 2 commits
Thursday: 5 commits
Friday: 3 commits

Bad:
Monday-Thursday: Working locally...
Friday: 1 massive commit with a week's work
```

5. Every Commit Triggers a Build

Automated systems build and test every change:

 Terminal

```
Commit pushed
|

CI server detects change
|

Pipeline executes automatically
|
+-- Success -> Green checkmark PASS
|
+-- Failure -> Red X, team notified FAIL
```

6. Keep the Build Fast

Fast feedback is essential. Target build times:

 Terminal

Build Stage	Target Time
Lint	< 30 seconds
Unit tests	< 5 minutes
Integration tests	< 10 minutes
Full pipeline	< 15 minutes

If build takes > 15 minutes, consider:

- Parallelizing tests
- Optimizing slow tests
- Splitting pipeline stages

7. Test in a Clone of Production

Test environments should mirror production:

 Terminal

```
Production Environment
+-- Ubuntu 22.04
+-- Node.js 20.x
+-- PostgreSQL 15
+-- Redis 7
+-- nginx 1.24
```

Terminal -- continued

```
CI Test Environment (should match!)
+-- Ubuntu 22.04
+-- Node.js 20.x
+-- PostgreSQL 15
+-- Redis 7
+-- nginx 1.24
```

8. Make It Easy to Get Latest Deliverables

Anyone should be able to get the latest working version:

Terminal

```
# Get latest artifacts
aws s3 cp s3://builds/latest/app.zip .

# Or use package registry
npm install @company/app@latest
docker pull company/app:latest
```

9. Everyone Can See What's Happening

Build status is visible to all:

Terminal

```
+-----+
| CI DASHBOARD |
+-----+
|
| main branch:    PASS Build #1234 passed (3m ago) |
| develop branch: PASS Build #567 passed (15m ago) |
| feature/auth:   FAIL Build #89 failed - Test failure (1h ago) |
| feature/api:    Build #90 in progress...         |
|
| Recent Activity: |
| +-- alice: Merged PR #142 into main                |
| +-- bob: Fixed failing test in feature/auth         |
| +-- carol: Opened PR #143 for review                |
|
+-----+
```

10. Automate Deployment

Deployment should be automated, not manual:

```

Terminal

# Not this:
ssh production-server
cd /var/www/app
git pull
npm install
npm run build
pm2 restart all

# This:
git push origin main # Triggers automated deployment

```

1.2.2 The CI Feedback Loop

CI creates a rapid feedback loop:



When the build breaks:

1. Stop what you're doing
2. Fix the build immediately
3. Don't commit more broken code on top

"The first rule of Continuous Integration is: when the build breaks, fixing it becomes the team's top priority."

1.2.3 CI Anti-Patterns

Ignoring Broken Builds:



Terminal

"The build's been red for a week, but we're too busy to fix it."

PASS Fix broken builds immediately. A red build is an emergency.

Infrequent Integration:



Terminal

Committing once a week with massive changes

PASS Commit multiple times daily with small changes

Skipping Tests:



Terminal

"I'll add tests later" or "Tests are too slow, skip them"

PASS Tests are non-negotiable. Optimize slow tests.

Not Running Pipeline Locally:



Terminal

"It works on my machine" -> Push -> CI fails

Terminal -- continued

PASS Run the same checks locally before pushing

Long-Lived Feature Branches:

Terminal

Feature branch that diverges for months

PASS Short-lived branches, merged within days

1.3

Building CI Pipelines with GitHub Actions

GitHub Actions is GitHub's built-in CI/CD platform. It's free for public repositories and has generous free tiers for private repositories.

1.3.1 GitHub Actions Concepts

Terminal

```
+-----+
|          GITHUB ACTIONS HIERARCHY          |
+-----+
|
| WORKFLOW
| +-- Defined in .github/workflows/*.yaml
| +-- Triggered by events (push, PR, schedule, etc.)
| +-- Contains one or more jobs
|
| JOB
| +-- Runs on a specific runner (ubuntu, windows, macos)
```

Terminal -- continued

```

| +-- Contains one or more steps
| +-- Jobs run in parallel by default
| +-- Can depend on other jobs
|
| STEP
| +-- Individual task within a job
| +-- Either runs a command or uses an action
| +-- Steps run sequentially
|
| ACTION
| +-- Reusable unit of code
| +-- Published in GitHub Marketplace
| +-- Example: actions/checkout, actions/setup-node
|
+-----+

```

Visual representation:

Terminal

```

Workflow: ci.yml
|
+-- Job: lint
|   +-- Step: Checkout code
|   +-- Step: Setup Node.js
|   +-- Step: Run linter
|
+-- Job: test (depends on: lint)
|   +-- Step: Checkout code
|   +-- Step: Setup Node.js
|   +-- Step: Install dependencies
|   +-- Step: Run tests
|
+-- Job: build (depends on: test)
    +-- Step: Checkout code
    +-- Step: Setup Node.js
    +-- Step: Build application
    +-- Step: Upload artifacts

```

1.3.2 Basic Workflow Structure

```
Terminal

# .github/workflows/ci.yml

# Workflow name (displayed in GitHub UI)
name: CI

# Triggers - when should this workflow run?
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

# Jobs to execute
jobs:
  # Job identifier
  build:
    # Runner environment
    runs-on: ubuntu-latest

    # Job steps
    steps:
      # Use a pre-built action
      - name: Checkout repository
        uses: actions/checkout@v4

      # Run a shell command
      - name: Display Node version
        run: node --version

      # Multi-line command
      - name: Install and test
        run: |
          npm ci
          npm test
```

1.3.3 Complete CI Pipeline Example

Here's a comprehensive CI pipeline for a Node.js application:

```
Terminal

# .github/workflows/ci.yml
name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

# Environment variables available to all jobs
env:
  NODE_VERSION: '20'

jobs:
  # =====
  # JOB 1: Code Quality Checks
  # =====
  lint:
    name: Lint & Format
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: ${ env.NODE_VERSION }
          cache: 'npm'

      - name: Install dependencies
        run: npm ci

      - name: Run ESLint
        run: npm run lint

      - name: Check Prettier formatting
        run: npm run format:check

      - name: Run TypeScript compiler
        run: npm run type-check

  # =====
  # JOB 2: Unit Tests
  # =====
  unit-tests:
```

Terminal -- continued

```

name: Unit Tests
runs-on: ubuntu-latest
needs: lint # Only run if lint passes

steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Setup Node.js
    uses: actions/setup-node@v4
    with:
      node-version: ${ env.NODE_VERSION }
      cache: 'npm'

  - name: Install dependencies
    run: npm ci

  - name: Run unit tests
    run: npm test -- --coverage --reporters=default --reporters=jest-junit
    env:
      JEST_JUNIT_OUTPUT_DIR: ./reports

  - name: Upload coverage report
    uses: actions/upload-artifact@v4
    with:
      name: coverage-report
      path: coverage/

  - name: Upload test results
    uses: actions/upload-artifact@v4
    if: always() # Upload even if tests fail
    with:
      name: test-results
      path: reports/junit.xml

# =====
# JOB 3: Integration Tests
# =====
integration-tests:
  name: Integration Tests
  runs-on: ubuntu-latest
  needs: lint

  # Service containers for integration tests
  services:
    postgres:
      image: postgres:15

```

Terminal -- continued

```
env:
  POSTGRES_USER: testuser
  POSTGRES_PASSWORD: testpass
  POSTGRES_DB: testdb
ports:
  - 5432:5432
options: >-
  --health-cmd pg_isready
  --health-interval 10s
  --health-timeout 5s
  --health-retries 5

redis:
  image: redis:7
  ports:
    - 6379:6379
  options: >-
    --health-cmd "redis-cli ping"
    --health-interval 10s
    --health-timeout 5s
    --health-retries 5

steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Setup Node.js
    uses: actions/setup-node@v4
    with:
      node-version: ${ env.NODE_VERSION }
      cache: 'npm'

  - name: Install dependencies
    run: npm ci

  - name: Run database migrations
    run: npm run db:migrate
    env:
      DATABASE_URL: postgresql://testuser:testpass@localhost:5432/testdb

  - name: Run integration tests
    run: npm run test:integration
    env:
      DATABASE_URL: postgresql://testuser:testpass@localhost:5432/testdb
      REDIS_URL: redis://localhost:6379

# =====
```

Terminal -- continued

```
# JOB 4: Build
# =====

build:
  name: Build Application
  runs-on: ubuntu-latest
  needs: [unit-tests, integration-tests]

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${ env.NODE_VERSION }
        cache: 'npm'

    - name: Install dependencies
      run: npm ci

    - name: Build application
      run: npm run build
      env:
        NODE_ENV: production

    - name: Upload build artifacts
      uses: actions/upload-artifact@v4
      with:
        name: build-output
        path: dist/
        retention-days: 7

# =====
# JOB 5: Security Scan
# =====

security:
  name: Security Scan
  runs-on: ubuntu-latest
  needs: lint

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
```

Terminal -- continued

```
    node-version: ${ env.NODE_VERSION }}
    cache: 'npm'

- name: Install dependencies
  run: npm ci

- name: Run npm audit
  run: npm audit --audit-level=high

- name: Run Snyk security scan
  uses: snyk/actions/node@master
  continue-on-error: true # Don't fail build, just report
  env:
    SNYK_TOKEN: ${ secrets.SNYK_TOKEN }}
  with:
    args: --severity-threshold=high

# =====
# JOB 6: E2E Tests (only on main/develop)
# =====
e2e-tests:
  name: E2E Tests
  runs-on: ubuntu-latest
  needs: build
  if: github.ref == 'refs/heads/main' || github.ref == 'refs/heads/develop'

  steps:
    - name: Checkout code
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${ env.NODE_VERSION }}
        cache: 'npm'

    - name: Install dependencies
      run: npm ci

    - name: Download build artifacts
      uses: actions/download-artifact@v4
      with:
        name: build-output
        path: dist/

    - name: Run Cypress tests
      uses: cypress-io/github-action@v6
```


Terminal -- continued

```

with:
  start: npm run start:test
  wait-on: 'http://localhost:3000'
  wait-on-timeout: 120

- name: Upload Cypress screenshots
  uses: actions/upload-artifact@v4
  if: failure()
  with:
    name: cypress-screenshots
    path: cypress/screenshots/

- name: Upload Cypress videos
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: cypress-videos
    path: cypress/videos/

```

1.3.4 Workflow Triggers

Terminal

```

on:
  # Push to specific branches
  push:
    branches:
      - main
      - 'release/**' # Wildcard pattern
    paths:
      - 'src/**'      # Only when src/ changes
      - '!*.md'       # Ignore markdown files

  # Pull request events
  pull_request:
    types: [opened, synchronize, reopened]
    branches: [main]

  # Scheduled runs (cron syntax)
  schedule:
    - cron: '0 2 * * *' # Daily at 2 AM UTC

  # Manual trigger
  workflow_dispatch:

```

Terminal -- continued

```
inputs:
  environment:
    description: 'Environment to deploy to'
    required: true
    default: 'staging'
    type: choice
    options:
      - staging
      - production

# Triggered by another workflow
workflow_call:
  inputs:
    version:
      required: true
      type: string

# Repository events
release:
  types: [published]

issues:
  types: [opened, labeled]
```

1.3.5 Job Dependencies and Parallelization

Terminal

```
jobs:
  # These run in parallel (no dependencies)
  lint:
    runs-on: ubuntu-latest
    steps: [...]

  security:
    runs-on: ubuntu-latest
    steps: [...]

# This waits for lint to complete
test:
  runs-on: ubuntu-latest
  needs: lint
  steps: [...]
```

Terminal -- continued

```
# This waits for both lint AND security
build:
  runs-on: ubuntu-latest
  needs: [lint, security]
  steps: [...]

# This waits for test AND build
deploy:
  runs-on: ubuntu-latest
  needs: [test, build]
  steps: [...]
```

Execution flow:

Terminal

```
+-----+ +-----+
| lint | | security |
+---+---+ +---+---+
|       | |
|       | |
+-----+ |
| test | |
+---+---+ |
|       | |
+-----+ +-----+
|       |
|
+-----+
| build |
+---+---+
|
+-----+
| deploy |
+-----+
```

1.3.6 Matrix Builds

Test across multiple versions and platforms:

```
Terminal

jobs:
  test:
    runs-on: ${ matrix.os }
    strategy:
      matrix:
        os: [ubuntu-latest, windows-latest, macos-latest]
        node-version: [18, 20, 22]
        exclude:
          # Don't test Node 18 on macOS
          - os: macos-latest
            node-version: 18
        include:
          # Add specific configuration
          - os: ubuntu-latest
            node-version: 20
            coverage: true
    fail-fast: false # Continue other jobs if one fails

  steps:
    - uses: actions/checkout@v4

    - name: Setup Node.js ${ matrix.node-version }
      uses: actions/setup-node@v4
      with:
        node-version: ${ matrix.node-version }

    - run: npm ci
    - run: npm test

    - name: Upload coverage
      if: matrix.coverage
      run: npm run coverage:upload
```

This creates 8 parallel jobs (3 OS × 3 Node versions - 1 exclusion).

1.3.7 Caching Dependencies

Speed up pipelines by caching dependencies:

```
Terminal

jobs:
  build:
    runs-on: ubuntu-latest
```

Terminal -- continued

```
steps:
  - uses: actions/checkout@v4

  # Automatic caching with setup-node
  - uses: actions/setup-node@v4
    with:
      node-version: '20'
      cache: 'npm' # Automatically caches node_modules

  - run: npm ci
  - run: npm run build

# Manual caching for more control
build-manual-cache:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

    - name: Cache node modules
      id: cache-npm
      uses: actions/cache@v4
      with:
        path: ~/.npm
        key: ${ runner.os }-node-${ hashFiles('**/package-lock.json') }
        restore-keys: |
          ${ runner.os }-node-

    - name: Cache build output
      uses: actions/cache@v4
      with:
        path: dist
        key: ${ runner.os }-build-${ hashFiles('src/**') }

    - run: npm ci
    - run: npm run build
```

1.3.8 Secrets and Environment Variables

```
jobs:
  deploy:
    runs-on: ubuntu-latest

    # Environment with protection rules
    environment:
      name: production
      url: https://example.com

    env:
      # Available to all steps in this job
      NODE_ENV: production

    steps:
      - uses: actions/checkout@v4

      - name: Deploy to production
        run: |
          echo "Deploying to $DEPLOY_URL"
          ./deploy.sh
        env:
          # Secrets from repository settings
          AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
          AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
          DEPLOY_URL: ${ vars.PRODUCTION_URL }

          # GitHub-provided variables
          GITHUB_SHA: ${ github.sha }
          GITHUB_REF: ${ github.ref }
```

Setting up secrets:

```
Repository Settings -> Secrets and variables -> Actions

Repository secrets:
+-- AWS_ACCESS_KEY_ID
+-- AWS_SECRET_ACCESS_KEY
+-- DATABASE_URL
+-- API_KEY
```

Terminal -- continued

```
Environment secrets (per environment):
+-- production
|   +-- DATABASE_URL (production database)
|   +-- API_KEY (production API key)
+-- staging
    +-- DATABASE_URL (staging database)
    +-- API_KEY (staging API key)
```

1.4

Continuous Deployment Strategies

Deploying to production requires careful strategies to minimize risk and enable quick rollbacks.

1.4.1 Deployment Strategies Overview

Terminal

```
+-----+
|          DEPLOYMENT STRATEGIES          |
+-----+
|
| RECREATE
| • Stop old version, start new version
| • Simple but causes downtime
| • Use for: Non-critical apps, major database migrations
|
| ROLLING
| • Gradually replace instances
| • Zero downtime
| • Use for: Most applications
|
| BLUE-GREEN
| • Two identical environments
| • Switch traffic instantly
| • Use for: Critical apps needing instant rollback
|
```

Terminal -- continued

```

|                                     |
| CANARY                             |
| • Deploy to small subset first     |
| • Gradually increase if healthy   |
| • Use for: Risk-averse deployments, A/B testing |
|                                     |
| FEATURE FLAGS                       |
| • Deploy code, enable features separately |
| • Instant enable/disable without deployment |
| • Use for: Trunk-based development, gradual rollouts |
|                                     |
+-----+

```

1.4.2 Recreate Deployment

The simplest strategy: stop everything, deploy, start everything.

Terminal

Before:

```

+-----+
| Load Balancer |
|               |
|   +-- Server 1: v1.0 |
|   +-- Server 2: v1.0 |
|   +-- Server 3: v1.0 |
+-----+

```

During deployment (DOWNTIME):

```

+-----+
| Load Balancer |
|               |
|   +-- Server 1: Updating... |
|   +-- Server 2: Updating... |
|   +-- Server 3: Updating... |
+-----+

```

After:

```

+-----+
| Load Balancer |
|               |
|   +-- Server 1: v2.0 |
|   +-- Server 2: v2.0 |
|   +-- Server 3: v2.0 |
+-----+

```


Terminal -- continued

+-----+

Pros:

- Simple to implement
- Clean state—no version mixing

Cons:

- Causes downtime
- All-or-nothing risk

1.4.3 Rolling Deployment

Update instances one at a time, maintaining availability:

```

Terminal

Step 1: Update Server 1
+-----+
| Load Balancer          |
| |                      |
|   +-- Server 1: v2.0   (updated) |
|   +-- Server 2: v1.0   |
|   +-- Server 3: v1.0   |
+-----+

Step 2: Update Server 2
+-----+
| Load Balancer          |
| |                      |
|   +-- Server 1: v2.0   |
|   +-- Server 2: v2.0   (updated) |
|   +-- Server 3: v1.0   |
+-----+

Step 3: Update Server 3
+-----+
| Load Balancer          |
| |                      |
|   +-- Server 1: v2.0   |
|   +-- Server 2: v2.0   |
|   +-- Server 3: v2.0   (updated) |
+-----+

```

Implementation (Kubernetes):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # Max extra pods during update
      maxUnavailable: 0 # Never reduce below desired count
  template:
    spec:
      containers:
        - name: myapp
          image: myapp:v2.0
```

Pros:

- Zero downtime
- Gradual rollout
- Easy to implement

Cons:

- Multiple versions running simultaneously
- Slower than recreate
- Rollback requires another rolling update

1.4.4 Blue-Green Deployment

Maintain two identical environments. Switch traffic instantly.

```
BLUE Environment (current production):
```

```
+-----+
|                                             |
| Server 1: v1.0                               |
| Server 2: v1.0      ---- 100% Traffic      |
| Server 3: v1.0                               |
|                                             |
```

Terminal -- continued

```

|
+-----+

GREEN Environment (staging new version):
+-----+
|
| Server 1: v2.0
| Server 2: v2.0      ---- 0% Traffic (testing)
| Server 3: v2.0
|
+-----+

After switch:
+-----+
|
| BLUE: v1.0          ---- 0% Traffic (standby for rollback)
|
| GREEN: v2.0         ---- 100% Traffic
|
+-----+

```

Implementation with AWS/Route 53:

Terminal

```

# GitHub Actions blue-green deployment
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to green environment
        run: |
          aws ecs update-service \
            --cluster production \
            --service myapp-green \
            --task-definition myapp:${{ github.sha }}

      - name: Wait for green to be healthy
        run: |
          aws ecs wait services-stable \
            --cluster production \
            --services myapp-green

      - name: Run smoke tests on green
        run: |

```

Terminal -- continued

```

curl -f https://green.example.com/health
npm run test:smoke -- --url=https://green.example.com

- name: Switch traffic to green
  run: |
    aws route53 change-resource-record-sets \
      --hosted-zone-id ${ secrets.HOSTED_ZONE_ID } \
      --change-batch file://switch-to-green.json

- name: Keep blue as rollback
  run: |
    echo "Blue environment available for rollback"
    echo "To rollback, switch DNS back to blue"

```

Pros:

- Instant switch and rollback
- Full testing before going live
- Zero downtime

Cons:

- Requires double infrastructure
- More expensive
- Database migrations are tricky

1.4.5 Canary Deployment

Deploy to a small percentage of users first, then gradually increase:

Terminal

Step 1: Deploy to 5% (canary)

```

+-----+
| Load Balancer |
| | |
| +-- 95% -- Production (v1.0): |
| | |
| +-- 5% -- Canary (v2.0): |
| | |
+-----+

```

Step 2: Monitor metrics. If healthy, increase to 25%

Terminal -- continued

```

+-----+
| Load Balancer |
| | |
| +-- 75% -- Production (v1.0): |
| | |
| +-- 25% -- Canary (v2.0): |
| | |
+-----+

Step 3: Continue to 50%, 75%, 100%

+-----+
| Load Balancer |
| | |
| +-- 100% -- New Production (v2.0): |
| | |
+-----+

```

Canary with Kubernetes and Istio:

Terminal

```

# VirtualService for traffic splitting
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: myapp
spec:
  hosts:
    - myapp.example.com
  http:
    - route:
        - destination:
            host: myapp-stable
            port:
              number: 80
          weight: 95
        - destination:
            host: myapp-canary
            port:
              number: 80
          weight: 5

```

Automated Canary Analysis:

```
Terminal

# GitHub Actions canary deployment
jobs:
  canary:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy canary (5%)
        run: kubectl apply -f canary-5-percent.yaml

      - name: Wait and analyze metrics
        run: |
          sleep 300 # Wait 5 minutes

          # Check error rate
          ERROR_RATE=$(curl -s "prometheus/api/v1/query?query=error_rate{version='canary'}")
          if [ "$ERROR_RATE" -gt "0.01" ]; then
            echo "Error rate too high, rolling back"
            kubectl apply -f rollback.yaml
            exit 1
          fi

          # Check latency
          LATENCY=$(curl -s "prometheus/api/v1/query?query=p99_latency{version='canary'}")
          if [ "$LATENCY" -gt "500" ]; then
            echo "Latency too high, rolling back"
            kubectl apply -f rollback.yaml
            exit 1
          fi

      - name: Increase to 25%
        run: kubectl apply -f canary-25-percent.yaml

      # ... continue pattern ...

      - name: Full rollout
        run: kubectl apply -f full-rollout.yaml
```

Pros:

- Minimal blast radius if issues
- Real production testing
- Data-driven promotion decisions

Cons:

- Complex to implement
- Requires good monitoring

- Multiple versions in production

1.4.6 Feature Flags

Deploy code to everyone but enable features selectively:

```

Terminal

// Feature flag implementation
const LaunchDarkly = require('launchdarkly-node-server-sdk');
const client = LaunchDarkly.init(process.env.LD_SDK_KEY);

app.get('/checkout', async (req, res) => {
  const user = { key: req.user.id, email: req.user.email };

  // Check if new checkout is enabled for this user
  const newCheckoutEnabled = await client.variation(
    'new-checkout-flow',
    user,
    false // Default value
  );

  if (newCheckoutEnabled) {
    return res.render('checkout-v2');
  } else {
    return res.render('checkout-v1');
  }
});

```

Feature flag strategies:

```

Terminal

+-----+
|               FEATURE FLAG STRATEGIES               |
+-----+
|
| BOOLEAN FLAG                                         |
| • Simple on/off                                     |
| • Example: dark_mode_enabled: true/false           |
|
| PERCENTAGE ROLLOUT                                  |
| • Gradually enable for more users                   |
| • Example: new_feature: 25% of users                |
|
| USER TARGETING                                       |

```

Terminal -- continued

```
| • Enable for specific users/groups |  
| • Example: beta_feature: [user_ids: 1, 2, 3] |  
|  
| ENVIRONMENT-BASED |  
| • Different values per environment |  
| • Example: debug_mode: true (dev), false (prod) |  
|  
| A/B TESTING |  
| • Different variants for different users |  
| • Example: checkout_button: "Buy Now" vs "Purchase" |  
|  
+-----+
```

Pros:

- Decouple deployment from release
- Instant enable/disable
- Enables A/B testing

Cons:

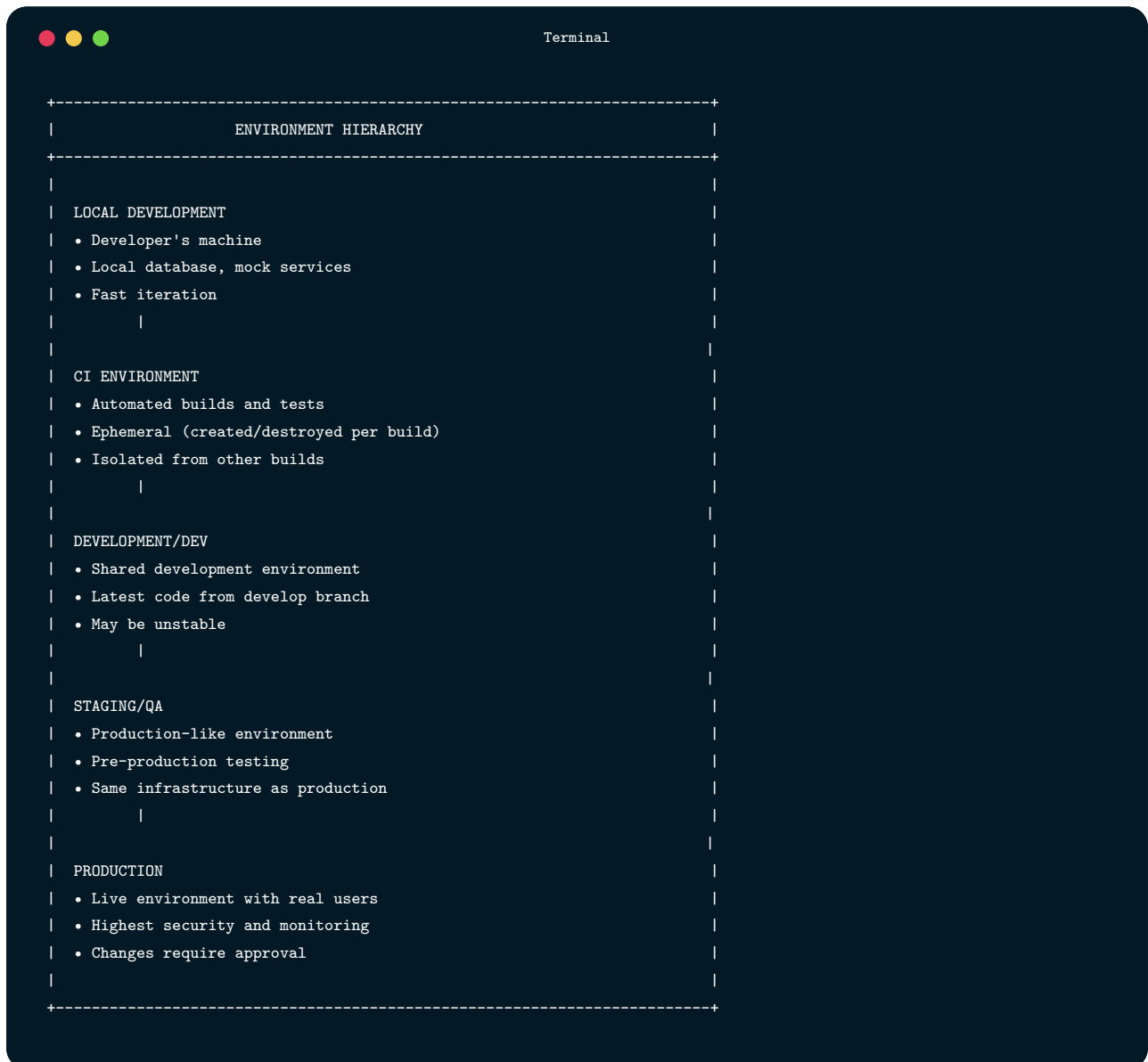
- Code complexity (if/else everywhere)
- Technical debt (old flags)
- Testing combinations is hard

1.5

Environment Management

Managing multiple environments (development, staging, production) is crucial for safe deployments.

1.5.1 Environment Hierarchy



1.5.2 Environment Configuration

Environment Variables:

```
Terminal

# .env.development
NODE_ENV=development
DATABASE_URL=postgresql://localhost:5432/app_dev
REDIS_URL=redis://localhost:6379
API_URL=http://localhost:3000
LOG_LEVEL=debug
DEBUG=true

# .env.staging
NODE_ENV=staging
DATABASE_URL=postgresql://staging-db.example.com:5432/app
REDIS_URL=redis://staging-redis.example.com:6379
API_URL=https://staging-api.example.com
LOG_LEVEL=info
DEBUG=false

# .env.production
NODE_ENV=production
DATABASE_URL=postgresql://prod-db.example.com:5432/app
REDIS_URL=redis://prod-redis.example.com:6379
API_URL=https://api.example.com
LOG_LEVEL=warn
DEBUG=false
```

Configuration Management:

```
Terminal

// config/index.js
const configs = {
  development: {
    database: {
      host: 'localhost',
      port: 5432,
      name: 'app_dev',
      pool: { min: 2, max: 10 }
    },
    cache: {
      ttl: 60, // Short TTL for dev
      enabled: false
    },
    features: {
      newCheckout: true, // Enable all features in dev
      darkMode: true
    }
  }
}
```

Terminal -- continued

```
    },

    staging: {
      database: {
        host: process.env.DB_HOST,
        port: 5432,
        name: 'app_staging',
        pool: { min: 5, max: 20 }
      },
      cache: {
        ttl: 300,
        enabled: true
      },
      features: {
        newCheckout: true,
        darkMode: true
      }
    },

    production: {
      database: {
        host: process.env.DB_HOST,
        port: 5432,
        name: 'app_prod',
        pool: { min: 10, max: 50 }
      },
      cache: {
        ttl: 3600,
        enabled: true
      },
      features: {
        newCheckout: false, // Gradually enable via feature flags
        darkMode: true
      }
    }
  }
};

const env = process.env.NODE_ENV || 'development';
module.exports = configs[env];
```

1.5.3 GitHub Actions Environments

```
Terminal

# .github/workflows/deploy.yml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  deploy-staging:
    runs-on: ubuntu-latest
    environment:
      name: staging
      url: https://staging.example.com

    steps:
      - uses: actions/checkout@v4

      - name: Deploy to staging
        run: ./deploy.sh staging
        env:
          AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
          AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }

  deploy-production:
    runs-on: ubuntu-latest
    needs: deploy-staging
    environment:
      name: production
      url: https://example.com

    steps:
      - uses: actions/checkout@v4

      - name: Deploy to production
        run: ./deploy.sh production
        env:
          AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
          AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
```

Environment Protection Rules:

 Terminal

Repository Settings -> Environments -> production

Protection Rules:

Required reviewers

- @team-leads

Wait timer

- 30 minutes after staging deploy

Restrict branches

- Only main branch can deploy

Custom deployment branch policy

- Selected branches: main, release/*

1.5.4 Secrets Management

Never commit secrets:

 Terminal

```
# .gitignore
.env
.env.*
!.env.example
*.pem
*.key
secrets/
```

Use environment-specific secrets:

 Terminal

```
# GitHub Actions
steps:
  - name: Deploy
    env:
      # Different secrets per environment
      DB_PASSWORD: ${ secrets.DB_PASSWORD } # Set per environment
      API_KEY: ${ secrets.API_KEY }
```

Secret rotation:

```
Terminal

# Scheduled secret rotation check
name: Secret Rotation Check

on:
  schedule:
    - cron: '0 9 * * 1' # Every Monday at 9 AM

jobs:
  check-secrets:
    runs-on: ubuntu-latest
    steps:
      - name: Check secret age
        run: |
          # Check if secrets are older than 90 days
          # Alert if rotation needed
          ./scripts/check-secret-age.sh

      - name: Send alert
        if: failure()
        uses: actions/github-script@v7
        with:
          script: |
            github.rest.issues.create({
              owner: context.repo.owner,
              repo: context.repo.repo,
              title: 'Secret rotation required',
              body: 'Secrets are older than 90 days and should be rotated.'
            })
```

1.6

Infrastructure as Code

Infrastructure as Code (IaC) treats infrastructure configuration as software—versioned, reviewed, and automated.

1.6.1 Why Infrastructure as Code?

Terminal		
+-----+-----+-----+		
MANUAL vs. INFRASTRUCTURE AS CODE		
+-----+-----+-----+		
MANUAL	INFRASTRUCTURE AS CODE	
+-----+-----+-----+		
Click through AWS console	Define in code files	
Document steps in wiki	Code IS the documentation	
"Works on my AWS account"	Reproducible anywhere	
Drift from documented state	Version controlled	
Slow to recreate	Fast to provision	
Hard to review changes	Pull request review	
Inconsistent environments	Identical environments	
Scary to modify	Confident changes	
+-----+-----+-----+		

1.6.2 Docker for Application Infrastructure

Dockerfile:

```

Terminal

# Build stage
FROM node:20-alpine AS builder

WORKDIR /app

# Copy package files first (better caching)
COPY package*.json ./
RUN npm ci

# Copy source and build
COPY . .
RUN npm run build

# Production stage
FROM node:20-alpine AS production

WORKDIR /app

# Create non-root user
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

```

Terminal -- continued

```
# Copy only production dependencies and build output
COPY --from=builder /app/package*.json ./
RUN npm ci --only=production

COPY --from=builder /app/dist ./dist

# Use non-root user
USER appuser

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1

EXPOSE 3000

CMD ["node", "dist/server.js"]
```

Docker Compose for local development:

Terminal

```
# docker-compose.yml
version: '3.8'

services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
      target: builder # Use builder stage for dev
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=development
      - DATABASE_URL=postgresql://postgres:postgres@db:5432/app_dev
      - REDIS_URL=redis://redis:6379
    volumes:
      - ./app
      - /app/node_modules # Don't override node_modules
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started
    command: npm run dev
```


Terminal -- continued

```
db:
  image: postgres:15
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: app_dev
  ports:
    - "5432:5432"
  volumes:
    - postgres_data:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U postgres"]
    interval: 5s
    timeout: 5s
    retries: 5

redis:
  image: redis:7-alpine
  ports:
    - "6379:6379"
  volumes:
    - redis_data:/data

# Test database for integration tests
db-test:
  image: postgres:15
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: app_test
  ports:
    - "5433:5432"

volumes:
  postgres_data:
  redis_data:
```

1.6.3 Terraform for Cloud Infrastructure

Basic Terraform structure:

```
Terminal

# main.tf
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

backend "s3" {
  bucket = "my-terraform-state"
  key     = "prod/terraform.tfstate"
  region = "us-east-1"
}

provider "aws" {
  region = var.aws_region
}

# VPC
resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true

  tags = {
    Name           = "${var.project_name}-vpc"
    Environment    = var.environment
  }
}

# Subnets
resource "aws_subnet" "public" {
  count          = 2
  vpc_id         = aws_vpc.main.id
  cidr_block     = "10.0.${count.index + 1}.0/24"
  availability_zone = data.aws_availability_zones.available.names[count.index]

  map_public_ip_on_launch = true

  tags = {
    Name = "${var.project_name}-public-${count.index + 1}"
  }
}

# RDS Database
resource "aws_db_instance" "main" {
```

Terminal -- continued

```

identifier      = "${var.project_name}-db"
engine          = "postgres"
engine_version  = "15"
instance_class  = var.db_instance_class
allocated_storage = 20

db_name = var.db_name
username = var.db_username
password = var.db_password

vpc_security_group_ids = [aws_security_group.db.id]
db_subnet_group_name    = aws_db_subnet_group.main.name

backup_retention_period = 7
skip_final_snapshot     = var.environment != "production"

tags = {
  Name      = "${var.project_name}-db"
  Environment = var.environment
}
}

# ECS Cluster
resource "aws_ecs_cluster" "main" {
  name = "${var.project_name}-cluster"

  setting {
    name = "containerInsights"
    value = "enabled"
  }
}

# ECS Service
resource "aws_ecs_service" "app" {
  name           = "${var.project_name}-service"
  cluster        = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.app.arn
  desired_count  = var.app_count
  launch_type    = "FARGATE"

  network_configuration {
    subnets      = aws_subnet.public[*].id
    security_groups = [aws_security_group.app.id]
  }

  load_balancer {
    target_group_arn = aws_lb_target_group.app.arn
  }
}

```

Terminal -- continued

```
    container_name = "app"
    container_port = 3000
  }
}
```

Variables:

Terminal

```
# variables.tf
variable "project_name" {
  description = "Name of the project"
  type       = string
  default    = "taskflow"
}

variable "environment" {
  description = "Deployment environment"
  type       = string
}

variable "aws_region" {
  description = "AWS region"
  type       = string
  default    = "us-east-1"
}

variable "db_instance_class" {
  description = "RDS instance class"
  type       = string
  default    = "db.t3.micro"
}

variable "app_count" {
  description = "Number of app instances"
  type       = number
  default    = 2
}
```

Environments with workspaces:

```
Terminal

# Create workspaces for each environment
terraform workspace new staging
terraform workspace new production

# Select workspace
terraform workspace select staging

# Apply with environment-specific variables
terraform apply -var-file="environments/staging.tfvars"
```

1.6.4 CI/CD for Infrastructure

```
Terminal

# .github/workflows/infrastructure.yml
name: Infrastructure

on:
  push:
    branches: [main]
    paths:
      - 'terraform/**'
  pull_request:
    branches: [main]
    paths:
      - 'terraform/**'

jobs:
  terraform-plan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.6.0

      - name: Terraform Init
        run: terraform init
        working-directory: terraform

      - name: Terraform Format Check
        run: terraform fmt -check
```

Terminal -- continued

```
    working-directory: terraform

  - name: Terraform Validate
    run: terraform validate
    working-directory: terraform

  - name: Terraform Plan
    run: terraform plan -out=tfplan
    working-directory: terraform
    env:
      AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
      AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }

  - name: Save plan
    uses: actions/upload-artifact@v4
    with:
      name: tfplan
      path: terraform/tfplan

terraform-apply:
  runs-on: ubuntu-latest
  needs: terraform-plan
  if: github.ref == 'refs/heads/main' && github.event_name == 'push'
  environment: production

  steps:
    - uses: actions/checkout@v4

    - name: Setup Terraform
      uses: hashicorp/setup-terraform@v3

    - name: Download plan
      uses: actions/download-artifact@v4
      with:
        name: tfplan
        path: terraform

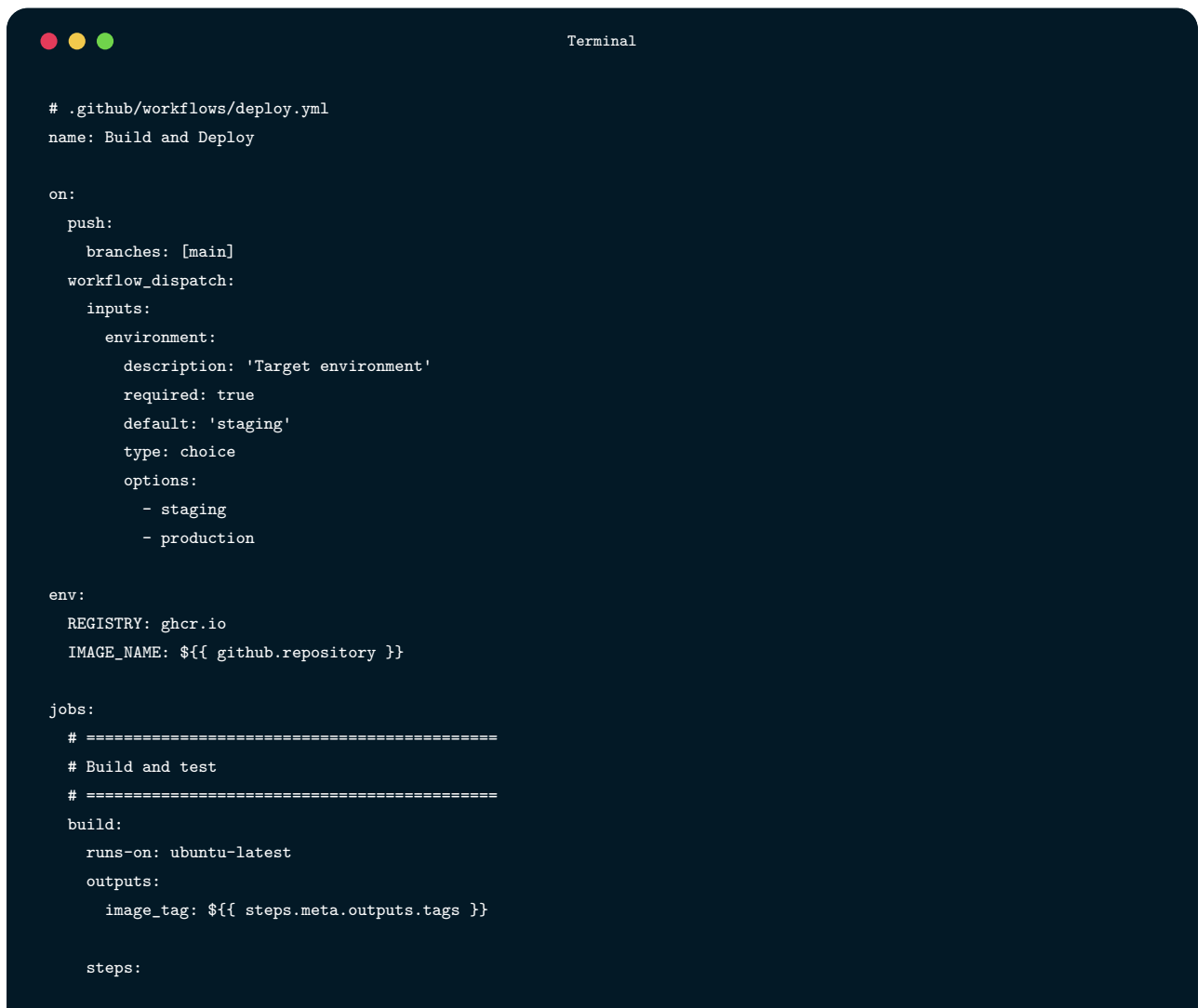
    - name: Terraform Init
      run: terraform init
      working-directory: terraform

    - name: Terraform Apply
      run: terraform apply -auto-approve tfplan
      working-directory: terraform
      env:
        AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
        AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
```

1.7

Deployment Automation

1.7.1 Complete Deployment Pipeline



```
# .github/workflows/deploy.yml
name: Build and Deploy

on:
  push:
    branches: [main]
  workflow_dispatch:
  inputs:
    environment:
      description: 'Target environment'
      required: true
      default: 'staging'
      type: choice
      options:
        - staging
        - production

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${ github.repository }

jobs:
  # =====
  # Build and test
  # =====
  build:
    runs-on: ubuntu-latest
    outputs:
      image_tag: ${ steps.meta.outputs.tags }

  steps:
```

Terminal -- continued

```
- name: Checkout
  uses: actions/checkout@v4

- name: Setup Node.js
  uses: actions/setup-node@v4
  with:
    node-version: '20'
    cache: 'npm'

- name: Install dependencies
  run: npm ci

- name: Run tests
  run: npm test

- name: Build application
  run: npm run build

- name: Log in to Container Registry
  uses: docker/login-action@v3
  with:
    registry: ${ env.REGISTRY }
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }

- name: Extract metadata
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: ${ env.REGISTRY }/${ env.IMAGE_NAME }
    tags: |
      type=sha,prefix=
      type=ref,event=branch

- name: Build and push Docker image
  uses: docker/build-push-action@v5
  with:
    context: .
    push: true
    tags: ${ steps.meta.outputs.tags }
    labels: ${ steps.meta.outputs.labels }
    cache-from: type=gha
    cache-to: type=gha,mode=max

# =====
# Deploy to staging
# =====
```


Terminal -- continued

```

deploy-staging:
  needs: build
  runs-on: ubuntu-latest
  environment:
    name: staging
    url: https://staging.example.com

  steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Configure AWS credentials
      uses: aws-actions/configure-aws-credentials@v4
      with:
        aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
        aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
        aws-region: us-east-1

    - name: Deploy to ECS
      run: |
        aws ecs update-service \
          --cluster staging-cluster \
          --service app-service \
          --force-new-deployment

    - name: Wait for deployment
      run: |
        aws ecs wait services-stable \
          --cluster staging-cluster \
          --services app-service

    - name: Run smoke tests
      run: |
        npm run test:smoke -- --url=https://staging.example.com

# =====
# Deploy to production (requires approval)
# =====
deploy-production:
  needs: deploy-staging
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  environment:
    name: production
    url: https://example.com

  steps:

```

Terminal -- continued

```

- name: Checkout
  uses: actions/checkout@v4

- name: Configure AWS credentials
  uses: aws-actions/configure-aws-credentials@v4
  with:
    aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
    aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
    aws-region: us-east-1

- name: Deploy to production (Blue-Green)
  run: |
    # Deploy to green environment
    aws ecs update-service \
      --cluster production-cluster \
      --service app-green \
      --force-new-deployment

    # Wait for green to be stable
    aws ecs wait services-stable \
      --cluster production-cluster \
      --services app-green

- name: Run production smoke tests
  run: |
    npm run test:smoke -- --url=https://green.example.com

- name: Switch traffic to green
  run: |
    aws elbv2 modify-listener \
      --listener-arn ${ secrets.LISTENER_ARN } \
      --default-actions Type=forward,TargetGroupArn=${ secrets.GREEN_TG_ARN }

- name: Verify production
  run: |
    sleep 30
    npm run test:smoke -- --url=https://example.com

- name: Create release
  uses: actions/github-script@v7
  with:
    script: |
      github.rest.repos.createRelease({
        owner: context.repo.owner,
        repo: context.repo.repo,
        tag_name: `v${new Date().toISOString().split('T')[0]}-${context.sha.substring(0, 7)}`,
        name: `Release ${new Date().toISOString().split('T')[0]}`,

```

Terminal -- continued

```
    body: `Deployed commit ${context.sha}`,  
    draft: false,  
    prerelease: false  
  })
```

1.7.2 Database Migrations in CI/CD

Terminal

```
# Database migration job  
migrate:  
  runs-on: ubuntu-latest  
  needs: build  
  environment: ${ github.event.inputs.environment || 'staging' }  
  
  steps:  
    - uses: actions/checkout@v4  
  
    - name: Setup Node.js  
      uses: actions/setup-node@v4  
      with:  
        node-version: '20'  
  
    - name: Install dependencies  
      run: npm ci  
  
    - name: Run migrations  
      run: npm run db:migrate  
      env:  
        DATABASE_URL: ${ secrets.DATABASE_URL }  
  
    - name: Verify migration  
      run: npm run db:verify  
      env:  
        DATABASE_URL: ${ secrets.DATABASE_URL }
```

Safe migration practices:

 Terminal

```
// migrations/20241209_add_user_role.js

// PASS Safe: Adding a column with default
exports.up = async (knex) => {
  await knex.schema.alterTable('users', (table) => {
    table.string('role').defaultTo('user');
  });
};

exports.down = async (knex) => {
  await knex.schema.alterTable('users', (table) => {
    table.dropColumn('role');
  });
};
```

 Terminal

```
// FAIL Dangerous: Renaming column (breaks running code)
// Instead, do it in phases:

// Phase 1: Add new column
exports.up_phase1 = async (knex) => {
  await knex.schema.alterTable('users', (table) => {
    table.string('full_name');
  });
  // Copy data
  await knex.raw('UPDATE users SET full_name = name');
};

// Phase 2: Deploy code that uses both columns
// Phase 3: Remove old column (after all code updated)
exports.up_phase3 = async (knex) => {
  await knex.schema.alterTable('users', (table) => {
    table.dropColumn('name');
  });
};
```

1.7.3 Rollback Procedures

```

# .github/workflows/rollback.yml
name: Rollback

on:
  workflow_dispatch:
    inputs:
      environment:
        description: 'Environment to rollback'
        required: true
        type: choice
        options:
          - staging
          - production
      version:
        description: 'Version to rollback to (leave empty for previous)'
        required: false

jobs:
  rollback:
    runs-on: ubuntu-latest
    environment: ${ github.event.inputs.environment }}

    steps:
      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v4
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: us-east-1

      - name: Get previous task definition
        id: previous
        run: |
          if [ -n "${ github.event.inputs.version }}" ]; then
            TASK_DEF="${ github.event.inputs.version }}"
          else
            # Get second most recent task definition
            TASK_DEF=$(aws ecs list-task-definitions \
              --family-prefix myapp \
              --sort DESC \
              --max-items 2 \
              --query 'taskDefinitionArns[1]' \
              --output text)
          fi
          echo "task_def=$TASK_DEF" >> $GITHUB_OUTPUT

```

Terminal -- continued

```

- name: Rollback ECS service
  run: |
    aws ecs update-service \
      --cluster ${github.event.inputs.environment}-cluster \
      --service app-service \
      --task-definition ${steps.previous.outputs.task_def}

- name: Wait for rollback
  run: |
    aws ecs wait services-stable \
      --cluster ${github.event.inputs.environment}-cluster \
      --services app-service

- name: Verify rollback
  run: |
    URL="https://${github.event.inputs.environment}.example.com"
    if [ "${github.event.inputs.environment}" = "production" ]; then
      URL="https://example.com"
    fi
    curl -f "$URL/health"

- name: Notify team
  uses: slackapi/slack-github-action@v1
  with:
    payload: |
      {
        "text": " Rollback completed for ${github.event.inputs.environment}",
        "blocks": [
          {
            "type": "section",
            "text": {
              "type": "mrkdwn",
              "text": "*Rollback completed*\n Environment: ${github.event.inputs.environment}\n Version: ${steps.previous.outputs.task_def}\n Triggered by: ${github.actor}"
            }
          }
        ]
      }
  env:
    SLACK_WEBHOOK_URL: ${secrets.SLACK_WEBHOOK_URL}

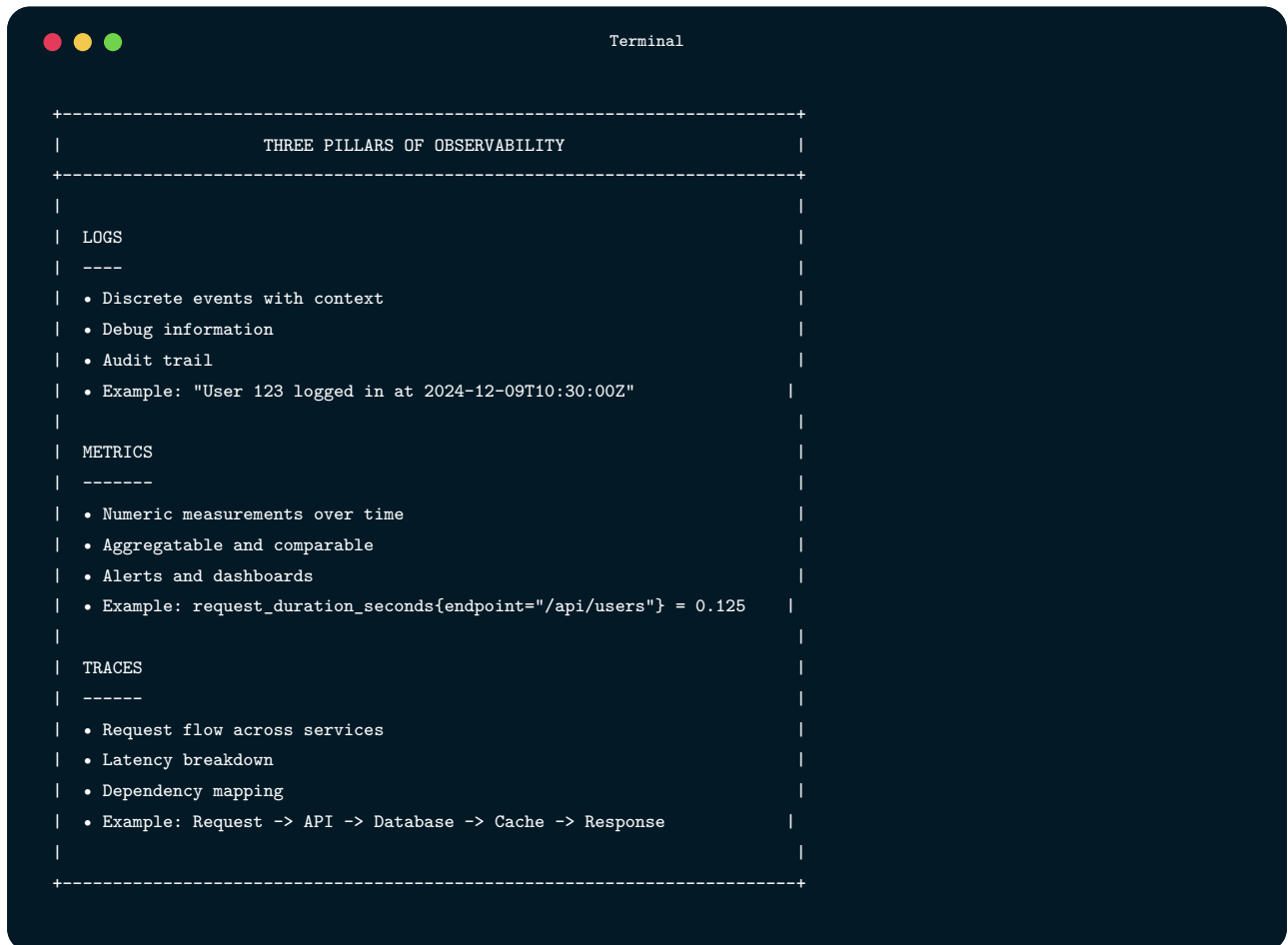
```

1.8

Monitoring and Observability

Deployment doesn't end when code reaches production. Monitoring ensures the deployment is healthy.

1.8.1 The Three Pillars of Observability



```
Terminal

+-----+
|               THREE PILLARS OF OBSERVABILITY               |
+-----+
|
| LOGS
| ----
| • Discrete events with context
| • Debug information
| • Audit trail
| • Example: "User 123 logged in at 2024-12-09T10:30:00Z"
|
| METRICS
| -----
| • Numeric measurements over time
| • Aggregatable and comparable
| • Alerts and dashboards
| • Example: request_duration_seconds{endpoint="/api/users"} = 0.125
|
| TRACES
| -----
| • Request flow across services
| • Latency breakdown
| • Dependency mapping
| • Example: Request -> API -> Database -> Cache -> Response
|
+-----+
```

1.8.2 Key Metrics to Monitor

```
Terminal

+-----+
|               KEY DEPLOYMENT METRICS               |
+-----+
|
| THE FOUR GOLDEN SIGNALS (Google SRE)
|
| 1. LATENCY
|   • Request duration
|   • p50, p95, p99 percentiles
|   • Alert: p99 > 500ms
|
| 2. TRAFFIC
|   • Requests per second
|   • Concurrent users
|   • Alert: Unusual spike or drop
|
| 3. ERRORS
|   • Error rate (5xx responses)
|   • Failed requests
|   • Alert: Error rate > 1%
|
| 4. SATURATION
|   • CPU utilization
|   • Memory usage
|   • Queue depth
|   • Alert: CPU > 80% for 5 minutes
|
+-----+
```

1.8.3 Health Checks

```
Terminal

// healthcheck.js
const express = require('express');
const db = require('./db');
const redis = require('./redis');

const router = express.Router();

// Basic liveness check (is the process running?)
```


Terminal -- continued

```

router.get('/health/live', (req, res) => {
  res.status(200).json({ status: 'ok' });
});

// Readiness check (is the app ready to serve traffic?)
router.get('/health/ready', async (req, res) => {
  const checks = {
    database: false,
    redis: false,
    memory: false
  };

  try {
    // Database check
    await db.raw('SELECT 1');
    checks.database = true;
  } catch (error) {
    console.error('Database health check failed:', error);
  }

  try {
    // Redis check
    await redis.ping();
    checks.redis = true;
  } catch (error) {
    console.error('Redis health check failed:', error);
  }

  // Memory check (under 90% usage)
  const memUsage = process.memoryUsage();
  const heapUsedPercent = memUsage.heapUsed / memUsage.heapTotal;
  checks.memory = heapUsedPercent < 0.9;

  const allHealthy = Object.values(checks).every(v => v);

  res.status(allHealthy ? 200 : 503).json({
    status: allHealthy ? 'healthy' : 'unhealthy',
    checks,
    timestamp: new Date().toISOString()
  });
});

// Detailed health for debugging
router.get('/health/details', async (req, res) => {
  res.json({
    version: process.env.APP_VERSION || 'unknown',
    commit: process.env.GIT_COMMIT || 'unknown',
  });
});

```

Terminal -- continued

```

    uptime: process.uptime(),
    memory: process.memoryUsage(),
    env: process.env.NODE_ENV,
    timestamp: new Date().toISOString()
  });
});

module.exports = router;

```

1.8.4 Post-Deployment Verification

Terminal

```

# Post-deployment verification in CI/CD
verify-deployment:
  runs-on: ubuntu-latest
  needs: deploy

  steps:
    - name: Wait for deployment to stabilize
      run: sleep 60

    - name: Check health endpoint
      run: |
        for i in {1..5}; do
          STATUS=$(curl -s -o /dev/null -w "%{http_code}" https://example.com/health/ready)
          if [ "$STATUS" = "200" ]; then
            echo "Health check passed"
            exit 0
          fi
          echo "Health check failed (attempt $i), waiting..."
          sleep 10
        done
        echo "Health check failed after 5 attempts"
        exit 1

    - name: Check error rate
      run: |
        # Query Prometheus/Datadog for error rate
        ERROR_RATE=$(curl -s "$PROMETHEUS_URL/api/v1/query?query=rate(http_requests_total{status!='5..'}[5m])")
        # Parse and check error rate
        # Alert if > 1%

    - name: Check response times

```

Terminal -- continued

```
run: |  
  # Run quick performance check  
  npm run test:performance -- --url=https://example.com --threshold=500ms  
  
- name: Rollback if unhealthy  
  if: failure()  
  run: |  
    echo "Deployment verification failed, initiating rollback"  
    gh workflow run rollback.yml -f environment=production
```

1.8.5 Alerting

```
Terminal

# Example Prometheus alerting rules
groups:
- name: deployment-alerts
  rules:
    - alert: HighErrorRate
      expr: rate(http_requests_total{status!="5.."}[5m]) / rate(http_requests_total[5m]) > 0.01
      for: 2m
      labels:
        severity: critical
      annotations:
        summary: High error rate detected
        description: Error rate is {{ $value | humanizePercentage }} over the last 5 minutes

    - alert: HighLatency
      expr: histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m])) > 0.5
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: High latency detected
        description: p99 latency is {{ $value }}s

    - alert: DeploymentFailed
      expr: kube_deployment_status_replicas_ready / kube_deployment_spec_replicas < 1
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: Deployment not fully ready
        description: Only {{ $value | humanizePercentage }} of pods are ready
```

1.9

Troubleshooting CI/CD

1.9.1 Common Issues and Solutions

```

Terminal

+-----+
|               COMMON CI/CD ISSUES               |
+-----+
|
| ISSUE: Build fails but works locally
| -----
| Causes:
| • Different Node/Python version
| • Missing environment variables
| • Cached dependencies out of sync
| • OS differences (Windows vs Linux)
|
| Solutions:
| • Match CI versions to local versions
| • Use .nvmrc or .python-version
| • Clear CI cache
| • Use Docker for consistency
|
| -----
|
| ISSUE: Flaky tests
| -----
| Causes:
| • Race conditions
| • Time-dependent tests
| • Shared test state
| • External dependencies
|
| Solutions:
| • Use proper async/await
| • Mock time-dependent code
| • Isolate test data
| • Mock external services
|
| -----
|
| ISSUE: Slow pipelines
| -----
| Causes:
| • No caching
| • Sequential jobs that could parallel
| • Large Docker images
| • Too many dependencies
|
| Solutions:

```

Terminal -- continued

```

| • Cache dependencies
| • Parallelize jobs
| • Use multi-stage Docker builds
| • Split test suites
|
| -----
|
| ISSUE: Deployment succeeds but app broken
| -----
|
| Causes:
| • Missing environment variables
| • Database migration issues
| • Incompatible dependencies
| • Configuration drift
|
| Solutions:
| • Comprehensive smoke tests
| • Health check endpoints
| • Staging environment that mirrors prod
| • Infrastructure as Code
|
+-----+

```

1.9.2 Debugging Techniques

Debugging GitHub Actions:

Terminal

```

jobs:
  debug:
    runs-on: ubuntu-latest
    steps:
      # Print all environment variables
      - name: Debug environment
        run: env | sort

      # Print GitHub context
      - name: Debug GitHub context
        run: echo '${{ toJson(github) }}'

      # Enable debug logging
      - name: Debug step
        run: echo "Debug info"

```

Terminal -- continued

```

env:
  ACTIONS_STEP_DEBUG: true

# SSH into runner for debugging
- name: Setup tmate session
  if: failure()
  uses: mxschmitt/action-tmate@v3
  timeout-minutes: 15

```

Debugging Docker builds:

Terminal

```

# Build with verbose output
docker build --progress=plain -t myapp .

# Build specific stage
docker build --target builder -t myapp:builder .

# Run intermediate layer
docker run -it myapp:builder sh

# Check image layers
docker history myapp

# Inspect image
docker inspect myapp

```

1.9.3 CI/CD Best Practices Checklist

Terminal

```

CI/CD BEST PRACTICES CHECKLIST
=====

CONTINUOUS INTEGRATION
Single source repository
Automated builds on every commit
Fast feedback (< 15 minutes)
Self-testing builds
Fix broken builds immediately
Keep the build green

```

Terminal -- continued

TESTING

- Unit tests with high coverage
- Integration tests for critical paths
- E2E tests for user journeys
- Tests run in CI
- No flaky tests

DEPLOYMENT

- Automated deployments
- Multiple environments (dev, staging, prod)
- Production-like staging
- Deployment approval for production
- Rollback procedure documented and tested

SECURITY

- Secrets in secret manager (not in code)
- Dependency scanning
- Security scanning in CI
- Least privilege for CI credentials
- Audit logging

MONITORING

- Health check endpoints
- Key metrics monitored
- Alerts configured
- Post-deployment verification
- Logging and tracing

DOCUMENTATION

- Pipeline documented
- Runbook for common issues
- Rollback procedure documented
- Environment configuration documented

1.10

Chapter Summary

Continuous Integration and Continuous Deployment transform how teams deliver software. By automating builds, tests, and deployments, teams can ship faster with higher quality and lower risk.

Key takeaways from this chapter:

- **Continuous Integration** means integrating code frequently, with automated builds and tests verifying each change. Problems are caught early when they're easiest to fix.
- **Continuous Delivery** ensures code is always in a deployable state, with push-button releases to production.
- **Continuous Deployment** goes further—every change that passes tests deploys automatically to production.
- **GitHub Actions** provides powerful CI/CD capabilities with workflows defined in YAML, jobs that run in parallel or sequence, and matrix builds for testing across configurations.
- **Deployment strategies** like rolling, blue-green, and canary deployments minimize risk and enable quick rollbacks.
- **Environment management** requires careful configuration of development, staging, and production environments with proper secrets management.
- **Infrastructure as Code** treats infrastructure like software—versioned, reviewed, and automated.
- **Monitoring and observability** are essential for knowing whether deployments are healthy. The three pillars—logs, metrics, and traces—provide visibility into system behavior.
- **Troubleshooting** CI/CD requires understanding common issues like environment differences, flaky tests, and slow pipelines.

1.11

Key Terms

Term	Definition
Continuous Integration (CI)	Practice of frequently integrating code with automated verification
Continuous Delivery	Keeping code always deployable with push-button releases
Continuous Deployment	Automatically deploying every change that passes tests
Pipeline	Automated sequence of build, test, and deploy stages
Workflow	GitHub Actions term for an automated process
Job	Unit of work in a CI/CD pipeline
Runner	Machine that executes CI/CD jobs
Artifact	File or package produced by a build
Blue-Green Deployment	Strategy using two identical environments for instant switching
Canary Deployment	Gradual rollout to small percentage of users
Rolling Deployment	Updating instances one at a time
Feature Flag	Toggle to enable/disable features without deployment
Infrastructure as Code	Managing infrastructure through version-controlled files
Health Check	Endpoint that reports application health
Rollback	Reverting to a previous version after failed deployment

1.12

Review Questions

1. Explain the difference between Continuous Integration, Continuous Delivery, and Continuous Deployment.
2. What are the core practices of Continuous Integration? Why is each important?
3. Describe the structure of a GitHub Actions workflow. What are workflows, jobs, and steps?
4. Compare blue-green, canary, and rolling deployment strategies. When would you use each?
5. Why is “fix broken builds immediately” a critical CI principle?
6. How do you securely manage secrets in CI/CD pipelines?
7. What is Infrastructure as Code? What problems does it solve?

8. Explain the purpose of staging environments. How should they relate to production?
 9. What are the four golden signals of monitoring? Why are they important for deployments?
 10. A deployment succeeds but users report errors. What steps would you take to diagnose and resolve the issue?
-

1.13

Hands-On Exercises

1.13.1 Exercise 9.1: Basic CI Pipeline

Create a CI pipeline for your project:

1. Create `.github/workflows/ci.yml`
2. Configure triggers for push and pull requests
3. Add jobs for:
 - Linting
 - Unit tests
 - Build
4. Verify the pipeline runs on a pull request
5. Add a README badge showing build status

1.13.2 Exercise 9.2: Matrix Testing

Extend your CI pipeline with matrix builds:

1. Test across multiple Node.js versions (18, 20, 22)
2. Test on multiple operating systems (ubuntu, windows)
3. Add a coverage job that only runs on one combination
4. Verify all combinations pass

1.13.3 Exercise 9.3: Automated Deployment

Set up automated deployment to a hosting platform:

1. Choose a platform (Vercel, Netlify, Render, or similar)
2. Create deployment workflow triggered by main branch
3. Add staging environment (deploy on all branches)
4. Add production environment with approval requirement
5. Document the deployment process

1.13.4 Exercise 9.4: Docker and CI

Containerize your application:

1. Create a Dockerfile for your application
2. Create docker-compose.yml for local development
3. Add Docker build and push to CI pipeline
4. Configure caching for faster builds
5. Test the container locally and in CI

1.13.5 Exercise 9.5: Health Checks and Monitoring

Implement health checks:

1. Add `/health/live` endpoint (basic liveness)
2. Add `/health/ready` endpoint (checks dependencies)
3. Add health check to Dockerfile
4. Configure CI to verify health after deployment
5. Document health check responses

1.13.6 Exercise 9.6: Rollback Procedure

Create and test a rollback procedure:

1. Create `rollback.yml` workflow
2. Accept environment and version as inputs
3. Implement rollback logic (revert to previous version)
4. Test rollback in staging environment
5. Document the rollback procedure

1.13.7 Exercise 9.7: Complete CI/CD Pipeline

Build a complete pipeline integrating all concepts:

1. Lint, test, and build on every commit
 2. Deploy to staging on develop branch
 3. Deploy to production on main with approval
 4. Include security scanning
 5. Post-deployment health verification
 6. Slack/Discord notification on deployment
 7. Document the entire pipeline
-

1.14

Further Reading

Books:

- Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.
- Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps Handbook*. IT Revolution Press.
- Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press.

Online Resources:

- GitHub Actions Documentation: <https://docs.github.com/en/actions>
- Docker Documentation: <https://docs.docker.com/>
- Terraform Documentation: <https://www.terraform.io/docs>
- Martin Fowler's CI/CD Articles: <https://martinfowler.com/articles/continuousIntegration.html>
- Google SRE Book: <https://sre.google/sre-book/table-of-contents/>

Tools:

- GitHub Actions: <https://github.com/features/actions>

- Docker: <https://www.docker.com/>
 - Terraform: <https://www.terraform.io/>
 - Kubernetes: <https://kubernetes.io/>
 - ArgoCD: <https://argoproj.github.io/cd/>
-

1.15

References

Fowler, M. (2006). Continuous Integration. Retrieved from <https://martinfowler.com/articles/continuousIntegration.html>

Humble, J., & Farley, D. (2010). *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley.

Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. IT Revolution Press.

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media.

GitHub. (2024). GitHub Actions Documentation. Retrieved from <https://docs.github.com/en/actions>

CHAPTER

02

DATA & APIs

Data Management and APIs

Designing durable data models and dependable application programming interfaces.

LEARNING OBJECTIVES

- ✓ By the end of this chapter, you will be able to:
 - ✓ Design relational database schemas using normalization principles
 - ✓ Write SQL queries for data manipulation and retrieval
 - ✓ Compare relational and NoSQL databases and choose appropriately for different use cases
 - ✓ Design RESTful APIs following industry best practices
 - ✓ Implement CRUD operations with proper HTTP methods and status codes
 - ✓ Create GraphQL schemas and resolvers for flexible data fetching
 - ✓ Document APIs using OpenAPI/Swagger specifications
 - ✓ Implement data validation and meaningful error handling
 - ✓ Apply caching strategies to improve API performance
 - ✓ Secure APIs with authentication and authorization

2.1

The Role of Data in Software Systems

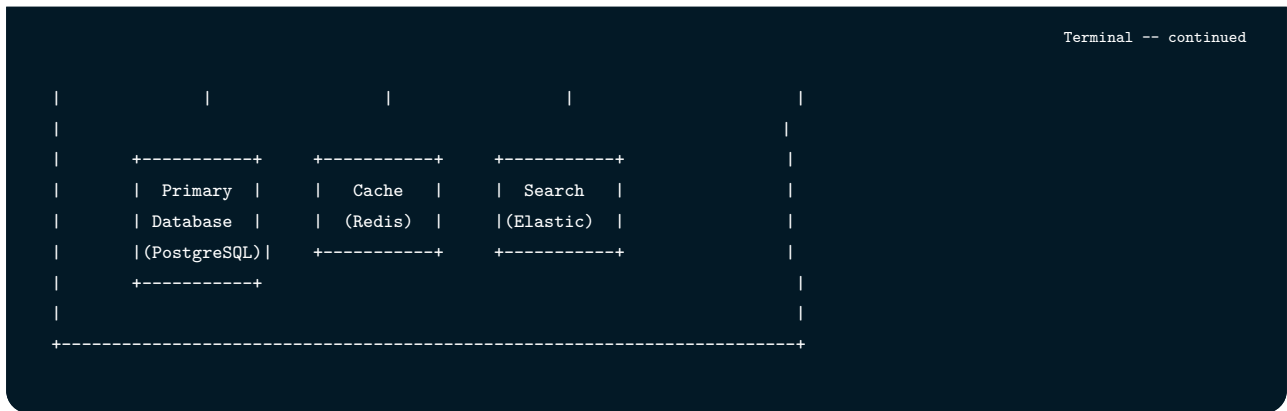
Data is the lifeblood of modern applications. Every action a user takes—creating an account, posting a message, completing a task—generates data that must be stored, organized, and retrieved efficiently. The decisions you make about data management ripple through every aspect of your application, affecting performance, scalability, maintainability, and user experience.

Consider a task management application like the one we’re building throughout this volume. When a user creates a task, that data must persist beyond the current session. When they log in from a different device, their tasks should appear. When they share a project with teammates, everyone needs to see the same information. These seemingly simple requirements demand careful thought about how data flows through your system.

2.1.1 Data Architecture Overview

Before diving into specific technologies, it’s important to understand how data typically moves through a modern application. The architecture below represents a common pattern you’ll encounter in production systems:





In this architecture, clients (web browsers, mobile apps) never communicate directly with databases. Instead, they interact with an API layer that serves as a gatekeeper. This separation provides several benefits: the API can validate requests before they reach the database, enforce business rules, handle authentication, and present a consistent interface regardless of how data is stored internally.

The primary database (often PostgreSQL, MySQL, or a similar relational database) serves as the authoritative source of truth. This is where your critical business data lives—user accounts, orders, transactions, and other information that must be accurate and durable.

Auxiliary data stores serve specialized purposes. A cache like Redis stores frequently-accessed data in memory for lightning-fast retrieval. A search engine like Elasticsearch provides sophisticated full-text search capabilities that would be slow or impossible with a traditional database. Many production systems use multiple data stores, each optimized for specific access patterns—a strategy called “polyglot persistence.”

2.1.2 Key Data Management Concerns

As you design your data layer, keep these fundamental concerns in mind:

Data Integrity ensures that data is accurate, consistent, and trustworthy. When a user transfers money between accounts, the total balance must remain constant—you can’t create or destroy money through a software bug. Database constraints, validations, and transactions protect integrity by enforcing rules about what data can exist and how it can change.

Data Security protects sensitive information from unauthorized access. User passwords must be hashed, not stored in plain text. Personal information must be encrypted. Access must be controlled so users can only see and modify their own data. Security isn’t an afterthought—it must be designed into your data layer from the beginning.

Data Availability ensures that data is accessible when needed. If your database server crashes, can users still access the application? Replication (maintaining copies on multiple servers), backups

(regular snapshots for disaster recovery), and failover mechanisms (automatic switching to backup systems) ensure availability.

Data Scalability means handling growing data volumes and request rates without degrading performance. When your application grows from 100 users to 100,000 users, your data layer must scale accordingly. Indexing (creating data structures for fast lookups), sharding (distributing data across multiple servers), and caching (storing frequently-accessed data in fast memory) enable scalability.

Data Consistency keeps data synchronized across systems. If a user updates their profile, that change should be visible everywhere immediately—or if not immediately, then eventually and predictably. Different applications have different consistency requirements, and understanding these trade-offs is essential for making good architectural decisions.

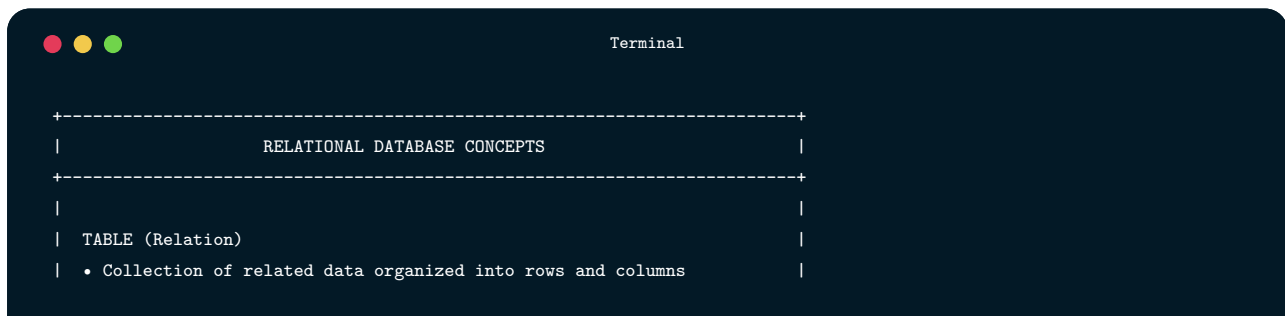
2.2

Relational Databases

Relational databases have been the foundation of data management since Edgar Codd introduced the relational model in 1970. They organize data into tables with rows and columns, using relationships to connect related data. Despite the rise of NoSQL alternatives, relational databases remain the most common choice for applications with structured data and complex querying needs.

2.2.1 Core Concepts

To work effectively with relational databases, you need to understand several foundational concepts. Let's explore each one:



```
Terminal

+-----+
|          RELATIONAL DATABASE CONCEPTS          |
+-----+
|
| TABLE (Relation)
| • Collection of related data organized into rows and columns
|
```

Terminal -- continued

```

| • Has a defined schema specifying column names and types |
| • Similar to a spreadsheet, but with strict typing |
| |
| COLUMN (Attribute) |
| • Single piece of data with a specific meaning |
| • Has a name (like "email") and data type (like VARCHAR) |
| • May have constraints (NOT NULL, UNIQUE, CHECK) |
| |
| ROW (Record/Tuple) |
| • Single instance representing one entity |
| • Contains values for each column defined in the table |
| • Each row should be uniquely identifiable |
| |
| PRIMARY KEY |
| • Column (or columns) that uniquely identifies each row |
| • Cannot contain NULL values |
| • Most commonly an auto-incrementing integer ID |
| |
| FOREIGN KEY |
| • Column that references the primary key of another table |
| • Creates relationships between tables |
| • Enforces referential integrity (can't reference non-existent data) |
| |
| INDEX |
| • Data structure that speeds up data retrieval |
| • Like a book's -index helps find information quickly |
| • Trade-off: faster reads but slower writes (index must be updated) |
| |
+-----+

```

A **Table** is the fundamental unit of data organization. Think of it as a spreadsheet where each column has a specific data type. A users table might have columns for `id` (integer), `email` (text), `name` (text), and `created_at` (timestamp). Unlike spreadsheets, databases enforce these types strictly—you can’t accidentally put a name in the email column.

Primary Keys solve the problem of identity. How do you refer to a specific user? Names aren’t unique (there might be multiple “John Smith” users), and emails can change. Instead, we assign each row a unique identifier—typically an auto-incrementing integer. This ID becomes the row’s permanent address within the database.

Foreign Keys create connections between tables. If each task belongs to a user, the tasks table includes a `user_id` column containing the ID of the user who owns that task. This creates a relationship: you can find all tasks belonging to a user, or find the user who owns a particular task.

Let’s visualize these concepts with a concrete example from our task management application:



This diagram shows three tables and their relationships. The arrows indicate foreign key references—the direction points from the referencing table to the referenced table. Notice that:

- Each task has a `user_id` pointing to a user (the task’s assignee)
- Each task optionally has a `project_id` pointing to a project
- Each project has an `owner_id` pointing to a user (the project owner)

These relationships create a web of connected data. A single query can traverse these connections to answer complex questions like “Show me all tasks in projects owned by users who joined this year.”

2.2.2 SQL Fundamentals

Structured Query Language (SQL) is the standard language for interacting with relational databases. Despite being over 50 years old, SQL remains essential because it provides a powerful, declarative way to describe what data you want without specifying how to retrieve it. The database engine optimizes query execution automatically.

SQL divides into two main categories: Data Definition Language (DDL) for creating and modifying database structure, and Data Manipulation Language (DML) for working with the data itself.

Data Definition Language (DDL)

DDL statements define the structure of your database—creating tables, adding columns, establishing constraints. These statements typically run during application setup or migrations, not during normal operation.

Let's create the tables for our task management application. We'll start with the users table:

```
CREATE TABLE users (  
  id SERIAL PRIMARY KEY,  
  email VARCHAR(255) UNIQUE NOT NULL,  
  name VARCHAR(100) NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

This statement creates a table with six columns. Let's understand each element:

- `SERIAL PRIMARY KEY` creates an auto-incrementing integer that uniquely identifies each row. PostgreSQL automatically assigns values (1, 2, 3, ...) when you insert new rows.
- `VARCHAR(255)` means variable-length text up to 255 characters. We use this for emails and names.
- `UNIQUE` ensures no two users can have the same email address. The database rejects insertions that would create duplicates.
- `NOT NULL` means this column must have a value—you can't create a user without an email.
- `DEFAULT CURRENT_TIMESTAMP` automatically sets the creation time when a row is inserted.

Now let's create the projects table, which references users:

```
CREATE TABLE projects (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  description TEXT,  
  owner_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

The `REFERENCES users(id)` clause creates a foreign key relationship. This tells the database that `owner_id` must contain a valid user ID—you can't create a project owned by a non-existent user. The `ON DELETE`

CASCADE clause specifies what happens when the referenced user is deleted: all their projects are automatically deleted too. This maintains referential integrity—you'll never have orphaned projects pointing to deleted users.

Finally, the tasks table with multiple relationships and constraints:

```
CREATE TABLE tasks (  
  id SERIAL PRIMARY KEY,  
  title VARCHAR(200) NOT NULL,  
  description TEXT,  
  user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  project_id INTEGER REFERENCES projects(id) ON DELETE SET NULL,  
  status VARCHAR(20) DEFAULT 'todo'  
    CHECK (status IN ('todo', 'in_progress', 'review', 'done')),  
  priority INTEGER DEFAULT 0 CHECK (priority BETWEEN 0 AND 4),  
  due_date DATE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

This table introduces several new concepts. The CHECK constraint validates data before insertion—status must be one of the four allowed values, and priority must be between 0 and 4. The database enforces these rules automatically, preventing invalid data from entering your system.

Notice that `project_id` has a different deletion behavior: `ON DELETE SET NULL`. If a project is deleted, tasks aren't deleted—instead, their `project_id` becomes NULL, indicating they're no longer associated with any project. This design decision reflects business logic: deleting a project shouldn't destroy the work recorded in its tasks.

After creating tables, we typically add indexes to speed up common queries:

```
CREATE INDEX idx_tasks_user_id ON tasks(user_id);  
CREATE INDEX idx_tasks_project_id ON tasks(project_id);  
CREATE INDEX idx_tasks_status ON tasks(status);  
CREATE INDEX idx_tasks_due_date ON tasks(due_date);
```

Each index creates a data structure (typically a B-tree) that allows the database to find rows quickly without scanning the entire table. The `idx_tasks_user_id` index makes queries like “find all tasks for user 123” fast, even with millions of tasks.

However, indexes aren't free. Each index consumes storage space and must be updated whenever data changes. Too many indexes slow down insertions and updates. The general rule: index columns that appear frequently in WHERE clauses or JOIN conditions.

Data Manipulation Language (DML)

DML statements work with the data itself: inserting new records, querying existing records, updating values, and deleting rows. These are the statements your application executes during normal operation.

Inserting Data

The INSERT statement adds new rows to a table:

```
INSERT INTO users (email, name, password_hash)
VALUES ('alice@example.com', 'Alice Johnson', '$2b$10$...');
```

Notice that we don't specify `id`, `created_at`, or `updated_at`—these columns have defaults. The database automatically assigns the next available ID and current timestamp.

You can insert multiple rows in a single statement, which is more efficient than separate INSERT statements:

```
INSERT INTO tasks (title, description, user_id, project_id, status, priority, due_date)
VALUES
  ('Design homepage mockup', 'Create wireframes and mockups', 1, 1, 'in_progress', 2, '2024-12-15'),
  ('Implement navigation', 'Build responsive nav component', 1, 1, 'todo', 1, '2024-12-20'),
  ('Write content', 'Draft copy for main pages', 1, 1, 'todo', 1, '2024-12-18');
```

This single statement creates three tasks atomically—either all three are created or none are. This atomicity becomes important when you need to ensure data consistency.

Querying Data

The SELECT statement retrieves data from tables. It's the most commonly used SQL statement and offers tremendous flexibility.

The simplest query retrieves all columns from a table:

```
SELECT * FROM users;
```

In practice, you should specify the columns you need rather than using *. This makes your code clearer and can improve performance:

```
SELECT id, name, email FROM users;
```

The WHERE clause filters results to rows matching specific conditions:

```
SELECT * FROM tasks WHERE status = 'todo';
```

You can combine multiple conditions with AND and OR:

```
SELECT * FROM tasks
WHERE status = 'in_progress'
  AND priority >= 2
  AND due_date < '2024-12-31';
```

This query finds high-priority tasks that are in progress and due before year's end. The database evaluates all conditions and returns only rows that satisfy all of them.

Sorting with ORDER BY arranges results in a specific order:

```
SELECT * FROM tasks ORDER BY due_date ASC, priority DESC;
```

This sorts tasks by due date (earliest first), and for tasks with the same due date, by priority (highest first). The ASC and DESC keywords specify ascending or descending order.

For large result sets, LIMIT and OFFSET enable pagination:


```
SELECT * FROM tasks ORDER BY id LIMIT 10 OFFSET 20;
```

This retrieves tasks 21-30 (skip 20, take 10). Combined with ORDER BY, this pattern enables “page 3 of results” functionality in your application.

Updating Data

The UPDATE statement modifies existing rows:

```
UPDATE tasks
SET status = 'done', updated_at = CURRENT_TIMESTAMP
WHERE id = 1;
```

The WHERE clause is critical—without it, UPDATE affects every row in the table. Always include WHERE unless you intentionally want to update all rows.

You can use expressions in updates:

```
UPDATE tasks
SET priority = priority + 1
WHERE due_date < CURRENT_DATE AND status != 'done';
```

This increases the priority of all overdue, incomplete tasks. The database evaluates `priority + 1` for each matching row.

Deleting Data

The DELETE statement removes rows:

```
DELETE FROM tasks WHERE id = 1;
```

Like UPDATE, DELETE without WHERE removes all rows—a dangerous operation. Most applications prefer “soft deletes” (marking rows as deleted rather than actually removing them) to preserve data and enable recovery.

```
DELETE FROM tasks
WHERE status = 'done'
  AND updated_at < NOW() - INTERVAL '30 days';
```

This cleanup query removes completed tasks older than 30 days. The `INTERVAL` syntax is PostgreSQL-specific; other databases have different date arithmetic syntax.

Joining Tables

The real power of relational databases emerges when you combine data from multiple tables. `JOIN` operations connect rows based on related columns.

An `INNER JOIN` returns only rows that have matches in both tables:

```
SELECT
  t.id,
  t.title,
  t.status,
  u.name AS assignee,
  p.name AS project_name
FROM tasks t
INNER JOIN users u ON t.user_id = u.id
INNER JOIN projects p ON t.project_id = p.id;
```

Let’s break down this query. We’re selecting from the `tasks` table (aliased as `t` for brevity). The first `JOIN` connects tasks to users: for each task, find the user whose ID matches the task’s `user_id`. The second `JOIN` connects to projects similarly.

The `AS` keyword creates column aliases—the results will show “assignee” and “project_name” rather than ambiguous “name” columns.

Important: `INNER JOIN` excludes tasks that don’t have a matching project (where `project_id` is `NULL`). If you want to include those tasks, use `LEFT JOIN` instead:

```
SELECT
  u.name,
  COUNT(t.id) AS task_count
```

Terminal -- continued

```
FROM users u
LEFT JOIN tasks t ON u.id = t.user_id
GROUP BY u.id, u.name;
```

A LEFT JOIN returns all rows from the left table (users) regardless of whether they have matching rows in the right table (tasks). Users with no tasks appear in results with NULL values for task columns. This query counts how many tasks each user has—including users with zero tasks.

The GROUP BY clause is essential here. Without it, the query would fail because we're mixing aggregate (COUNT) and non-aggregate (name) columns. GROUP BY tells the database to compute one result row per user.

Here's a more complex example that generates project statistics:

Terminal

```
SELECT
  p.name AS project,
  COUNT(CASE WHEN t.status = 'done' THEN 1 END) AS completed,
  COUNT(CASE WHEN t.status != 'done' THEN 1 END) AS remaining,
  COUNT(t.id) AS total
FROM projects p
LEFT JOIN tasks t ON p.id = t.project_id
GROUP BY p.id, p.name
ORDER BY total DESC;
```

This query demonstrates conditional aggregation using CASE expressions. For each project, we count tasks in different categories. The CASE expression returns 1 when the condition is true and NULL otherwise; COUNT ignores NULLs, so we effectively count only matching rows.

Aggregations and Subqueries

Aggregate functions compute values across multiple rows. The most common aggregates are:

- COUNT(*) - number of rows
- SUM(column) - total of values
- AVG(column) - average value
- MIN(column) and MAX(column) - smallest and largest values

```
SELECT
  COUNT(*) AS total_tasks,
  COUNT(CASE WHEN status = 'done' THEN 1 END) AS completed_tasks,
  AVG(estimated_hours) AS avg_estimate,
  SUM(estimated_hours) AS total_hours
FROM tasks;
```

This query computes overall statistics for all tasks. Note that AVG ignores NULL values—if some tasks don't have estimated_hours, they're excluded from the average calculation.

GROUP BY creates separate aggregations for each group:

```
SELECT
  status,
  COUNT(*) AS count,
  AVG(priority) AS avg_priority
FROM tasks
GROUP BY status;
```

This produces one row per status value, showing how many tasks are in each status and their average priority.

The HAVING clause filters groups (as opposed to WHERE, which filters individual rows):

```
SELECT
  user_id,
  COUNT(*) AS task_count
FROM tasks
GROUP BY user_id
HAVING COUNT(*) > 5;
```

This finds users with more than 5 tasks. The filtering happens after grouping—you can't use aggregate functions in WHERE clauses.

Subqueries embed one query inside another:

```

SELECT * FROM tasks
WHERE user_id IN (
    SELECT id FROM users WHERE email LIKE '%@company.com'
);

```

The inner query finds all user IDs with company email addresses. The outer query then finds all tasks belonging to those users. This is equivalent to a JOIN but sometimes reads more naturally.

Common Table Expressions (CTEs) provide a cleaner way to write complex queries:

```

WITH task_stats AS (
    SELECT
        user_id,
        COUNT(*) AS total_tasks,
        COUNT(CASE WHEN status = 'done' THEN 1 END) AS completed_tasks
    FROM tasks
    GROUP BY user_id
)
SELECT
    u.name,
    ts.total_tasks,
    ts.completed_tasks,
    ROUND(ts.completed_tasks::DECIMAL / NULLIF(ts.total_tasks, 0) * 100, 1) AS completion_rate
FROM users u
JOIN task_stats ts ON u.id = ts.user_id
ORDER BY completion_rate DESC;

```

The WITH clause defines a temporary result set (task_stats) that we can reference in the main query. This improves readability for complex queries and can sometimes improve performance by allowing the database to materialize intermediate results.

The NULLIF(ts.total_tasks, 0) prevents division by zero—if a user has zero tasks, the expression returns NULL instead of causing an error.

2.2.3 Database Normalization

Normalization is the process of organizing data to minimize redundancy and dependency. Redundant data wastes storage and, more importantly, creates opportunities for inconsistency. If a customer's

address is stored in multiple places, updating it requires changing multiple records—miss one, and your data becomes inconsistent.

Normalization follows a series of “normal forms,” each building on the previous:

```

Terminal

+-----+
|              NORMALIZATION FORMS              |
+-----+
|
| FIRST NORMAL FORM (1NF)                        |
| • Each column contains atomic (indivisible) values |
| • No repeating groups or arrays within a single column |
| • Each row is unique (has a primary key)         |
|
| SECOND NORMAL FORM (2NF)                       |
| • Meets all 1NF requirements                     |
| • All non-key columns depend on the entire primary key |
| • No partial dependencies (relevant for composite keys) |
|
| THIRD NORMAL FORM (3NF)                        |
| • Meets all 2NF requirements                     |
| • No transitive dependencies                     |
| • Non-key columns depend only on the primary key, not on each other |
|
+-----+

```

Let’s see normalization in action. Imagine we start with this denormalized orders table:

```

Terminal

BEFORE (Denormalized - Violates 1NF, 2NF, 3NF):
+-----+
| orders                                     |
+-----+
| order_id | customer | customer_email | products | product_prices |
+-----+
| 1        | Alice    | alice@email.com | A, B, C | 10, 20, 30     |
| 2        | Alice    | alice@email.com | B, D    | 20, 40         |
| 3        | Bob      | bob@email.com   | A       | 10             |
+-----+

```

This structure has multiple problems:

1NF Violation: The products column contains multiple values (“A, B, C”). This makes it impossible to query efficiently—how do you find all orders containing product B? You’d need string manipulation, which is slow and error-prone.

2NF Violation: Customer information (name, email) is repeated for every order. If Alice changes her email, you must update multiple rows. Miss one, and her email is inconsistent across orders.

3NF Violation: Customer email depends on customer name, not on order_id. This is a “transitive dependency”—email depends on customer, which depends on order. Such dependencies cause update anomalies.

The normalized design separates concerns into distinct tables:

Terminal

AFTER (Normalized to 3NF):

customers			products		
id (PK)	email		id (PK)	name	price
1	alice@...		1	A	10
2	bob@...		2	B	20
			3	C	30
			4	D	40

orders			order_items (junction table)		
id (PK)	customer_id		order_id	product_id	quantity
1	1		1	1	1
2	1		1	2	1
3	2		1	3	1
			2	2	1
			2	4	1
			3	1	1

Now each piece of information is stored exactly once. Customer data lives in the customers table. Product data lives in the products table. Orders reference customers by ID. The order_items “junction table” connects orders to products, enabling a many-to-many relationship (an order can contain many products; a product can appear in many orders).

To recreate the original denormalized view, we join the tables:

```

SELECT
  o.id AS order_id,
  c.name AS customer,
  c.email,
  STRING_AGG(p.name, ', ') AS products,
  SUM(oi.price_at_time * oi.quantity) AS total
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
GROUP BY o.id, c.name, c.email;

```

This query joins all four tables to produce a result similar to our original denormalized structure. The `STRING_AGG` function concatenates product names into a comma-separated list.

Notice the `price_at_time` column in `order_items`. This is a deliberate denormalization—we store the price at the time of purchase rather than referencing the products table. Why? Product prices change over time, but an order’s total should remain constant. This is an example of intentional denormalization for business requirements.

2.2.4 Transactions and ACID Properties

Real-world operations often require multiple database changes that must succeed or fail together. Consider transferring money between bank accounts: you must debit one account AND credit another. If the system crashes between these operations, money would vanish or appear from nowhere.

Transactions solve this problem by grouping operations into atomic units. The database guarantees that either all operations in a transaction complete successfully, or none of them do.

```

+-----+
|          ACID PROPERTIES          |
+-----+
|
|  ATOMICITY                        |
|  All operations complete successfully, or none do. There's no partial |
|  -stateyou can't have "half a transaction." If anything fails, all   |
|  changes are rolled back as if the transaction never happened.        |
|
|  CONSISTENCY                      |
|  A transaction brings the database from one valid state to another.    |

```


Terminal -- continued

```

| All constraints and rules remain satisfied. You can't end up with      |
| invalid data, even if the transaction is interrupted.                |
|                                                                       |
| ISOLATION                                                            |
| Concurrent transactions don't interfere with each other. Each       |
| transaction sees a consistent snapshot of the database. Two users    |
| booking the last airplane seat can't both succeed.                  |
|                                                                       |
| DURABILITY                                                            |
| Once a transaction commits, it stays committed even if the server    |
| crashes immediately afterward. The database writes to stable storage |
| before confirming the commit.                                         |
|                                                                       |
+-----+

```

Here's a transaction that transfers money between accounts:

Terminal

```

BEGIN TRANSACTION;

UPDATE accounts
SET balance = balance - 100
WHERE id = 1 AND balance >= 100;

UPDATE accounts
SET balance = balance + 100
WHERE id = 2;

INSERT INTO transfers (from_account, to_account, amount, created_at)
VALUES (1, 2, 100, CURRENT_TIMESTAMP);

COMMIT;

```

The `BEGIN TRANSACTION` statement starts a new transaction. All subsequent operations are part of this transaction. The `COMMIT` statement finalizes the transaction, making all changes permanent. If anything goes wrong before `COMMIT`, you can issue `ROLLBACK` to undo all changes.

The `WHERE` clause `balance >= 100` is crucial—it prevents overdrafts. If the account doesn't have sufficient funds, no rows are updated, and your application code should check this and roll back the transaction.

In application code, transaction management typically looks like this:

```

Terminal

async function transferFunds(fromAccountId, toAccountId, amount) {
  // Start a transaction
  const trx = await db.transaction();

  try {
    // Attempt to debit source account
    // The WHERE clause ensures sufficient balance
    const debited = await trx('accounts')
      .where('id', fromAccountId)
      .where('balance', '>=', amount)
      .decrement('balance', amount);

    // Check if debit succeeded (row was updated)
    if (debited === 0) {
      throw new Error('Insufficient funds');
    }

    // Credit destination account
    await trx('accounts')
      .where('id', toAccountId)
      .increment('balance', amount);

    // Record the transfer for audit trail
    await trx('transfers').insert({
      from_account: fromAccountId,
      to_account: toAccountId,
      amount,
      created_at: new Date()
    });

    // All operations -succeededcommit the transaction
    await trx.commit();

    return { success: true };
  } catch (error) {
    // Something went -wrongrollback all changes
    await trx.rollback();
    throw error;
  }
}

```

This code demonstrates several important patterns. First, we create a transaction object (`trx`) and use it for all database operations. This ensures all operations are part of the same transaction. Second, we check the result of the debit operation—if no rows were updated (insufficient funds), we throw an

error. Third, we wrap everything in try/catch to ensure we roll back on any error. Finally, we only commit after all operations succeed.

The beauty of transactions is that you don't need to write cleanup code for each failure scenario. No matter where the error occurs—after the debit but before the credit, or after recording the transfer—the rollback undoes everything atomically.

2.3

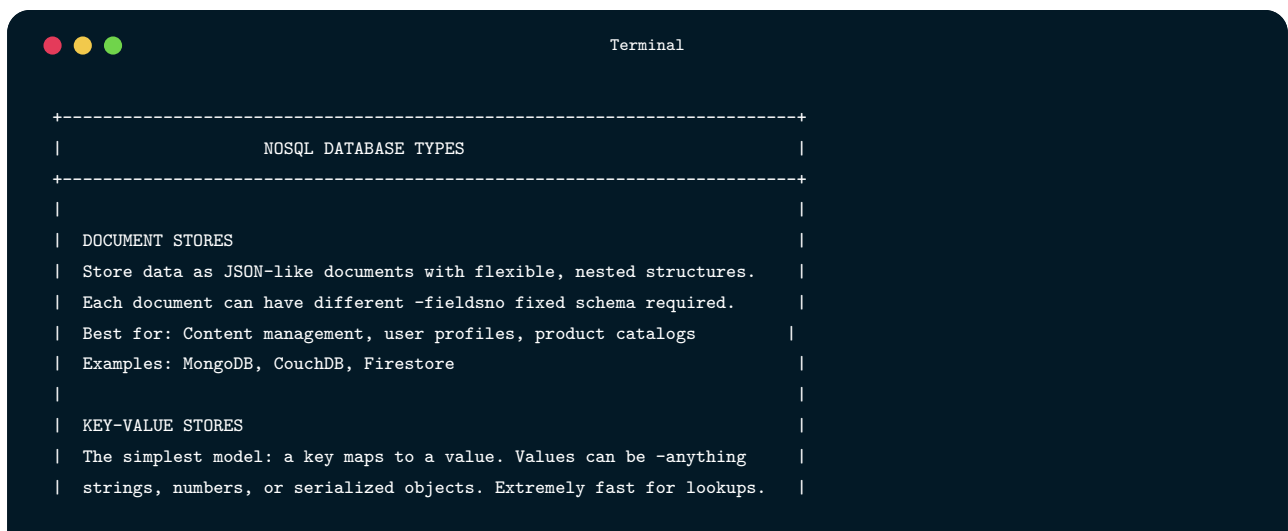
NoSQL Databases

While relational databases excel at structured data with complex relationships, they're not the best tool for every situation. **NoSQL databases** emerged to address specific use cases where relational databases struggle: massive scale, flexible schemas, high write throughput, or specialized data models.

The term “NoSQL” originally meant “No SQL,” but it's now commonly understood as “Not Only SQL”—acknowledging that these databases complement rather than replace relational databases.

2.3.1 Types of NoSQL Databases

NoSQL databases fall into four main categories, each optimized for different use cases:



```

Terminal
+-----+
|               NOSQL DATABASE TYPES               |
+-----+
|
| DOCUMENT STORES                                  |
| Store data as JSON-like documents with flexible, nested structures. |
| Each document can have different -fieldsno fixed schema required.   |
| Best for: Content management, user profiles, product catalogs      |
| Examples: MongoDB, CouchDB, Firestore                               |
|
| KEY-VALUE STORES                                  |
| The simplest model: a key maps to a value. Values can be -anything  |
| strings, numbers, or serialized objects. Extremely fast for lookups. |
  
```

Terminal -- continued

```

| Best for: Caching, sessions, feature flags, real-time data |
| Examples: Redis, Memcached, Amazon DynamoDB |
|
| COLUMN-FAMILY STORES |
| Store data in columns rather than rows, enabling efficient queries |
| over specific columns across billions of rows. |
| Best for: Analytics, time-series data, IoT sensor data |
| Examples: Apache Cassandra, HBase, ScyllaDB |
|
| GRAPH DATABASES |
| Store entities (nodes) and relationships (edges) as first-class |
| citizens. Optimized for traversing connections between data. |
| Best for: Social networks, recommendations, fraud detection |
| Examples: Neo4j, Amazon Neptune, ArangoDB |
|
+-----+

```

2.3.2 Document Databases (MongoDB)

Document databases store data as self-contained documents, typically in JSON or a binary JSON variant (BSON). Unlike relational tables with fixed columns, documents can have varying structures. This flexibility makes document databases excellent for evolving schemas and hierarchical data.

Here's how a user might be represented in MongoDB:

Terminal

```

{
  "_id": ObjectId("507f1f77bcf86cd799439011"),
  "email": "alice@example.com",
  "name": "Alice Johnson",
  "profile": {
    "avatar": "https://example.com/avatars/alice.jpg",
    "bio": "Software developer",
    "location": "San Francisco"
  },
  "tasks": [
    {
      "title": "Complete project proposal",
      "status": "in_progress",
      "priority": 2,
      "tags": ["work", "urgent"],
      "due_date": ISODate("2024-12-15")
    }
  ],
}

```

```
Terminal -- continued

{
  "title": "Review pull requests",
  "status": "todo",
  "priority": 1,
  "tags": ["work"],
  "due_date": ISODate("2024-12-10")
},
{
  "settings": {
    "theme": "dark",
    "notifications": true,
    "language": "en"
  },
  "created_at": ISODate("2024-01-15"),
  "updated_at": ISODate("2024-12-09")
}
```

Notice several differences from relational design. First, the document contains nested objects (`profile`, `settings`) that would require separate tables in a relational database. Second, the `tasks` array embeds related data directly within the user document. Third, different users could have different fields—one might have a `company` field that others lack.

This embedding pattern eliminates JOINS for common access patterns. If you typically fetch a user with their tasks, having everything in one document means a single database round-trip instead of multiple queries.

Let's see how to work with MongoDB in Node.js:

```
Terminal

const { MongoClient } = require('mongodb');

// Connect to MongoDB
const client = new MongoClient(process.env.MONGODB_URI);
const db = client.db('taskflow');
```

Creating documents uses the `insertOne` or `insertMany` methods:

```

Terminal

// Insert a single user
await db.collection('users').insertOne({
  email: 'bob@example.com',
  name: 'Bob Smith',
  tasks: [],
  created_at: new Date()
});

// Insert multiple documents at once
await db.collection('users').insertMany([
  { email: 'carol@example.com', name: 'Carol White', tasks: [] },
  { email: 'dave@example.com', name: 'Dave Brown', tasks: [] }
]);

```

MongoDB automatically generates unique `_id` fields if you don't provide them. These ObjectIds include a timestamp, making them roughly sortable by creation time.

Querying documents offers flexible filtering:

```

Terminal

// Find a single document by field value
const user = await db.collection('users')
  .findOne({ email: 'alice@example.com' });

// Find documents with embedded array matching
// This finds users who have at least one in_progress task
const busyUsers = await db.collection('users')
  .find({ 'tasks.status': 'in_progress' })
  .toArray();

// Project specific fields (like SQL SELECT)
// Only returns name and email, not the entire document
const userNames = await db.collection('users')
  .find({}, { projection: { name: 1, email: 1 } })
  .toArray();

```

The dot notation (`tasks.status`) allows querying nested fields and array elements. This is powerful but requires understanding how MongoDB handles array queries—`'tasks.status': 'in_progress'` finds documents where ANY task has that status.

Updating documents can modify the entire document or specific fields:

```
Terminal

// Update a single field using $set
// Other fields remain unchanged
await db.collection('users').updateOne(
  { email: 'alice@example.com' },
  { $set: { 'profile.bio': 'Senior developer' } }
);

// Add an element to an array using $push
await db.collection('users').updateOne(
  { email: 'alice@example.com' },
  {
    $push: {
      tasks: {
        title: 'New task',
        status: 'todo',
        created_at: new Date()
      }
    }
  }
);

// Update a specific array element using $ positional operator
// This updates the status of the task with title "Complete project proposal"
await db.collection('users').updateOne(
  {
    email: 'alice@example.com',
    'tasks.title': 'Complete project proposal'
  },
  { $set: { 'tasks.$.status': 'done' } }
);
```

The `$` positional operator is crucial for updating array elements. It refers to the first array element that matched the query condition. Without it, you'd need to know the exact array index.

MongoDB's update operators (`$set`, `$push`, `$pull`, `$inc`, etc.) enable atomic modifications without reading and rewriting entire documents. This is both more efficient and safer for concurrent updates.

2.3.3 Key-Value Stores (Redis)

Redis is an in-memory data store that excels at speed. Because data lives in RAM rather than on disk, Redis can handle millions of operations per second with sub-millisecond latency. This makes it ideal for caching, session storage, and real-time features.

The trade-off is capacity—RAM is more expensive than disk storage, so Redis typically holds a subset of your data: frequently accessed items, temporary data, or data that needs extremely fast access.

Let's explore Redis's versatile data structures:

```
Terminal

const Redis = require('ioredis');
const redis = new Redis(process.env.REDIS_URL);
```

Basic key-value operations are the foundation:

```
Terminal

// Store a simple value
await redis.set('user:1:name', 'Alice');

// Retrieve a value
const name = await redis.get('user:1:name'); // 'Alice'

// Set with automatic expiration (TTL)
// This key will disappear after 1 hour
await redis.set('session:abc123', JSON.stringify({ userId: 1 }), 'EX', 3600);

// Check remaining time-to-live
const ttl = await redis.ttl('session:abc123'); // seconds remaining
```

The key naming convention (`user:1:name`) is a Redis best practice. Using colons to create hierarchical namespaces makes keys self-documenting and easier to manage.

Expiration (TTL) is essential for cache management. Without it, cached data would accumulate forever. By setting appropriate TTLs, stale data automatically disappears.

Hashes store object-like structures efficiently:

```
Terminal

// Store multiple fields at once
await redis.hset('user:1', {
  name: 'Alice',
  email: 'alice@example.com',
  role: 'admin'
});

// Retrieve all fields
```


Terminal -- continued

```
const user = await redis.hgetall('user:1');
// { name: 'Alice', email: 'alice@example.com', role: 'admin' }

// Retrieve single field
const email = await redis.hget('user:1', 'email');

// Increment a numeric field atomically
await redis.hincrby('user:1', 'login_count', 1);
```

Hashes are more memory-efficient than storing each field as a separate key. They also enable atomic operations on individual fields without reading/writing the entire object.

Lists provide ordered collections:

Terminal

```
// Push to the front of a list (newest first)
await redis.lpush('notifications:1', 'New message');
await redis.lpush('notifications:1', 'Task assigned');

// Get a range of elements (0 to -1 means all)
const notifications = await redis.lrange('notifications:1', 0, -1);
// ['Task assigned', 'New message']

// Pop from the list (remove and return)
const latest = await redis.lpop('notifications:1');

// Trim to keep only recent items
await redis.ltrim('notifications:1', 0, 99); // Keep only 100 newest
```

Lists are perfect for activity feeds, queues, and recent items. The `lpush/rpop` combination creates a queue (FIFO), while `lpush/lpop` creates a stack (LIFO).

Sets store unique values:

Terminal

```
// Add members (duplicates are ignored)
await redis.sadd('user:1:tags', 'developer', 'team-lead', 'remote');
await redis.sadd('user:1:tags', 'developer'); // No -effectalready exists

// Get all members
const tags = await redis.smembers('user:1:tags');
```

Terminal -- continued

```
// ['developer', 'team-lead', 'remote']

// Check membership
const isRemote = await redis.sismember('user:1:tags', 'remote'); // 1 (true)

// Set operations
const user1Tags = await redis.smembers('user:1:tags');
const user2Tags = await redis.smembers('user:2:tags');
const commonTags = await redis.sinter('user:1:tags', 'user:2:tags'); // Intersection
```

Sets are useful for tags, unique visitors, online users, and any scenario where you need fast membership testing without duplicates.

Sorted sets add scoring for automatic ordering:

Terminal

```
// Add members with scores
await redis.zadd('leaderboard', 100, 'alice', 85, 'bob', 92, 'carol');

// Get top performers (highest scores first)
const topThree = await redis.zrevrange('leaderboard', 0, 2, 'WITHSCORES');
// ['alice', '100', 'carol', '92', 'bob', '85']

// Get rank (position) of a member
const aliceRank = await redis.zrevrank('leaderboard', 'alice'); // 0 (first place)

// Increment a score
await redis.zincrby('leaderboard', 10, 'bob'); // Bob now has 95 points
```

Sorted sets are perfect for leaderboards, priority queues, and time-based indexes (using timestamps as scores).

Implementing a caching layer is one of Redis's most common uses:

Terminal

```
async function getUserWithCache(userId) {
  const cacheKey = `user:${userId}`;

  // Step 1: Check the cache
  const cached = await redis.get(cacheKey);
  if (cached) {
```

```

Terminal -- continued

    console.log('Cache hit!');
    return JSON.parse(cached);
  }

  // Step 2: Cache -missfetch from primary database
  console.log('Cache -missfetching from database');
  const user = await db('users').where('id', userId).first();

  // Step 3: Store in cache for future requests
  if (user) {
    await redis.set(cacheKey, JSON.stringify(user), 'EX', 300); // 5 minutes
  }

  return user;
}

```

This “cache-aside” pattern checks the cache first, falls back to the database on miss, and populates the cache for future requests. The 5-minute TTL balances freshness against database load.

Cache invalidation is equally important—when data changes, the cache must be updated or cleared:

```

Terminal

async function updateUser(userId, updates) {
  // Update the primary database
  await db('users').where('id', userId).update(updates);

  // Invalidate the cache (delete, don't update)
  // Next read will fetch fresh data from database
  await redis.del(`user:${userId}`);
}

```

Deleting rather than updating the cache is usually safer. It ensures the next read gets fresh data from the authoritative source (the database), avoiding any possibility of the cache and database becoming inconsistent.

2.3.4 Choosing Between SQL and NoSQL

The choice between SQL and NoSQL isn’t about which is “better”—it’s about which fits your specific requirements. Here’s a framework for making this decision:



```

+-----+
|          SQL vs. NOSQL COMPARISON          |
+-----+
| RELATIONAL (SQL) | NOSQL |
+-----+
| Fixed schema      | Flexible schema |
| ACID transactions | Eventual consistency (usually) |
| Complex queries (JOINS) | Simple queries, denormalized data |
| Vertical scaling  | Horizontal scaling |
| Mature tooling    | Newer, varied tooling |
| Strong consistency | High availability |
+-----+

```

Choose a relational database when:

- Data has clear relationships that benefit from JOINS
- ACID compliance is required (financial transactions, inventory)
- You need complex queries and ad-hoc reporting
- Schema is well-defined and relatively stable
- Data integrity is paramount

Examples: Banking systems, e-commerce orders, ERP systems, traditional web applications

Choose NoSQL when:

- Schema evolves frequently or varies between records
- Massive scale is required (millions of writes per second)
- Simple access patterns (key-based lookup, document retrieval)
- Data is naturally hierarchical or document-shaped
- High availability is more important than consistency

Examples: Content management systems, real-time analytics, IoT sensor data, user session storage

Many production systems use both (polyglot persistence):

- PostgreSQL for core business data (users, orders, payments)
- Redis for caching and sessions
- Elasticsearch for full-text search
- MongoDB for flexible content or audit logs

This approach uses each database for what it does best. The complexity cost is worthwhile when different data has genuinely different requirements.

2.4

RESTful API Design

With data stored in databases, we need a way for applications to access it. **REST (Representational State Transfer)** is an architectural style that has become the dominant approach for web APIs. REST APIs use HTTP methods to perform operations on resources, providing a uniform, stateless interface.

2.4.1 REST Principles

REST is built on several key principles that guide API design:

A terminal window with a dark blue background and white text. The title bar at the top says "Terminal" and has three colored window control buttons (red, yellow, green) on the left. The content of the terminal is a list of REST principles, each preceded by a vertical bar and a dashed line above and below the list. The principles are: CLIENT-SERVER SEPARATION, STATELESSNESS, CACHEABILITY, UNIFORM INTERFACE, and LAYERED SYSTEM. Each principle is followed by a brief description of its meaning and how it applies to API design.

```
+-----+
|               REST PRINCIPLES               |
+-----+
|
| CLIENT-SERVER SEPARATION                      |
| Clients and servers evolve independently. The server doesn't know or |
| care whether it's serving a web app, mobile app, or CLI tool.       |
|
| STATELESSNESS                                |
| Each request contains all information needed to process it. The     |
| server doesn't remember previous requests. This simplifies -scaling |
| any server can handle any request.
|
| CACHEABILITY                                  |
| Responses indicate whether they can be cached. This improves
| performance and reduces server load.
|
| UNIFORM INTERFACE                            |
| All resources are accessed through a consistent interface: URIs
| identify resources, HTTP methods perform operations, standard
| formats (JSON) represent data.
|
| LAYERED SYSTEM                              |
| Clients can't tell whether they're connected directly to the server |
|
```

Terminal -- continued

```
| or through intermediaries (load balancers, caches, gateways). |
| | |
+-----+-----+
```

The **statelessness** principle deserves emphasis. In a REST API, the server doesn't maintain session state between requests. If a user is logged in, every request must include authentication information (typically a token). This seems redundant, but it enables horizontal scaling—since any server can handle any request, you can add servers to handle more load without worrying about session affinity.

2.4.2 Resource-Oriented Design

In REST, everything is a **resource**—a conceptual entity that can be identified, retrieved, and manipulated. Resources are identified by URIs (Uniform Resource Identifiers) and typically represent nouns in your domain: users, tasks, projects, comments.

Good URI design follows consistent conventions:

Terminal

```
+-----+-----+
| RESOURCE NAMING CONVENTIONS |
+-----+-----+
|
| USE NOUNS, NOT VERBS |
| Resources are things, not actions. |
| PASS /users           FAIL /getUsers |
| PASS /tasks           FAIL /createTask |
| PASS /projects/123/tasks FAIL /getTasksForProject |
|
| USE PLURAL NOUNS |
| Consistency makes the API predictable. |
| PASS /users       FAIL /user |
| PASS /users/123   FAIL /user/123 |
|
| USE HIERARCHY FOR RELATIONSHIPS |
| Nested resources show ownership or containment. |
| PASS /projects/123/tasks (tasks belonging to project 123) |
| PASS /users/456/notifications (notifications for user 456) |
|
| USE LOWERCASE AND HYPHENS |
| URIs are case-sensitive; consistency prevents errors. |
| PASS /task-comments      FAIL /taskComments |
| PASS /user-profiles      FAIL /UserProfiles |
```

Terminal -- continued

For our task management API, here's how resources map to URIs:

```

/users           Collection of all users
/users/123       Single user with ID 123
/users/123/tasks Tasks assigned to user 123
/users/123/projects Projects owned by user 123

/projects        Collection of all projects
/projects/456     Single project with ID 456
/projects/456/tasks Tasks within project 456
/projects/456/members Members of project 456

/tasks           Collection of all tasks
/tasks/789       Single task with ID 789
/tasks/789/comments Comments on task 789
/tasks/789/attachments Files attached to task 789

```

Notice how the hierarchy expresses relationships. `/projects/456/tasks` and `/users/123/tasks` might return different (or overlapping) sets of tasks, filtered by project or assignee respectively.

2.4.3 HTTP Methods and CRUD Operations

REST uses HTTP methods to indicate what operation to perform on a resource. Each method has specific semantics that clients and servers agree on:

```

+-----+
| HTTP METHODS AND OPERATIONS |
+-----+
| Method | Operation | Description |
+-----+
| GET    | Read     | Retrieve resource(s). -Safe doesn't modify |
|        |          | data. Cacheable. |
+-----+
| POST   | Create   | Create a new resource. The server assigns |

```

Terminal -- continued

		the ID and returns the created resource.	
PUT	Replace	Replace an entire resource. Client provides all fields; missing fields are cleared.	
PATCH	Update	Partial update. Client provides only the fields to change.	
DELETE	Delete	Remove a resource. Often returns empty response (204 No Content).	

Two important properties distinguish these methods:

Safety: GET requests are “safe”—they only retrieve data without side effects. You can call GET as many times as you want without changing anything. This allows aggressive caching and prefetching.

Idempotency: GET, PUT, and DELETE are idempotent—calling them multiple times has the same effect as calling once. If you PUT the same data twice, the resource ends up in the same state. POST is NOT idempotent—each POST typically creates a new resource.

Here’s how these methods apply to our tasks resource:

Terminal

```

GET    /api/tasks           List all tasks (with pagination)
GET    /api/tasks/123      Get task with ID 123
POST   /api/tasks       Create a new task
PUT    /api/tasks/123   Replace task 123 entirely
PATCH /api/tasks/123   Update specific fields of task 123
DELETE /api/tasks/123   Delete task 123

GET    /api/tasks?status=todo      Filter tasks by status
GET    /api/tasks?page=2&limit=20  Paginate results
GET    /api/tasks?sort=due_date&order=asc  Sort results

```

Query parameters modify GET requests—filtering, pagination, and sorting change which resources are returned and in what order, but they don’t create or modify resources.

2.4.4 HTTP Status Codes

Status codes communicate the result of an API request. Using appropriate codes makes your API self-documenting and helps clients handle responses correctly:



A terminal window with a dark blue background and white text. The title bar says "Terminal". Inside, a table of HTTP status codes is displayed, enclosed in a dashed border. The table is organized into sections: SUCCESS (2xx), CLIENT ERRORS (4xx), and SERVER ERRORS (5xx).

ESSENTIAL STATUS CODES		
SUCCESS (2xx)		
200 OK	Request succeeded. Body contains result.	
201 Created	Resource created. Body contains new resource.	
204 No Content	Success, but no body (common for DELETE).	
CLIENT ERRORS (4xx) - Problem with the request		
400 Bad Request	Malformed request (invalid JSON, wrong format).	
401 Unauthorized	Authentication required or failed.	
403 Forbidden	Authenticated, but not authorized for this.	
404 Not Found	Resource doesn't exist.	
409 Conflict	Request conflicts with current state.	
422 Unprocessable	Valid request, but validation failed.	
429 Too Many	Rate limit exceeded.	
SERVER ERRORS (5xx) - Problem on the server		
500 Internal Error	Unexpected server error (bug, crash).	
502 Bad Gateway	Invalid response from upstream server.	
503 Unavailable	Server temporarily overloaded or down.	

The distinction between 401 and 403 often confuses developers:

- **401 Unauthorized:** “I don’t know who you are. Please authenticate.”
- **403 Forbidden:** “I know who you are, but you can’t do this.”

Similarly, 400 vs 422:

- **400 Bad Request:** The request is malformed (can’t parse JSON, missing required header).
- **422 Unprocessable Entity:** The request is valid, but the data fails validation (email format wrong, title too long).

2.4.5 Implementing a RESTful API

Let's build a complete REST API for tasks using Express.js. We'll structure the code into layers: routes (HTTP handling), services (business logic), and a clean separation of concerns.

First, the route handlers that define our endpoints:

```
Terminal

// routes/tasks.js
const express = require('express');
const router = express.Router();
const { authenticate } = require('../middleware/auth');
const { validate } = require('../middleware/validate');
const taskSchema = require('../schemas/task');
const taskService = require('../services/taskService');
```

This file begins by importing dependencies. We separate concerns: authentication middleware verifies the user, validation middleware checks request data, and the service layer handles business logic. This structure makes each piece testable and replaceable.

The list endpoint returns paginated, filtered tasks:

```
Terminal

// GET /api/tasks - List tasks with filtering and pagination
router.get('/', authenticate, async (req, res, next) => {
  try {
    // Extract query parameters with defaults
    const {
      page = 1,
      limit = 20,
      status,
      priority,
      sort = 'created_at',
      order = 'desc'
    } = req.query;

    // Call service layer with parsed parameters
    const result = await taskService.list({
      userId: req.user.id,           // From auth middleware
      page: parseInt(page),          // Convert string to number
      limit: Math.min(parseInt(limit), 100), // Cap at 100 to prevent abuse
      filters: { status, priority },
      sort,
      order
    });
```

```

Terminal -- continued

});

// Return data with pagination metadata
res.json({
  data: result.tasks,
  pagination: {
    page: result.page,
    limit: result.limit,
    total: result.total,
    totalPages: Math.ceil(result.total / result.limit)
  }
});
} catch (error) {
  next(error); // Pass to error handling middleware
}
});

```

Several design decisions here merit explanation. The `authenticate` middleware runs before our handler, populating `req.user` with the authenticated user's information. Query parameters are strings, so we parse them to numbers. We cap `limit` at 100 to prevent clients from requesting enormous result sets. The response includes pagination metadata so clients know how to fetch more results.

The single-item endpoint retrieves one task:

```

Terminal

// GET /api/tasks/:id - Get single task
router.get('/:id', authenticate, async (req, res, next) => {
  try {
    // Service checks ownership internally
    const task = await taskService.getById(req.params.id, req.user.id);

    if (!task) {
      return res.status(404).json({
        error: {
          code: 'TASK_NOT_FOUND',
          message: 'Task not found'
        }
      });
    }

    res.json({ data: task });
  } catch (error) {
    next(error);
  }
}

```

Terminal -- continued

```

    }
  });

```

Notice the error response structure—we include both a machine-readable code (`TASK_NOT_FOUND`) and a human-readable message. This allows clients to handle specific errors programmatically while still displaying meaningful messages to users.

Creating a task validates input and returns the created resource:

Terminal

```

// POST /api/tasks - Create task
router.post('/', authenticate, validate(taskSchema.create), async (req, res, next) => {
  try {
    // Validation already passed (middleware would have returned 422)
    // Add authenticated user as owner
    const task = await taskService.create({
      ...req.body,
      userId: req.user.id
    });

    // 201 Created with the new resource
    res.status(201).json({ data: task });
  } catch (error) {
    next(error);
  }
});

```

The `validate(taskSchema.create)` middleware runs before our handler, ensuring `req.body` contains valid data. If validation fails, the middleware returns a 422 response and our handler never runs. This keeps validation logic centralized and reusable.

The update endpoint demonstrates PATCH semantics—partial updates:

Terminal

```

// PATCH /api/tasks/:id - Partial update
router.patch('/:id', authenticate, validate(taskSchema.patch), async (req, res, next) => {
  try {
    // Service updates only provided fields
    const task = await taskService.update(req.params.id, req.user.id, req.body);

```

Terminal -- continued

```

    if (!task) {
      return res.status(404).json({
        error: {
          code: 'TASK_NOT_FOUND',
          message: 'Task not found'
        }
      });
    }

    res.json({ data: task });
  } catch (error) {
    next(error);
  }
});

```

With PATCH, clients send only the fields they want to change. If you want to update just the status, send { "status": "done" }—other fields remain unchanged. This differs from PUT, where you'd send the entire resource and unspecified fields would be cleared.

Finally, deletion:

Terminal

```

// DELETE /api/tasks/:id - Delete task
router.delete('/:id', authenticate, async (req, res, next) => {
  try {
    const deleted = await taskService.delete(req.params.id, req.user.id);

    if (!deleted) {
      return res.status(404).json({
        error: {
          code: 'TASK_NOT_FOUND',
          message: 'Task not found'
        }
      });
    }

    // 204 No Content - success but nothing to return
    res.status(204).send();
  } catch (error) {
    next(error);
  }
});

module.exports = router;

```

```
Terminal

// services/taskService.js
const db = require('../db');

class TaskService {
  async list({ userId, page, limit, filters, sort, order }) {
    // Start building query
    const query = db('tasks')
      .where('user_id', userId)
      .orderBy(sort, order);

    // Apply filters conditionally
    if (filters.status) {
      query.where('status', filters.status);
    }
    if (filters.priority) {
      query.where('priority', filters.priority);
    }

    // Get total count for pagination (before limit/offset)
    const countQuery = query.clone();
    const [{ count }] = await countQuery.count('* as count');

    // Apply pagination
    const tasks = await query
      .limit(limit)
      .offset((page - 1) * limit);

    return {
      tasks,
      page,
      limit,
      total: parseInt(count)
    };
  }
}
```



Terminal

```
async getById(id, userId) {  
  // Combine id check and ownership check in one query  
  return db('tasks')  
    .where({ id, user_id: userId })  
    .first();  
}
```



Terminal

```
async create(data) {  
  // Insert and return the created row  
  const [task] = await db('tasks')  
    .insert({  
      title: data.title,  
      description: data.description,  
      user_id: data.userId,  
      project_id: data.projectId,  
      status: data.status || 'todo',  
      priority: data.priority || 0,  
      due_date: data.dueDate  
    })  
    .returning('*'); // PostgreSQL returns the inserted row  
  
  return task;  
}
```



Terminal

```
async update(id, userId, data) {  
  // Only update provided fields  
  const [task] = await db('tasks')  
    .where({ id, user_id: userId })  
    .update({  
      ...data, // Spread provided fields  
    })  
}
```

Terminal -- continued

```
        updated_at: db.fn.now()      // Always update timestamp
      })
      .returning('*');

    return task; // undefined if no row matched
  }

  async delete(id, userId) {
    const deleted = await db('tasks')
      .where({ id, user_id: userId })
      .del();

    return deleted > 0; // True if a row was deleted
  }
}

module.exports = new TaskService();
```

2.4.6 Response Structure

Terminal

```
// Success - single resource
{
  "data": {
    "id": 123,
    "title": "Complete project",
    "status": "in_progress",
    "created_at": "2024-12-09T10:30:00Z"
  }
}

// Success - collection with pagination
{
  "data": [
    { "id": 123, "title": "Task 1" },
```


Terminal -- continued

```
    { "id": 124, "title": "Task 2" }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 45,
    "totalPages": 3
  }
}

// Error
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request data",
    "details": [
      { "field": "title", "message": "Title is required" },
      { "field": "priority", "message": "Priority must be between 0 and 4" }
    ]
  }
}
```

2.5

GraphQL

2.5.1 The Problem GraphQL Solves

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the word "Terminal". The terminal displays a GraphQL query for a "Dashboard" type. The query is indented and uses curly braces for object and list literals. It includes fields for "me" (name, avatar), "myTasks" (id, title, status, assignee name), and "myProjects" (name, taskCount).

```
query Dashboard {  
  me {  
    name  
    avatar  
  }  
  myTasks(limit: 5) {  
    id  
    title  
    status  
    assignee {  
      name  
    }  
  }  
  myProjects {  
    name  
    taskCount  
  }  
}
```

2.5.2 GraphQL Schema

```
Terminal

# schema.graphql

# Enums define a fixed set of values
enum TaskStatus {
  TODO
  IN_PROGRESS
  REVIEW
  DONE
}

enum TaskPriority {
  LOW
  MEDIUM
  HIGH
  URGENT
}
```

```
Terminal

# Object types define the shape of resources
type User {
  id: ID! # ! means non-nullable
  email: String!
  name: String!
  avatar: String # No ! means nullable
  tasks(status: TaskStatus, limit: Int): [Task!]! # Returns list of Tasks
  projects: [Project!]!
  createdAt: DateTime!
}
```

```
Terminal

type Task {
  id: ID!
  title: String!
  description: String
  status: TaskStatus!
  priority: TaskPriority!
  assignee: User!           # Relationship to User
  project: Project          # Optional relationship
  comments: [Comment!]!
  dueDate: DateTime
  createdAt: DateTime!
  updatedAt: DateTime!
}

type Project {
  id: ID!
  name: String!
  description: String
  owner: User!
  members: [User!]!
  tasks(status: TaskStatus): [Task!]!
  taskCount: Int!           # Computed field
  completedTaskCount: Int!
  createdAt: DateTime!
}
```

```
Terminal

input CreateTaskInput {
  title: String!
  description: String
  projectId: ID
  priority: TaskPriority = MEDIUM # Default value
  dueDate: DateTime
}

input UpdateTaskInput {
  title: String
  description: String
  status: TaskStatus
}
```

Terminal -- continued

```
priority: TaskPriority
dueDate: DateTime
}
```

Terminal

```
type Query {
  # Current authenticated user
  me: User!

  # Look up specific resources
  user(id: ID!): User
  task(id: ID!): Task
  project(id: ID!): Project

  # List resources with filtering
  tasks(
    status: TaskStatus
    priority: TaskPriority
    projectId: ID
    limit: Int
    offset: Int
  ): [Task!]!
}
```

Terminal

```
type Mutation {
  createTask(input: CreateTaskInput!): Task!
  updateTask(id: ID!, input: UpdateTaskInput!): Task!
  deleteTask(id: ID!): Boolean!

  addComment(taskId: ID!, text: String!): Comment!
```

Terminal -- continued

```
}
```

2.5.3 Writing GraphQL Queries

Terminal

```
query GetMe {  
  me {  
    id  
    name  
    email  
  }  
}
```

Terminal

```
{  
  "data": {  
    "me": {  
      "id": "123",  
      "name": "Alice",  
      "email": "alice@example.com"  
    }  
  }  
}
```

```
Terminal

query GetTask($taskId: ID!) {
  task(id: $taskId) {
    id
    title
    status
    dueDate
  }
}
```

```
Terminal

{
  "query": "...",
  "variables": { "taskId": "789" }
}
```

```
Terminal

query GetUserWithTasks($userId: ID!) {
  user(id: $userId) {
    id
    name
    tasks(status: IN_PROGRESS, limit: 10) {
      id
      title
      priority
      project {
        id
        name
      }
    }
  }
}
```

```
Terminal

query Dashboard {
  me {
    id
    name
    tasks(limit: 5) {
      id
      title
      status
      dueDate
    }
  }

  projects {
    id
    name
    taskCount
    completedTaskCount
  }

  urgentTasks: tasks(priority: URGENT, status: TODO) {
    id
    title
    dueDate
    project {
      name
    }
  }
}
```

```
Terminal

fragment TaskFields on Task {
  id
  title
  status
  priority
  dueDate
}
```


Terminal -- continued

```
}

query GetProjectTasks($projectId: ID!) {
  project(id: $projectId) {
    name
    tasks {
      ...TaskFields
      assignee {
        name
      }
    }
  }
}
```

2.5.4 GraphQL Mutations

Terminal

```
mutation CreateTask($input: CreateTaskInput!) {
  createTask(input: $input) {
    id
    title
    status
    createdAt
  }
}
```

Terminal

```
{
  "input": {
    "title": "Review documentation",
    "projectId": "123",
    "priority": "HIGH",
    "dueDate": "2024-12-15"
  }
}
```

Terminal -- continued

```
}  
}
```

Terminal

```
mutation BatchUpdate {  
  task1: updateTask(id: "1", input: { status: DONE }) {  
    id  
    status  
  }  
  task2: updateTask(id: "2", input: { status: IN_PROGRESS }) {  
    id  
    status  
  }  
}
```

2.5.5 Implementing GraphQL Resolvers

Terminal

```
// resolvers.js  
const db = require('./db');  
  
const resolvers = {  
  // Root Query resolvers  
  Query: {  
    me: async (_, __, { user }) => {  
      // The third argument is context, containing authenticated user  
      if (!user) throw new Error('Not authenticated');  
      return db('users').where('id', user.id).first();  
    }  
  }  
}
```

Terminal -- continued

```

  },

  task: async (_, { id }, { user }) => {
    if (!user) throw new Error('Not authenticated');
    return db('tasks').where({ id, user_id: user.id }).first();
  },

  tasks: async (_, { status, priority, projectId, limit = 20, offset = 0 }, { user }) => {
    if (!user) throw new Error('Not authenticated');

    // Build query with conditional filters
    const query = db('tasks').where('user_id', user.id);

    if (status) query.where('status', status.toLowerCase());
    if (priority) query.where('priority', priorityToNumber(priority));
    if (projectId) query.where('project_id', projectId);

    return query
      .limit(limit)
      .offset(offset)
      .orderBy('created_at', 'desc');
  },
},

```

- 1.
- 2.
- 3.
- 4.

Terminal

```

Mutation: {
  createTask: async (_, { input }, { user }) => {
    if (!user) throw new Error('Not authenticated');

    const [task] = await db('tasks')
      .insert({
        title: input.title,
        description: input.description,

```

Terminal -- continued

```

    user_id: user.id,
    project_id: input.projectId,
    priority: priorityToNumber(input.priority),
    due_date: input.dueDate,
    status: 'todo'
  })
  .returning('*');

  return task;
},

updateTask: async (_, { id, input }, { user }) => {
  if (!user) throw new Error('Not authenticated');

  // Build update object from provided fields
  const updates = { updated_at: db.fn.now() };
  if (input.title) updates.title = input.title;
  if (input.description !== undefined) updates.description = input.description;
  if (input.status) updates.status = input.status.toLowerCase();
  if (input.priority) updates.priority = priorityToNumber(input.priority);
  if (input.dueDate) updates.due_date = input.dueDate;

  const [task] = await db('tasks')
    .where({ id, user_id: user.id })
    .update(updates)
    .returning('*');

  return task;
},
},

```

Terminal

```

// Resolvers for Task type fields
Task: {
  // Resolve the assignee relationship
  assignee: (task) => {
    return db('users').where('id', task.user_id).first();
  },

  // Resolve the optional project relationship
  project: (task) => {
    if (!task.project_id) return null;

```

Terminal -- continued

```
    return db('projects').where('id', task.project_id).first();
  },

  // Resolve comments list
  comments: (task) => {
    return db('comments')
      .where('task_id', task.id)
      .orderBy('created_at', 'asc');
  },

  // Transform database values to GraphQL enum format
  status: (task) => task.status.toUpperCase(),
  priority: (task) => numberToPriority(task.priority),
},

// Resolvers for Project type fields
Project: {
  owner: (project) => {
    return db('users').where('id', project.owner_id).first();
  },

  tasks: (project, { status }) => {
    const query = db('tasks').where('project_id', project.id);
    if (status) query.where('status', status.toLowerCase());
    return query;
  },

  // Computed field - count tasks
  taskCount: async (project) => {
    const [{ count }] = await db('tasks')
      .where('project_id', project.id)
      .count('* as count');
    return parseInt(count);
  },
},
};
```

2.5.6 The N+1 Problem and DataLoader

```
Terminal

query {
  tasks(limit: 100) {
    title
    assignee {
      name
    }
  }
}
```

```
Terminal

const DataLoader = require('dataloader');

// Batch function receives array of keys, returns array of results in same order
const createUserLoader = () => new DataLoader(async (userIds) => {
  // One query for all requested users
  const users = await db('users').whereIn('id', userIds);

  // Return in same order as requested IDs
  const userMap = new Map(users.map(u => [u.id, u]));
  return userIds.map(id => userMap.get(id));
});
```

```
Terminal

// Create fresh loaders per request (in context)
const context = ({ req }) => ({
  user: authenticate(req),
  loaders: {
```

Terminal -- continued

```
    user: createUserLoader(),
    project: createProjectLoader(),
  }
});

// Use loader in resolver
const resolvers = {
  Task: {
    assignee: (task, _, { loaders }) => {
      return loaders.user.load(task.user_id);
    },
    project: (task, _, { loaders }) => {
      if (!task.project_id) return null;
      return loaders.project.load(task.project_id);
    },
  },
};
```

2.6

API Documentation

2.6.1 OpenAPI Specification

```
Terminal

openapi: 3.0.3
info:
  title: TaskFlow API
  description: |
    API for the TaskFlow task management application.

  ## Authentication
  All endpoints except `/auth/login` and `/auth/register` require
  authentication. Include the JWT token in the Authorization header:
  ...

  Authorization: Bearer <token>
  ...

version: 1.0.0

servers:
- url: https://api.taskflow.com/v1
  description: Production
- url: http://localhost:3000/v1
  description: Local development
```

```
Terminal

paths:
  /tasks:
    get:
      summary: List tasks
      description: |
        Returns a paginated list of tasks for the authenticated user.
        Results can be filtered by status, priority, and project.
      tags:
        - Tasks
      security:
        - bearerAuth: []
      parameters:
        - name: status
          in: query
          description: Filter by task status
          schema:
            type: string
            enum: [todo, in_progress, review, done]
```


Terminal -- continued

```
- name: page
  in: query
  description: Page number (1-indexed)
  schema:
    type: integer
    default: 1
    minimum: 1
- name: limit
  in: query
  description: Results per page (max 100)
  schema:
    type: integer
    default: 20
    minimum: 1
    maximum: 100
```

Terminal

```
responses:
  '200':
    description: Paginated list of tasks
    content:
      application/json:
        schema:
          type: object
          properties:
            data:
              type: array
              items:
                $ref: '#/components/schemas/Task'
            pagination:
              $ref: '#/components/schemas/Pagination'
  '401':
    $ref: '#/components/responses/Unauthorized'
```

```
components:
  schemas:
    Task:
      type: object
      required:
        - id
        - title
        - status
      properties:
        id:
          type: integer
          example: 123
        title:
          type: string
          example: Review pull request
          minLength: 1
          maxLength: 200
        description:
          type: string
          nullable: true
        status:
          type: string
          enum: [todo, in_progress, review, done]
        priority:
          type: integer
          minimum: 0
          maximum: 3
        dueDate:
          type: string
          format: date
          nullable: true
```

2.6.2 Serving Documentation

```
const swaggerUi = require('swagger-ui-express');
const YAML = require('yamljs');
```

Terminal -- continued

```
const swaggerDocument = YAML.load('./openapi.yaml');

// Serve documentation at /api-docs
app.use('/api-docs', swaggerUi.serve, swaggerUi.setup(swaggerDocument, {
  customCss: '.swagger-ui .topbar { display: none }',
  customSiteTitle: 'TaskFlow API Documentation'
}));
```

2.7

Data Validation

2.7.1 Validation with Joi

Terminal

```
const Joi = require('joi');

const taskSchema = {
  create: Joi.object({
    title: Joi.string()
      .min(1)
      .max(200)
      .required()
      .messages({

```

Terminal -- continued

```
    'string.empty': 'Title cannot be empty',
    'string.max': 'Title cannot exceed 200 characters',
    'any.required': 'Title is required'
  )),

  description: Joi.string()
    .max(2000)
    .allow('', null), // Empty string and null are valid

  projectId: Joi.number()
    .integer()
    .positive()
    .allow(null),

  priority: Joi.number()
    .integer()
    .min(0)
    .max(3)
    .default(0), // Use 0 if not provided

  dueDate: Joi.date()
    .iso()
    .greater('now') // Must be in the future
    .allow(null)
  )),

  // Update schema - all fields optional but at least one required
  update: Joi.object({
    title: Joi.string().min(1).max(200),
    description: Joi.string().max(2000).allow('', null),
    status: Joi.string().valid('todo', 'in_progress', 'review', 'done'),
    priority: Joi.number().integer().min(0).max(3),
    dueDate: Joi.date().iso().allow(null)
  }).min(1) // At least one field must be provided
};
```

```
Terminal

const validate = (schema) => {
  return (req, res, next) => {
    const { error, value } = schema.validate(req.body, {
      abortEarly: false,    // Return all errors, not just first
      stripUnknown: true    // Remove fields not in schema
    });

    if (error) {
      // Transform Joi errors into our API format
      const details = error.details.map(detail => ({
        field: detail.path.join('.'),
        message: detail.message
      }));

      return res.status(422).json({
        error: {
          code: 'VALIDATION_ERROR',
          message: 'Invalid request data',
          details
        }
      });
    }

    // Replace body with validated/sanitized version
    req.body = value;
    next();
  };
};
```

2.7.2 Centralized Error Handling

```
Terminal

// Custom error classes for different scenarios
class AppError extends Error {
  constructor(code, message, statusCode = 500, details = null) {
    super(message);
    this.code = code;
  }
}
```

Terminal -- continued

```

    this.statusCode = statusCode;
    this.details = details;
    this.isOperational = true; // Distinguishes from programming bugs
  }
}

class NotFoundError extends AppError {
  constructor(resource = 'Resource') {
    super('NOT_FOUND', `${resource} not found`, 404);
  }
}

class ValidationError extends AppError {
  constructor(details) {
    super('VALIDATION_ERROR', 'Invalid request data', 422, details);
  }
}

```

Terminal

```

// Error handling middleware (must have 4 parameters)
const errorHandler = (err, req, res, next) => {
  // Log for debugging
  console.error('Error:', {
    message: err.message,
    code: err.code,
    stack: err.stack,
    path: req.path
  });

  // Handle our custom errors
  if (err instanceof AppError) {
    return res.status(err.statusCode).json({
      error: {
        code: err.code,
        message: err.message,
        ...(err.details && { details: err.details })
      }
    });
  }

  // Handle database constraint violations

```

Terminal -- continued

```
if (err.code === '23505') { // PostgreSQL unique violation
  return res.status(409).json({
    error: {
      code: 'CONFLICT',
      message: 'Resource already exists'
    }
  });
}

// Unknown errors - don't leak details in production
res.status(500).json({
  error: {
    code: 'INTERNAL_ERROR',
    message: process.env.NODE_ENV === 'production'
      ? 'An unexpected error occurred'
      : err.message
  }
});
};

// Register as last middleware
app.use(errorHandler);
```

2.8

API Security

2.8.1 Authentication with JWT

```
Terminal

const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');

async function login(email, password) {
  // Find user by email
  const user = await db('users').where('email', email).first();

  if (!user) {
    throw new UnauthorizedError('Invalid credentials');
  }

  // Verify password against stored hash
  const validPassword = await bcrypt.compare(password, user.password_hash);

  if (!validPassword) {
    throw new UnauthorizedError('Invalid credentials');
  }

  // Generate signed token
  const token = jwt.sign(
    { userId: user.id, email: user.email }, // Payload
    process.env.JWT_SECRET,                // Secret key
    { expiresIn: '24h' }                   // Options
  );

  return { token, user: { id: user.id, name: user.name, email: user.email } };
}
```

```
Terminal

const authenticate = async (req, res, next) => {
  try {
    // Extract token from header
```


Terminal -- continued

```

const authHeader = req.headers.authorization;

if (!authHeader || !authHeader.startsWith('Bearer ')) {
  throw new UnauthorizedError('No token provided');
}

const token = authHeader.substring(7); // Remove 'Bearer ' prefix

// Verify signature and decode payload
const decoded = jwt.verify(token, process.env.JWT_SECRET);

// Optionally verify user still exists (handles deleted accounts)
const user = await db('users').where('id', decoded.userId).first();

if (!user) {
  throw new UnauthorizedError('User no longer exists');
}

// Attach user to request for downstream handlers
req.user = user;
next();

} catch (error) {
  if (error.name === 'TokenExpiredError') {
    return next(new UnauthorizedError('Token expired'));
  }
  if (error.name === 'JsonWebTokenError') {
    return next(new UnauthorizedError('Invalid token'));
  }
  next(error);
}
};

```

2.8.2 Rate Limiting



Terminal

```

const rateLimit = require('express-rate-limit');

// General API rate limit
const apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minute window

```

Terminal -- continued

```
max: 100,                // 100 requests per window
message: {
  error: {
    code: 'RATE_LIMIT_EXCEEDED',
    message: 'Too many requests, please try again later'
  }
}
});

// Strict limit for authentication (prevents brute force)
const authLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, // 1 hour window
  max: 10,                  // 10 attempts per hour
  skipSuccessfulRequests: true, // Don't count successful logins
  message: {
    error: {
      code: 'AUTH_RATE_LIMIT',
      message: 'Too many login attempts, please try again later'
    }
  }
});

app.use('/api/', apiLimiter);
app.use('/api/auth/login', authLimiter);
```

2.9

Caching Strategies

2.9.1 Cache-Aside Pattern

```
Terminal

+-----+
|                CACHE-ASIDE PATTERN                |
+-----+
|
| READ FLOW:
| 1. Application checks cache for data
| 2. If found (cache hit), return cached data
| 3. If not found (cache miss), fetch from database
| 4. Store result in cache for future requests
| 5. Return data to client
|
| WRITE FLOW:
| 1. Update database (source of truth)
| 2. Invalidate (delete) cached data
| 3. Next read will fetch fresh data and repopulate cache
|
+-----+
```

```
Terminal

const CACHE_TTL = 300; // 5 minutes

async function getTaskWithCache(taskId) {
  const cacheKey = `task:${taskId}`;

  // Step 1: Check cache
  const cached = await redis.get(cacheKey);
  if (cached) {
    console.log('Cache hit');
    return JSON.parse(cached);
  }

  // Step 2: Cache miss - fetch from database
  console.log('Cache miss - querying database');
  const task = await db('tasks').where('id', taskId).first();

  // Step 3: Populate cache for future requests
  if (task) {
    await redis.set(cacheKey, JSON.stringify(task), 'EX', CACHE_TTL);
  }
}
```

Terminal -- continued

```
}  
  
return task;  
}
```

Terminal

```
async function updateTask(taskId, updates) {  
  // Update the source of truth  
  const [task] = await db('tasks')  
    .where('id', taskId)  
    .update(updates)  
    .returning('*');  
  
  // Invalidate cache - next read will fetch fresh data  
  await redis.del(`task:${taskId}`);  
  
  // Also invalidate related caches  
  await redis.del(`user:${task.user_id}:tasks`);  
  
  return task;  
}
```

2.9.2 Cache Invalidation Challenges

-

-
-
-
-
-
-

2.10

Chapter Summary

2.11

Key Terms

Term	Definition
Foreign Key	Column that references a primary key in another table, creating relationships
ACID	Properties (Atomicity, Consistency, Isolation, Durability) ensuring reliable transactions
REST	Architectural style using resources, HTTP methods, and stateless communication
GraphQL	Query language allowing clients to specify exactly what data they need
OpenAPI	Specification standard for documenting REST APIs

Term	Definition
Rate Limiting	Controlling request frequency to prevent abuse
N+1 Problem	Performance issue where fetching N items causes N+1 database queries

2.12

Review Questions

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.

2.13

Hands-On Exercises

2.13.1 Exercise 10.1: Database Design

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

2.13.2 Exercise 10.2: SQL Query Practice

- 1.
- 2.
- 3.
- 4.
- 5.

2.13.3 Exercise 10.3: REST API Implementation

- 1.
- 2.
- 3.
- 4.
- 5.

2.13.4 Exercise 10.4: API Documentation

- 1.
- 2.
- 3.
- 4.
- 5.

2.13.5 Exercise 10.5: Caching Implementation

- 1.
- 2.
- 3.
- 4.
- 5.

2.13.6 Exercise 10.6: GraphQL Alternative

- 1.
- 2.
- 3.
- 4.

2.14

Further Reading

-
-
-

-
-
-
-
-

2.15

References

Part II.

Production Systems

Cloud Services and Deployment

Packaging, deploying, scaling, and operating software on modern cloud platforms.

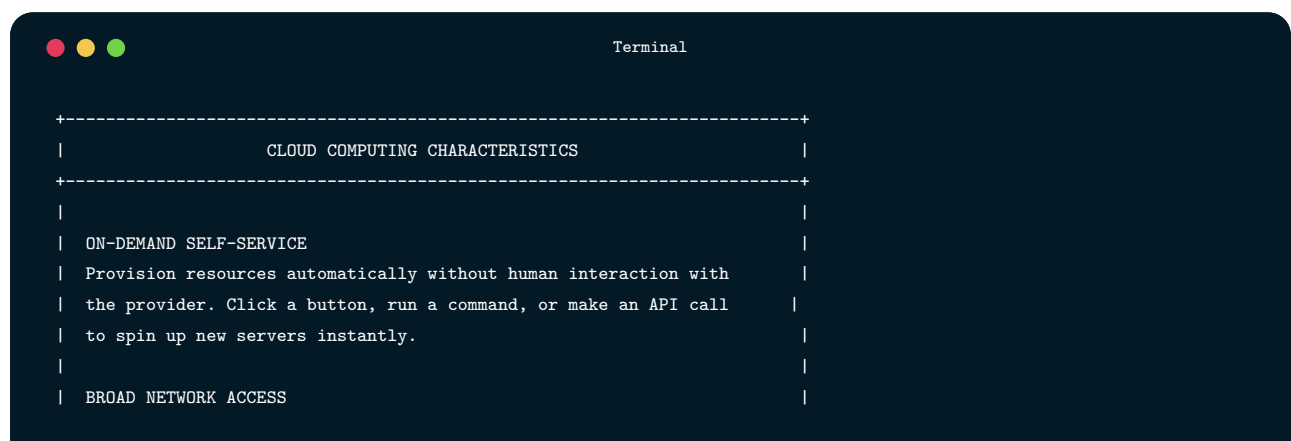
LEARNING OBJECTIVES

- ✓ By the end of this chapter, you will be able to:
 - ✓ Explain the fundamental concepts of cloud computing and its service models
 - ✓ Compare major cloud providers (AWS, Google Cloud, Azure) and their core services
 - ✓ Containerize applications using Docker with best practices for production
 - ✓ Orchestrate containers using Kubernetes for scalable deployments
 - ✓ Design serverless architectures using functions-as-a-service
 - ✓ Implement infrastructure as code for reproducible deployments
 - ✓ Apply cloud security best practices and cost optimization strategies
 - ✓ Choose appropriate cloud services for different application requirements

3.1

The Cloud Computing Revolution

3.1.1 What is Cloud Computing?



Terminal -- continued

```

| Access services from anywhere via standard network protocols. |
| Your infrastructure is available globally, not tied to a physical |
| location. |
| |
| RESOURCE POOLING |
| Provider's resources serve multiple customers from the same physical |
| infrastructure. This multi-tenancy enables economies of scale that |
| individual organizations couldn't achieve alone. |
| |
| RAPID ELASTICITY |
| Scale resources up or down quickly based on demand. Handle traffic |
| spikes without planning months ahead, and scale down during quiet |
| periods to save costs. |
| |
| MEASURED SERVICE |
| Pay for what you use, measured automatically. No upfront costs for |
| hardware; operating expenses replace capital expenses. |
| |
+-----+

```

3.1.2 Cloud Service Models

Terminal

```

+-----+
|          CLOUD SERVICE MODELS          |
+-----+
| |
| +-----+ |
| |          SOFTWARE AS A SERVICE (SaaS)          | |
| | Complete applications delivered over the internet | |
| | You manage: Just your data and user access      | |
| | Provider manages: Everything else                | |
| | Examples: Gmail, Salesforce, Slack, GitHub      | |
| | +-----+ |
+-----+

```

Terminal -- continued

```

|                                     |
|                                     | More abstraction, less control |
| +-----+                         +-----+ |
| |                                     | | | |
| | PLATFORM AS A SERVICE (PaaS)      | |
| | Platform for building and deploying applications | |
| | You manage: Application code, data | |
| | Provider manages: Runtime, OS, servers, storage, networking | |
| | Examples: Heroku, Google App Engine, AWS Elastic Beanstalk | |
| | +-----+                         +-----+ |
| |                                     | |
| |                                     | |
| | +-----+                         +-----+ |
| | |                                     | |
| | | INFRASTRUCTURE AS A SERVICE (IaaS) | |
| | | Raw computing resources: VMs, storage, networks | |
| | | You manage: OS, runtime, middleware, applications, data | |
| | | Provider manages: Virtualization, servers, storage, networking | |
| | | Examples: AWS EC2, Google Compute Engine, Azure VMs | |
| | | +-----+                         +-----+ |
| | |                                     | |
| | |                                     | Less abstraction, more control |
| | | +-----+                         +-----+ |
| | | |                                     | |
| | | | ON-PREMISES / BARE METAL        | |
| | | | You own and manage everything    | |
| | | | +-----+                         +-----+ |
| | | |                                     | |
| | | | +-----+                         +-----+ |

```

3.1.3 Major Cloud Providers

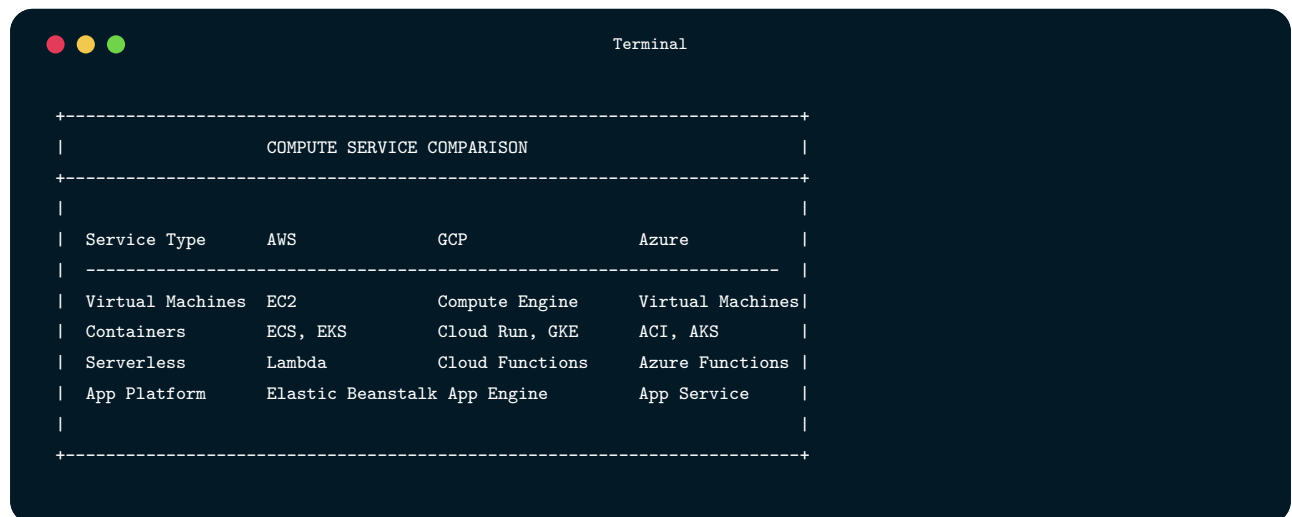
```
Terminal

+-----+
|          MAJOR CLOUD PROVIDERS          |
+-----+
|
|  AMAZON WEB SERVICES (AWS)
|  • Market leader (~32% market share)
|  • Broadest service catalog (200+ services)
|  • Most mature ecosystem and documentation
|  • Strengths: Breadth, enterprise features, global reach
|  • Key services: EC2, S3, Lambda, RDS, DynamoDB, EKS
|
|  GOOGLE CLOUD PLATFORM (GCP)
|  • Strong in data analytics and machine learning
|  • Kubernetes expertise (Google created Kubernetes)
|  • Excellent network performance
|  • Strengths: BigQuery, AI/ML, Kubernetes, developer experience
|  • Key services: Compute Engine, Cloud Storage, BigQuery, GKE
|
|  MICROSOFT AZURE
|  • Strong enterprise integration (Active Directory, Office 365)
|  • Hybrid cloud leadership (Azure Arc, Azure Stack)
|  • Comprehensive compliance certifications
|  • Strengths: Enterprise, hybrid cloud, .NET ecosystem
|  • Key services: Virtual Machines, Blob Storage, Azure Functions
|
+-----+
```


3.2

Core Cloud Services

3.2.1 Compute Services



A terminal window with a dark blue background and three colored window control buttons (red, yellow, green) in the top-left corner. The title bar reads "Terminal". Inside the terminal, a table titled "COMPUTE SERVICE COMPARISON" is displayed using ASCII art borders. The table compares AWS, GCP, and Azure services across four categories: Service Type, Virtual Machines, Containers, and Serverless. The App Platform row is partially filled.

Service Type	AWS	GCP	Azure
Virtual Machines	EC2	Compute Engine	Virtual Machines
Containers	ECS, EKS	Cloud Run, GKE	ACI, AKS
Serverless	Lambda	Cloud Functions	Azure Functions
App Platform	Elastic Beanstalk	App Engine	App Service



Terminal

```
# Create a new EC2 instance
aws ec2 run-instances \
  --image-id ami-0c55b159cbfafa1f0 \      # Amazon Linux 2 AMI
  --instance-type t3.micro \              # 2 vCPU, 1GB RAM
  --key-name my-key-pair \                # SSH key for access
  --security-group-ids sg-903004f8 \      # Firewall rules
  --subnet-id subnet-6e7f829e \          # Network placement
  --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=web-server}]'
```

3.2.2 Storage Services

```
Terminal

const { S3Client, PutObjectCommand, GetObjectCommand } = require('@aws-sdk/client-s3');

// Create S3 client - credentials come from environment or IAM role
const s3Client = new S3Client({ region: 'us-east-1' });

async function uploadFile(bucket, key, body, contentType) {
  // PutObjectCommand uploads data to S3
  const command = new PutObjectCommand({
    Bucket: bucket,           // S3 bucket name (globally unique)
    Key: key,                 // Object path within bucket
    Body: body,               // File contents (Buffer, string, or stream)
    ContentType: contentType // MIME type for proper handling
  });

  await s3Client.send(command);

  // Construct the URL where the object can be accessed
  return `https://${bucket}.s3.amazonaws.com/${key}`;
}

async function downloadFile(bucket, key) {
  const command = new GetObjectCommand({
    Bucket: bucket,
    Key: key
  });

  const response = await s3Client.send(command);

  // Response.Body is a readable stream
  // Convert to string for text content
  return response.Body.transformToString();
}
```

3.2.3 Database Services

```
Terminal

+-----+
|               MANAGED DATABASE SERVICES               |
+-----+
|
| RELATIONAL DATABASES                                  |
| AWS: RDS (MySQL, PostgreSQL, Oracle, SQL Server), Aurora |
| GCP: Cloud SQL, Cloud Spanner                         |
| Azure: Azure SQL, Azure Database for PostgreSQL/MySQL  |
|
| Benefits: Automated backups, read replicas, automatic failover, |
| point-in-time recovery, managed patching                |
|
| NOSQL DATABASES                                       |
| AWS: DynamoDB (key-value), DocumentDB (document), ElastiCache |
| GCP: Firestore (document), Cloud Bigtable (wide-column), Memorystore |
| Azure: Cosmos DB (multi-model), Azure Cache for Redis      |
|
| Benefits: Automatic scaling, global distribution, single-digit |
| millisecond latency, serverless options                  |
|
+-----+
```

```
Terminal

const { Pool } = require('pg');

// Connection string from environment variable
// Format: postgresql://user:password@host:port/database
const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  ssl: {
    rejectUnauthorized: true // Verify SSL certificate
  },
  max: 20, // Connection pool size
  idleTimeoutMillis: 30000, // Close idle connections after 30s
  connectionTimeoutMillis: 2000 // Fail fast if can't connect
});

// RDS handles: backups, failover, patching, monitoring
// Your application just uses standard PostgreSQL

async function getUsers() {
  const client = await pool.connect();
  try {
    const result = await client.query('SELECT * FROM users LIMIT 10');
    return result.rows;
  } finally {
    client.release(); // Return connection to pool
  }
}
```

3.2.4 Networking Services



Terminal -- continued

+-----+



Terminal

```
// Example: Health check endpoint for load balancer
// The load balancer periodically calls this endpoint
// to verify the instance is healthy

app.get('/health', async (req, res) => {
  try {
    // Check database connectivity
    await db.raw('SELECT 1');

    // Check Redis connectivity
    await redis.ping();

    // Check available memory (fail if critically low)
    const memUsage = process.memoryUsage();
    const memoryOk = memUsage.heapUsed < memUsage.heapTotal * 0.95;

    if (!memoryOk) {
      return res.status(503).json({
        status: 'unhealthy',
        reason: 'Memory pressure'
      });
    }

    res.json({
      status: 'healthy',
      timestamp: new Date().toISOString()
    });
  } catch (error) {
    // Return 503 so load balancer stops sending traffic
    res.status(503).json({
      status: 'unhealthy',
```

Terminal -- continued

```
    error: error.message
  });
}
```

3.3

Containerization with Docker

3.3.1 The Problem Containers Solve



3.3.2 Docker Fundamentals

```
Terminal

# Dockerfile for a Node.js application

# Stage 1: Build stage
# Use Node 20 on Alpine Linux (small base image, ~50MB)
FROM node:20-alpine AS builder

# Set working directory inside the container
# All subsequent commands run relative to this directory
WORKDIR /app

# Copy package files first (separate from source code)
# This leverages Docker's layer caching - if package.json hasn't changed,
# npm install can be skipped on subsequent builds
COPY package*.json ./

# Install ALL dependencies (including devDependencies for building)
RUN npm ci

# Now copy application source code
# This layer changes frequently, but previous layers are cached
COPY . .

# Build the application (TypeScript compilation, bundling, etc.)
RUN npm run build

# Stage 2: Production stage
# Start fresh with a clean base image
FROM node:20-alpine AS production

# Run as non-root user for security
# Alpine includes a 'node' user we can use
USER node

# Set working directory
WORKDIR /app

# Copy package files and install ONLY production dependencies
COPY --chown=node:node package*.json ./
RUN npm ci --only=production
```

Terminal -- continued

```
# Copy built application from builder stage
# We don't need source code or devDependencies
COPY --chown=node:node --from=builder /app/dist ./dist

# Document which port the application uses
# (doesn't actually expose it - that's done at runtime)
EXPOSE 3000

# Set environment to production
ENV NODE_ENV=production

# Health check - Docker monitors container health
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
    CMD wget --quiet --tries=1 --spider http://localhost:3000/health || exit 1

# Command to run when container starts
CMD ["node", "dist/index.js"]
```

```
Terminal

# Build the image and tag it with a name
docker build -t my-app:1.0.0 .

# The build output shows each layer being created:
# => [builder 1/6] FROM node:20-alpine
# => [builder 2/6] WORKDIR /app
# => [builder 3/6] COPY package*.json ./
# => [builder 4/6] RUN npm ci
# => [builder 5/6] COPY . .
# => [builder 6/6] RUN npm run build
# => [production 1/5] FROM node:20-alpine
# ...

# Run the container
docker run -d \
  --name my-app \
  -p 3000:3000 \
  -e DATABASE_URL=postgresql://... \
  my-app:1.0.0

# Explanation of flags:
# -d: Run in background (detached mode)
# --name: Give the container a memorable name
# -p 3000:3000: Map host port 3000 to container port 3000
# -e: Set environment variables
# my-app:1.0.0: Image name and tag to run
```

3.3.3 Docker Compose for Local Development

```
Terminal

# docker-compose.yml
# Defines all services needed to run the application locally
```

Terminal -- continued

```

version: '3.8'

services:
  # Main application
  app:
    build:
      context: .
      dockerfile: Dockerfile
      target: builder # Use builder stage for hot reload
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: development
      DATABASE_URL: postgresql://postgres:password@db:5432/taskflow
      REDIS_URL: redis://redis:6379
    volumes:
      # Mount source code for hot reload
      # Changes on host immediately reflect in container
      - ./src:/app/src
      - ./package.json:/app/package.json
    depends_on:
      db:
        condition: service_healthy
      redis:
        condition: service_started
    # Override CMD for development (enables hot reload)
    command: npm run dev

  # PostgreSQL database
  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
      POSTGRES_DB: taskflow
    ports:
      - "5432:5432" # Expose for local database tools
    volumes:
      # Persist data between container restarts
      - postgres_data:/var/lib/postgresql/data
      # Run initialization scripts on first startup
      - ./scripts/init.sql:/docker-entrypoint-initdb.d/init.sql
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 5

```

Terminal -- continued

```
# Redis for caching and sessions
redis:
  image: redis:7-alpine
  ports:
    - "6379:6379"
  volumes:
    - redis_data:/data
  # Enable persistence
  command: redis-server --appendonly yes

# Database admin UI (development only)
adminer:
  image: adminer
  ports:
    - "8080:8080"
  depends_on:
    - db

# Named volumes persist data across container restarts
volumes:
  postgres_data:
  redis_data:
```

```
Terminal

# Start all services in the background
docker compose up -d

# View logs from all services
docker compose logs -f

# View logs from specific service
docker compose logs -f app

# Stop all services
docker compose down

# Stop and remove volumes (deletes database data!)
docker compose down -v

# Rebuild images after Dockerfile changes
docker compose build
docker compose up -d
```

3.3.4 Container Best Practices

```
Terminal

+-----+
|          CONTAINER BEST PRACTICES          |
+-----+
|
|  SECURITY                                  |
|  • Run as non-root user                    |
|  • Use minimal base images (Alpine, distroless) |
|  • Don't store secrets in images (use environment variables) |
|  • Scan images for vulnerabilities          |
|  • Keep base images updated                |
|
|  SIZE OPTIMIZATION                        |
|  • Use multi-stage builds                  |
|  • Choose small base images                |
|
```

Terminal -- continued

```
| • Minimize layer count (combine RUN commands) |
| • Use .dockerignore to exclude unnecessary files |
| • Remove package manager caches after installing |
| |
| RELIABILITY |
| • Implement health checks |
| • Use specific version tags, not 'latest' |
| • Make containers stateless (store state externally) |
| • Handle signals properly (graceful shutdown) |
| • Log to stdout/stderr (not files) |
| |
+-----+
```

Terminal

```
// Graceful shutdown handler
const server = app.listen(3000);

process.on('SIGTERM', async () => {
  console.log('SIGTERM received, starting graceful shutdown');

  // Stop accepting new requests
  server.close(async () => {
    console.log('HTTP server closed');

    // Close database connections
    await db.destroy();
    console.log('Database connections closed');

    // Close Redis connection
```


Terminal -- continued

```
    await redis.quit();
    console.log('Redis connection closed');

    console.log('Graceful shutdown complete');
    process.exit(0);
  });

  // Force shutdown if graceful shutdown takes too long
  setTimeout(() => {
    console.error('Forced shutdown after timeout');
    process.exit(1);
  }, 30000);
});
```

3.4

Container Orchestration with Kubernetes

3.4.1 Why Kubernetes?

-
-
-

-
-



3.4.2 Core Kubernetes Concepts

```
# kubernetes/deployment.yaml
# Defines the desired state for our application pods

apiVersion: apps/v1
kind: Deployment
metadata:
  name: taskflow-api
  labels:
    app: taskflow
    component: api
spec:
  # Run 3 replicas for high availability
  replicas: 3

  # How to identify pods managed by this Deployment
  selector:
    matchLabels:
      app: taskflow
      component: api

  # Strategy for updating pods
  strategy:
    type: RollingUpdate
```

Terminal -- continued

```
rollingUpdate:
  # During updates, allow up to 1 extra pod temporarily
  maxSurge: 1
  # During updates, ensure at least 2 pods are always running
  maxUnavailable: 1

# Pod template - defines what each pod looks like
template:
  metadata:
    labels:
      app: taskflow
      component: api
  spec:
    # Run pods on different nodes when possible (anti-affinity)
    affinity:
      podAntiAffinity:
        preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
            podAffinityTerm:
              labelSelector:
                matchLabels:
                  app: taskflow
                  component: api
              topologyKey: kubernetes.io/hostname

  containers:
    - name: api
      image: myregistry/taskflow-api:1.2.0

    # Resource limits and requests
    resources:
      requests:
        # Minimum resources guaranteed
        memory: "256Mi"
        cpu: "250m" # 250 millicores = 0.25 CPU
      limits:
        # Maximum resources allowed
        memory: "512Mi"
        cpu: "500m"

    ports:
      - containerPort: 3000

    # Environment variables from ConfigMap and Secrets
    env:
      - name: NODE_ENV
        value: "production"
```

Terminal -- continued

```
- name: PORT
  value: "3000"
- name: DATABASE_URL
  valueFrom:
    secretKeyRef:
      name: taskflow-secrets
      key: database-url
- name: REDIS_URL
  valueFrom:
    configMapKeyRef:
      name: taskflow-config
      key: redis-url

# Readiness probe - is the pod ready to receive traffic?
readinessProbe:
  httpGet:
    path: /health/ready
    port: 3000
  initialDelaySeconds: 5
  periodSeconds: 10
  failureThreshold: 3

# Liveness probe - is the pod still alive?
livenessProbe:
  httpGet:
    path: /health/live
    port: 3000
  initialDelaySeconds: 15
  periodSeconds: 20
  failureThreshold: 3

# Startup probe - has the pod finished starting?
startupProbe:
  httpGet:
    path: /health/live
    port: 3000
  initialDelaySeconds: 0
  periodSeconds: 5
  failureThreshold: 30 # 30 * 5 = 150s max startup time
```

-
-
-

```
Terminal

# kubernetes/service.yaml
# Creates a stable network endpoint for the API pods

apiVersion: v1
kind: Service
metadata:
  name: taskflow-api
  labels:
    app: taskflow
    component: api
spec:
  type: ClusterIP # Internal-only; use LoadBalancer for external access

# Which pods receive traffic from this service
selector:
  app: taskflow
  component: api

ports:
- name: http
  port: 80 # Port exposed by the service
  targetPort: 3000 # Port on the pods
  protocol: TCP
```

```
Terminal

# kubernetes/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: taskflow-config
data:
  redis-url: "redis://redis-service:6379"
  log-level: "info"
  feature-flags: |
    {
      "newDashboard": true,
      "betaFeatures": false
    }

---
# kubernetes/secret.yaml
# Note: In practice, use sealed-secrets or external-secrets
# Never commit actual secrets to version control!

apiVersion: v1
kind: Secret
metadata:
  name: taskflow-secrets
type: Opaque
data:
  # Values are base64 encoded (NOT encrypted!)
  # Use: echo -n "value" | base64
  database-url: cG9zdGdyZXNxbDovL3VzZXI6cGFzc0Bob3N00jU0MzIvZGI=
  jwt-secret: c3VwZXItc2VjcmV0LWtleS1jaGFuZ2UtdGhpcw==
```

3.4.3 Deploying to Kubernetes

```
Terminal

# Apply all manifests in a directory
kubectl apply -f kubernetes/
```

Terminal -- continued

```

# Watch deployment progress
kubectl rollout status deployment/taskflow-api

# View running pods
kubectl get pods -l app=taskflow

# Example output:
# NAME                                READY   STATUS    RESTARTS   AGE
# taskflow-api-7d9f8c6b5-abc12        1/1     Running   0           2m
# taskflow-api-7d9f8c6b5-def34        1/1     Running   0           2m
# taskflow-api-7d9f8c6b5-ghi56        1/1     Running   0           2m

# View detailed pod information
kubectl describe pod taskflow-api-7d9f8c6b5-abc12

# View pod logs
kubectl logs taskflow-api-7d9f8c6b5-abc12

# Follow logs in real-time
kubectl logs -f taskflow-api-7d9f8c6b5-abc12

# Execute a command in a running pod (for debugging)
kubectl exec -it taskflow-api-7d9f8c6b5-abc12 -- /bin/sh

```

Terminal

```

# Update to a new image version
kubectl set image deployment/taskflow-api api=myregistry/taskflow-api:1.3.0

# Or edit the manifest and apply again
kubectl apply -f kubernetes/deployment.yaml

# Watch the rollout
kubectl rollout status deployment/taskflow-api

# If something goes wrong, rollback to previous version
kubectl rollout undo deployment/taskflow-api

# View rollout history
kubectl rollout history deployment/taskflow-api

```


3.4.4 Horizontal Pod Autoscaling

```
Terminal

# kubernetes/hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: taskflow-api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: taskflow-api

  # Scaling boundaries
  minReplicas: 2    # Never scale below 2 for availability
  maxReplicas: 10   # Never scale above 10 for cost control

  # Metrics that trigger scaling
  metrics:
    # Scale based on CPU utilization
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70 # Target 70% CPU usage

    # Scale based on memory utilization
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80 # Target 80% memory usage

  # Scaling behavior customization
  behavior:
    scaleDown:
      # Wait 5 minutes before scaling down (prevents flapping)
```

Terminal -- continued

```
stabilizationWindowSeconds: 300
policies:
  - type: Percent
    value: 50          # Remove at most 50% of pods
    periodSeconds: 60  # Per minute
scaleUp:
  # Scale up more aggressively than down
  stabilizationWindowSeconds: 0
  policies:
    - type: Percent
      value: 100        # Can double pod count
      periodSeconds: 60
    - type: Pods
      value: 4          # Or add up to 4 pods
      periodSeconds: 60
```

3.4.5 Managed Kubernetes Services

Terminal

```
+-----+
|          MANAGED KUBERNETES SERVICES          |
+-----+
|
| AWS: Amazon EKS (Elastic Kubernetes Service) |
| • Integrates with AWS services (IAM, VPC, ALB, EBS) |
| • EKS Anywhere for hybrid deployments |
| • Fargate option for serverless pods |
|
| GCP: Google Kubernetes Engine (GKE) |
```

Terminal -- continued

```
| • Most mature managed Kubernetes (Google created K8s) |
| • Autopilot mode for fully managed node pools |
| • Excellent network performance and observability |
| |
| Azure: Azure Kubernetes Service (AKS) |
| • Strong enterprise integration (Active Directory) |
| • Azure Arc for hybrid/multi-cloud |
| • Virtual nodes for serverless containers |
| |
| All managed services provide: |
| • Managed control plane (automatic updates, high availability) |
| • Integration with cloud networking and storage |
| • IAM integration for security |
| • Monitoring and logging integration |
| |
+-----+
```

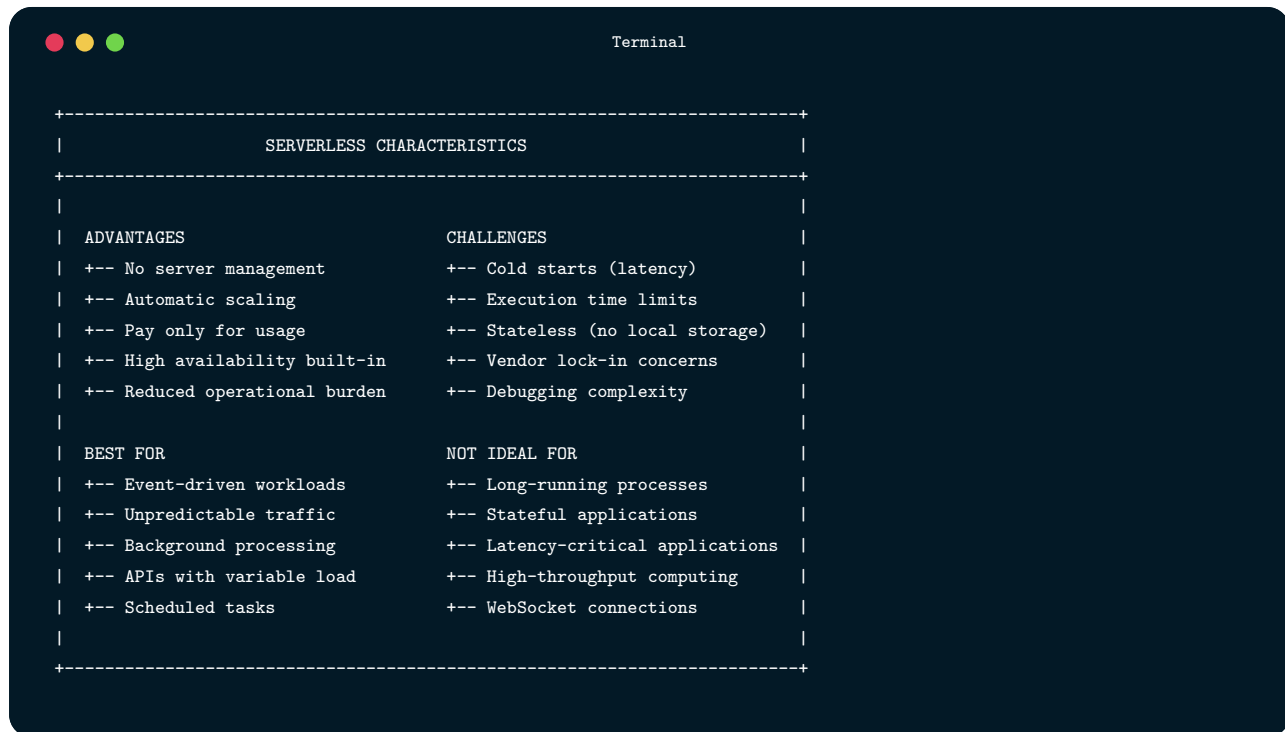
3.5

Serverless Computing

3.5.1 What is Serverless?

-

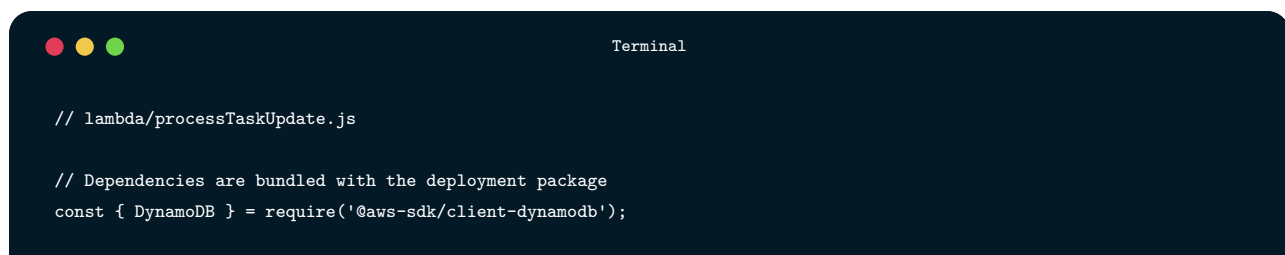
-
-
-



A terminal window titled "Terminal" with three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays a table with the title "SERVERLESS CHARACTERISTICS". The table is divided into two main sections: "ADVANTAGES" and "CHALLENGES". Under "ADVANTAGES", there are five items: "No server management", "Automatic scaling", "Pay only for usage", "High availability built-in", and "Reduced operational burden". Under "CHALLENGES", there are five items: "Cold starts (latency)", "Execution time limits", "Stateless (no local storage)", "Vendor lock-in concerns", and "Debugging complexity". Below these, there are two more sections: "BEST FOR" and "NOT IDEAL FOR". "BEST FOR" lists: "Event-driven workloads", "Unpredictable traffic", "Background processing", "APIs with variable load", and "Scheduled tasks". "NOT IDEAL FOR" lists: "Long-running processes", "Stateful applications", "Latency-critical applications", "High-throughput computing", and "WebSocket connections". The table is enclosed in a dashed border.

SERVERLESS CHARACTERISTICS	
ADVANTAGES	CHALLENGES
+-- No server management	+-- Cold starts (latency)
+-- Automatic scaling	+-- Execution time limits
+-- Pay only for usage	+-- Stateless (no local storage)
+-- High availability built-in	+-- Vendor lock-in concerns
+-- Reduced operational burden	+-- Debugging complexity
BEST FOR	NOT IDEAL FOR
+-- Event-driven workloads	+-- Long-running processes
+-- Unpredictable traffic	+-- Stateful applications
+-- Background processing	+-- Latency-critical applications
+-- APIs with variable load	+-- High-throughput computing
+-- Scheduled tasks	+-- WebSocket connections

3.5.2 AWS Lambda



A terminal window titled "Terminal" with three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays the following code:

```
// lambda/processTaskUpdate.js

// Dependencies are bundled with the deployment package
const { DynamoDB } = require('@aws-sdk/client-dynamodb');
```

Terminal -- continued

```

const { SNS } = require('@aws-sdk/client-sns');

// Initialize clients outside the handler
// These are reused across invocations (when warm)
const dynamodb = new DynamoDB({ region: 'us-east-1' });
const sns = new SNS({ region: 'us-east-1' });

/**
 * Lambda handler function
 *
 * @param {Object} event - Trigger event data (structure depends on trigger type)
 * @param {Object} context - Runtime information (function name, timeout, etc.)
 * @returns {Object} Response (structure depends on trigger type)
 */
exports.handler = async (event, context) => {
  console.log('Processing event:', JSON.stringify(event, null, 2));
  console.log('Remaining time:', context.getRemainingTimeInMillis(), 'ms');

  try {
    // Parse the incoming request (API Gateway format)
    const body = JSON.parse(event.body);
    const taskId = event.pathParameters?.taskId;

    // Validate input
    if (!taskId || !body.status) {
      return {
        statusCode: 400,
        headers: {
          'Content-Type': 'application/json',
          'Access-Control-Allow-Origin': '*' // CORS
        },
        body: JSON.stringify({ error: 'Missing taskId or status' })
      };
    }

    // Update task in DynamoDB
    const updateResult = await dynamodb.updateItem({
      TableName: process.env.TASKS_TABLE,
      Key: {
        taskId: { S: taskId }
      },
      UpdateExpression: 'SET #status = :status, updatedAt = :now',
      ExpressionAttributeNames: {
        '#status': 'status' // status is a reserved word
      },
      ExpressionAttributeValues: {
        ':status': { S: body.status },

```

Terminal -- continued

```
    ':now': { S: new Date().toISOString() }
  },
  ReturnValues: 'ALL_NEW'
});

// If task is completed, send notification
if (body.status === 'done') {
  await sns.publish({
    TopicArn: process.env.NOTIFICATIONS_TOPIC,
    Message: JSON.stringify({
      type: 'TASK_COMPLETED',
      taskId: taskId,
      timestamp: new Date().toISOString()
    }),
    MessageAttributes: {
      eventType: {
        DataType: 'String',
        StringValue: 'TASK_COMPLETED'
      }
    }
  });
}

// Return success response
return {
  statusCode: 200,
  headers: {
    'Content-Type': 'application/json',
    'Access-Control-Allow-Origin': '*'
  },
  body: JSON.stringify({
    message: 'Task updated successfully',
    task: unmarshallDynamoItem(updateResult.Attributes)
  })
};

} catch (error) {
  console.error('Error processing task update:', error);

  // Return error response
  return {
    statusCode: 500,
    headers: {
      'Content-Type': 'application/json',
      'Access-Control-Allow-Origin': '*'
    },
    body: JSON.stringify({ error: 'Internal server error' })
  };
}
```

Terminal -- continued

```
    };  
  }  
};  
  
// Helper to convert DynamoDB item format to plain object  
function unmarshallDynamoItem(item) {  
  const result = {};  
  for (const [key, value] of Object.entries(item)) {  
    if (value.S) result[key] = value.S;  
    else if (value.N) result[key] = Number(value.N);  
    else if (value.BOOL !== undefined) result[key] = value.BOOL;  
    else if (value.NULL) result[key] = null;  
  }  
  return result;  
}
```

3.5.3 Infrastructure as Code for Lambda

Terminal

```
# serverless.yml  
# Serverless Framework configuration  
  
service: taskflow-api  
  
# Use this specific version to avoid breaking changes  
frameworkVersion: '3'
```

Terminal -- continued

```

provider:
  name: aws
  runtime: nodejs20.x
  region: us-east-1
  stage: ${opt:stage, 'dev'} # Default to 'dev' if not specified

# Environment variables available to all functions
environment:
  TASKS_TABLE: ${self:service}-tasks-${self:provider.stage}
  NOTIFICATIONS_TOPIC:
    Ref: NotificationsTopic # Reference to CloudFormation resource

# IAM permissions for functions
iam:
  role:
    statements:
      - Effect: Allow
        Action:
          - dynamodb:GetItem
          - dynamodb:PutItem
          - dynamodb:UpdateItem
          - dynamodb>DeleteItem
          - dynamodb:Query
          - dynamodb:Scan
        Resource:
          - !GetAtt TasksTable.Arn
          - !Join ['/', [!GetAtt TasksTable.Arn, 'index/*']]
      - Effect: Allow
        Action:
          - sns:Publish
        Resource:
          - !Ref NotificationsTopic

# Lambda functions
functions:
  # Create task
  createTask:
    handler: src/handlers/tasks.create
    events:
      - http:
          path: tasks
          method: post
          cors: true

  # Get task
  getTask:
    handler: src/handlers/tasks.get

```


Terminal -- continued

```

events:
  - http:
      path: tasks/{taskId}
      method: get
      cors: true

# Update task
updateTask:
  handler: src/handlers/tasks.update
  events:
    - http:
        path: tasks/{taskId}
        method: patch
        cors: true

# Process notifications (triggered by SNS)
processNotification:
  handler: src/handlers/notifications.process
  events:
    - sns:
        arn: !Ref NotificationsTopic

# Increase timeout for notification processing
timeout: 30

# Scheduled cleanup of old tasks
cleanupOldTasks:
  handler: src/handlers/tasks.cleanup
  events:
    - schedule: rate(1 day) # Run daily
  timeout: 300 # 5 minutes for batch processing

# AWS resources to create
resources:
  Resources:
    # DynamoDB table for tasks
    TasksTable:
      Type: AWS::DynamoDB::Table
      Properties:
        TableName: ${self:provider.environment.TASKS_TABLE}
        BillingMode: PAY_PER_REQUEST # On-demand pricing
        AttributeDefinitions:
          - AttributeName: taskId
            AttributeType: S
          - AttributeName: userId
            AttributeType: S
          - AttributeName: status
            AttributeType: S

```

Terminal -- continued

```
KeySchema:
  - AttributeName: taskId
    KeyType: HASH
GlobalSecondaryIndexes:
  - IndexName: userId-status-index
    KeySchema:
      - AttributeName: userId
        KeyType: HASH
      - AttributeName: status
        KeyType: RANGE
    Projection:
      ProjectionType: ALL

# SNS topic for notifications
NotificationsTopic:
  Type: AWS::SNS::Topic
  Properties:
    TopicName: ${self:service}-notifications-${self:provider.stage}

plugins:
  - serverless-offline # Local development
  - serverless-webpack # Bundle and minimize code
```

3.5.4 Serverless Patterns

```

Terminal

+-----+
|          SERVERLESS ARCHITECTURE PATTERNS          |
+-----+
|
|  API BACKEND                                     |
|  API Gateway -> Lambda -> DynamoDB               |
|  Perfect for: CRUD APIs, mobile backends, webhooks |
|
|  EVENT PROCESSING                               |
|  S3 upload -> Lambda -> Process -> Store results   |
|  Perfect for: Image processing, data transformation, ETL |
|
|  STREAM PROCESSING                              |
|  Kinesis/SQS -> Lambda -> Process -> Store/Forward |
|  Perfect for: Real-time analytics, log processing, IoT data |
|
|  SCHEDULED JOBS                                 |
|  CloudWatch Events -> Lambda -> Perform task       |
|  Perfect for: Cleanup jobs, reports, data sync     |
|
|  FAN-OUT PATTERN                                |
|  SNS -> Multiple Lambda functions in parallel     |
|  Perfect for: Notifications, multi-target processing |
|
+-----+

```

```

Terminal

// lambda/processImageUpload.js

const { S3 } = require('@aws-sdk/client-s3');
const sharp = require('sharp');

const s3 = new S3({ region: 'us-east-1' });

// Define thumbnail sizes
const THUMBNAİL_SIZES = [
  { name: 'small', width: 150, height: 150 },
  { name: 'medium', width: 300, height: 300 },
  { name: 'large', width: 600, height: 600 }
];

/**

```

Terminal -- continued

```

* Triggered when an image is uploaded to the source bucket
* Creates thumbnails and stores them in the destination bucket
*/
exports.handler = async (event) => {
  console.log('Processing S3 event:', JSON.stringify(event, null, 2));

  // S3 events can contain multiple records (batch)
  const results = await Promise.all(
    event.Records.map(record => processImage(record))
  );

  return {
    processed: results.length,
    results
  };
};

async function processImage(record) {
  const bucket = record.s3.bucket.name;
  const key = decodeURIComponent(record.s3.object.key.replace(/\+/g, ' '));

  console.log(`Processing image: ${bucket}/${key}`);

  try {
    // Download original image
    const original = await s3.getObject({
      Bucket: bucket,
      Key: key
    });

    // Read image data into buffer
    const imageBuffer = await streamToBuffer(original.Body);

    // Generate thumbnails in parallel
    const thumbnails = await Promise.all(
      THUMBNAİL_SIZES.map(size => generateThumbnail(imageBuffer, size))
    );

    // Upload thumbnails to destination bucket
    await Promise.all(
      thumbnails.map((thumbnail, index) => {
        const size = THUMBNAİL_SIZES[index];
        const thumbnailKey = key.replace(
          /(\\.[^.]*)$/,
          `-${size.name}$1`
        );
      })
    );
  }
}

```

Terminal -- continued

```
    return s3.putObject({
      Bucket: process.env.DESTINATION_BUCKET,
      Key: thumbnailKey,
      Body: thumbnail,
      ContentType: 'image/jpeg'
    });
  })
};

console.log(`Successfully processed: ${key}`);
return { key, status: 'success' };

} catch (error) {
  console.error(`Error processing ${key}:`, error);
  return { key, status: 'error', error: error.message };
}
}

async function generateThumbnail(imageBuffer, { width, height }) {
  return sharp(imageBuffer)
    .resize(width, height, {
      fit: 'cover',
      position: 'center'
    })
    .jpeg({ quality: 80 })
    .toBuffer();
}

async function streamToBuffer(stream) {
  const chunks = [];
  for await (const chunk of stream) {
    chunks.push(chunk);
  }
  return Buffer.concat(chunks);
}
```

3.5.5 When to Use Serverless

```
Terminal

+-----+
|          SERVERLESS DECISION FRAMEWORK          |
+-----+
|
| CHOOSE SERVERLESS WHEN:
|
| PASS Traffic is unpredictable or spiky
|   -> Pay only for actual usage, automatic scaling
|
| PASS You want minimal operational overhead
|   -> No servers to patch, no capacity planning
|
| PASS Workloads are event-driven
|   -> Natural fit for triggers (HTTP, S3, queues, schedules)
|
| PASS Execution time is short (<15 minutes)
|   -> Lambda has a 15-minute maximum
|
| PASS Team is small and wants to focus on code
|   -> Reduces DevOps burden significantly
|
| CHOOSE CONTAINERS/VMS WHEN:
|
| FAIL Workloads are long-running
|   -> Lambda timeout limits; containers run indefinitely
|
| FAIL Latency is critical (sub-100ms consistently)
|   -> Cold starts add unpredictable latency
|
| FAIL Traffic is steady and predictable
|   -> Reserved capacity is often cheaper
|
| FAIL You need persistent connections (WebSockets)
|   -> Serverless functions are short-lived
|
| FAIL Vendor lock-in is a concern
|   -> Containers are portable; Lambda code requires changes
|
+-----+
```

3.6

Infrastructure as Code

3.6.1 Benefits of Infrastructure as Code



```
Terminal

+-----+
|          INFRASTRUCTURE AS CODE BENEFITS          |
+-----+
|
| REPRODUCIBILITY                                     |
| Create identical environments every time. Development, staging, and |
| production are truly equivalent, eliminating "works on my machine." |
|
| VERSION CONTROL                                     |
| Track every infrastructure change. See who changed what, when, and   |
| why. Roll back problematic changes by reverting commits.             |
|
| CODE REVIEW                                         |
| Infrastructure changes go through pull requests. Team members review |
| changes before they're applied, catching mistakes early.            |
|
| DOCUMENTATION                                       |
| The code IS the documentation. No more outdated wiki pages or       |
| forgotten manual steps.                                              |
|
| AUTOMATION                                          |
```

Terminal -- continued

```
| Apply changes automatically through CI/CD. No more manual clicking |
| through consoles or running scripts by hand. |
| |
| DISASTER RECOVERY |
| Recreate your entire infrastructure from code. If a region fails, |
| spin up everything in a new region quickly. |
| |
+-----+
```

3.6.2 Terraform

Terminal

```
# terraform/main.tf
# Terraform configuration for TaskFlow production infrastructure

terraform {
  required_version = ">= 1.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# Store state remotely for team collaboration
backend "s3" {
  bucket      = "taskflow-terraform-state"
  key         = "production/terraform.tfstate"
  region      = "us-east-1"
  encrypt     = true
  dynamodb_table = "terraform-locks" # Prevent concurrent modifications
}

provider "aws" {
  region = var.aws_region
}
```


Terminal -- continued

```
default_tags {
  tags = {
    Project      = "TaskFlow"
    Environment = var.environment
    ManagedBy    = "Terraform"
  }
}

# Variables allow configuration without code changes
variable "aws_region" {
  description = "AWS region to deploy to"
  type        = string
  default     = "us-east-1"
}

variable "environment" {
  description = "Environment name (e.g., production, staging)"
  type        = string
}

variable "app_instance_type" {
  description = "EC2 instance type for application servers"
  type        = string
  default     = "t3.medium"
}

variable "db_instance_class" {
  description = "RDS instance class"
  type        = string
  default     = "db.t3.medium"
}
```

```

# terraform/network.tf
# VPC and networking configuration

# Create a VPC with specified CIDR block
resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_hostnames = true
  enable_dns_support  = true

  tags = {
    Name = "taskflow-${var.environment}-vpc"
  }
}

# Create public subnets in multiple availability zones
resource "aws_subnet" "public" {
  count            = 2
  vpc_id           = aws_vpc.main.id
  cidr_block       = "10.0.${count.index + 1}.0/24"
  availability_zone = data.aws_availability_zones.available.names[count.index]
  map_public_ip_on_launch = true

  tags = {
    Name = "taskflow-${var.environment}-public-${count.index + 1}"
    Type = "Public"
  }
}

# Create private subnets for databases
resource "aws_subnet" "private" {
  count            = 2
  vpc_id           = aws_vpc.main.id
  cidr_block       = "10.0.${count.index + 10}.0/24"
  availability_zone = data.aws_availability_zones.available.names[count.index]

  tags = {
    Name = "taskflow-${var.environment}-private-${count.index + 1}"
    Type = "Private"
  }
}

# Internet gateway for public subnets
resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "taskflow-${var.environment}-igw"
  }
}

```

Terminal -- continued

```

    }
  }

  # Route table for public subnets
  resource "aws_route_table" "public" {
    vpc_id = aws_vpc.main.id

    route {
      cidr_block = "0.0.0.0/0"
      gateway_id = aws_internet_gateway.main.id
    }

    tags = {
      Name = "taskflow-${var.environment}-public-rt"
    }
  }

  # Associate public subnets with route table
  resource "aws_route_table_association" "public" {
    count          = length(aws_subnet.public)
    subnet_id      = aws_subnet.public[count.index].id
    route_table_id = aws_route_table.public.id
  }

  # Data source to get available AZs
  data "aws_availability_zones" "available" {
    state = "available"
  }

```

Terminal

```

# terraform/database.tf
# RDS PostgreSQL configuration

# Subnet group for RDS (must span multiple AZs)
resource "aws_db_subnet_group" "main" {
  name          = "taskflow-${var.environment}"
  subnet_ids    = aws_subnet.private[*].id
}

```

Terminal -- continued

```

tags = {
  Name = "taskflow-${var.environment}-db-subnet-group"
}
}

# Security group for RDS
resource "aws_security_group" "database" {
  name          = "taskflow-${var.environment}-db-sg"
  description   = "Security group for RDS database"
  vpc_id        = aws_vpc.main.id

  # Allow PostgreSQL from application security group only
  ingress {
    from_port     = 5432
    to_port       = 5432
    protocol      = "tcp"
    security_groups = [aws_security_group.application.id]
  }

  # No egress rules needed for RDS

  tags = {
    Name = "taskflow-${var.environment}-db-sg"
  }
}

# RDS PostgreSQL instance
resource "aws_db_instance" "main" {
  identifier = "taskflow-${var.environment}"

  # Engine configuration
  engine          = "postgres"
  engine_version  = "15.4"
  instance_class  = var.db_instance_class

  # Storage configuration
  allocated_storage    = 20
  max_allocated_storage = 100 # Enable storage autoscaling
  storage_type          = "gp3"
  storage_encrypted     = true

  # Database configuration
  db_name   = "taskflow"
  username = "taskflow_admin"
  password = var.db_password # From environment or secrets manager

  # Network configuration

```

Terminal -- continued

```

db_subnet_group_name = aws_db_subnet_group.main.name
vpc_security_group_ids = [aws_security_group.database.id]
publicly_accessible   = false # Only accessible from within VPC

# Backup configuration
backup_retention_period = 7
backup_window           = "03:00-04:00"
maintenance_window     = "Mon:04:00-Mon:05:00"

# High availability
multi_az = var.environment == "production" ? true : false

# Performance insights (monitoring)
performance_insights_enabled      = true
performance_insights_retention_period = 7

# Deletion protection
deletion_protection = var.environment == "production" ? true : false
skip_final_snapshot = var.environment != "production"

tags = {
  Name = "taskflow-${var.environment}-db"
}
}

```

Terminal

```

# terraform/outputs.tf
# Values to expose after apply

output "vpc_id" {
  description = "ID of the VPC"
  value       = aws_vpc.main.id
}

output "database_endpoint" {
  description = "RDS instance endpoint"
  value       = aws_db_instance.main.endpoint
  sensitive   = false
}

```

Terminal -- continued

```

}

output "database_connection_string" {
  description = "Database connection string"
  value       = "postgresql://${aws_db_instance.main.username}:PASSWORD@${aws_db_instance.main.endpoint}/${aws_db_instance.main.db_name}"
  sensitive   = true
}

```

3.6.3 Terraform Workflow

Terminal

```

# Initialize Terraform (download providers, set up backend)
terraform init

# Preview changes (don't apply yet)
terraform plan -var="environment=production"

# The plan shows what will be created, modified, or destroyed:
# + aws_upc.main will be created
# + aws_subnet.public[0] will be created
# + aws_subnet.public[1] will be created
# ...

# Apply changes (after reviewing plan)
terraform apply -var="environment=production"

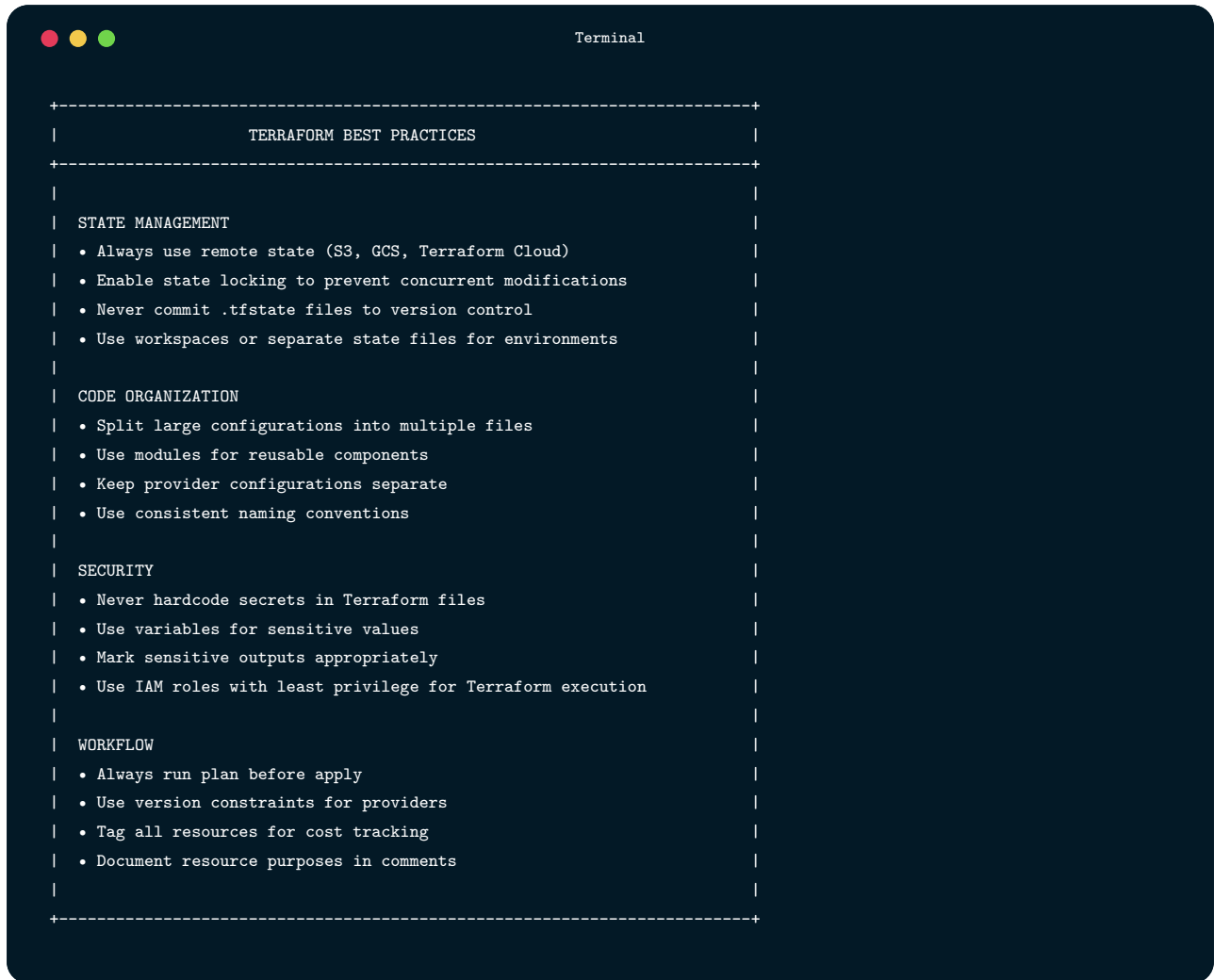
# Terraform prompts for confirmation before making changes
# Type 'yes' to proceed

# View current state
terraform show

# Destroy all resources (careful!)
terraform destroy -var="environment=staging"

```

3.6.4 Terraform Best Practices

A terminal window with a dark blue background and white text. The title bar at the top shows three colored circles (red, yellow, green) on the left and the word "Terminal" on the right. The content of the terminal is a list of Terraform best practices, enclosed in a dashed border. The practices are categorized into five sections: STATE MANAGEMENT, CODE ORGANIZATION, SECURITY, and WORKFLOW. Each section contains a list of specific recommendations.

```
+-----+
|               TERRAFORM BEST PRACTICES               |
+-----+
|
| STATE MANAGEMENT
| • Always use remote state (S3, GCS, Terraform Cloud)
| • Enable state locking to prevent concurrent modifications
| • Never commit .tfstate files to version control
| • Use workspaces or separate state files for environments
|
| CODE ORGANIZATION
| • Split large configurations into multiple files
| • Use modules for reusable components
| • Keep provider configurations separate
| • Use consistent naming conventions
|
| SECURITY
| • Never hardcode secrets in Terraform files
| • Use variables for sensitive values
| • Mark sensitive outputs appropriately
| • Use IAM roles with least privilege for Terraform execution
|
| WORKFLOW
| • Always run plan before apply
| • Use version constraints for providers
| • Tag all resources for cost tracking
| • Document resource purposes in comments
|
+-----+
```

3.7

Cloud Security Best Practices

3.7.1 Identity and Access Management

```
Terminal

+-----+
|               IAM BEST PRACTICES               |
+-----+
|
|  USERS AND ROLES                               |
|  • Never use root account for daily operations |
|  • Create individual IAM users for each person |
|  • Use roles for applications (not access keys)|
|  • Require MFA for all human users             |
|
|  PERMISSIONS                                   |
|  • Start with no permissions, add only what's  |
|    needed                                     |
|  • Use AWS managed policies where appropriate |
|  • Scope permissions to specific resources when|
|    possible                                   |
|  • Regularly audit and remove unused permissions|
|
|  CREDENTIALS                                   |
|  • Rotate access keys regularly               |
|  • Never embed credentials in code             |
|  • Use temporary credentials (STS) when possible|
|  • Store secrets in Secrets Manager or Parameter|
|    Store                                       |
+-----+
```

```
Terminal

{
  "Version": "2012-10-17",
  "Statement": [
    {
```


Terminal -- continued

```
"Sid": "AllowDynamoDBAccess",
"Effect": "Allow",
"Action": [
  "dynamodb:GetItem",
  "dynamodb:PutItem",
  "dynamodb:UpdateItem",
  "dynamodb:DeleteItem",
  "dynamodb:Query"
],
"Resource": [
  "arn:aws:dynamodb:us-east-1:123456789012:table/taskflow-tasks",
  "arn:aws:dynamodb:us-east-1:123456789012:table/taskflow-tasks/index/*"
]
},
{
  "Sid": "AllowS3Access",
  "Effect": "Allow",
  "Action": [
    "s3:GetObject",
    "s3:PutObject"
  ],
  "Resource": "arn:aws:s3:::taskflow-uploads/*"
}
]
}
```

3.7.2 Secrets Management

Terminal

```
// Using AWS Secrets Manager

const { SecretsManager } = require('@aws-sdk/client-secrets-manager');

const secretsManager = new SecretsManager({ region: 'us-east-1' });
```

Terminal -- continued

```
// Cache secrets to avoid repeated API calls
let cachedSecrets = null;
let cacheExpiry = 0;
const CACHE_DURATION = 300000; // 5 minutes

async function getSecrets() {
  // Return cached secrets if still valid
  if (cachedSecrets && Date.now() < cacheExpiry) {
    return cachedSecrets;
  }

  // Fetch secrets from Secrets Manager
  const response = await secretsManager.getSecretValue({
    SecretId: 'taskflow/production'
  });

  // Parse JSON secrets
  cachedSecrets = JSON.parse(response.SecretString);
  cacheExpiry = Date.now() + CACHE_DURATION;

  return cachedSecrets;
}

// Usage
async function connectToDatabase() {
  const secrets = await getSecrets();

  return new Pool({
    host: secrets.DB_HOST,
    database: secrets.DB_NAME,
    user: secrets.DB_USER,
    password: secrets.DB_PASSWORD,
    ssl: true
  });
}
```

3.7.3 Network Security

```
Terminal

+-----+
|           NETWORK SECURITY LAYERS           |
+-----+
|
| LAYER 1: VPC ISOLATION                      |
| • Resources in private subnets have no public IP |
| • NAT Gateway for outbound internet access only |
| • VPC Flow Logs for traffic monitoring          |
|
| LAYER 2: SECURITY GROUPS                    |
| • Stateful firewall at instance level          |
| • Allow only required ports from required sources |
| • Reference other security groups (not IP ranges when possible) |
|
| LAYER 3: NETWORK ACLS                      |
| • Stateless firewall at subnet level          |
| • Additional layer for sensitive subnets      |
| • Deny rules for known bad actors            |
|
| LAYER 4: APPLICATION SECURITY              |
| • TLS everywhere (even internal traffic)      |
| • Input validation                          |
| • WAF for public endpoints                  |
|
+-----+
```

3.8

Cost Optimization

3.8.1 Understanding Cloud Pricing

-
-
-
-

3.8.2 Cost Optimization Strategies

```
Terminal

+-----+
| COST OPTIMIZATION STRATEGIES |
+-----+
|
| RIGHT-SIZING
| • Monitor actual resource utilization
| • Downsize over-provisioned instances
| • Use auto-scaling instead of provisioning for peak
|
| PRICING MODELS
| • Spot instances for fault-tolerant workloads (70-90% savings)
| • Reserved instances for steady workloads (30-60% savings)
| • Savings Plans for flexible commitments
|
| ARCHITECTURE
| • Use serverless for variable workloads
| • Cache aggressively to reduce database load
| • Compress data to reduce storage and transfer costs
|
| GOVERNANCE
| • Tag resources for cost allocation
| • Set up billing alerts
| • Regular cost reviews
| • Delete unused resources automatically
|
+-----+
```

```
Terminal

# terraform/cost-controls.tf

# Create a budget alert
resource "aws_budgets_budget" "monthly" {
  name           = "taskflow-${var.environment}-monthly"
  budget_type    = "COST"
  limit_amount   = var.monthly_budget_limit
  limit_unit     = "USD"
  time_unit      = "MONTHLY"

  notification {
    comparison_operator = "GREATER_THAN"
    threshold           = 80
    threshold_type      = "PERCENTAGE"
    notification_type    = "ACTUAL"
    subscriber_email_addresses = var.alert_email_addresses
  }

  notification {
    comparison_operator = "GREATER_THAN"
    threshold           = 100
    threshold_type      = "PERCENTAGE"
    notification_type    = "FORECASTED"
    subscriber_email_addresses = var.alert_email_addresses
  }
}

# Lambda to clean up old resources
resource "aws_lambda_function" "cleanup" {
  function_name = "taskflow-resource-cleanup"
  handler       = "cleanup.handler"
  runtime       = "nodejs20.x"

  # Run weekly
  # (CloudWatch Events rule not shown)

  environment {
    variables = {
      MAX_SNAPSHOT_AGE_DAYS = "30"
      MAX_LOG_RETENTION_DAYS = "90"
    }
  }
}
```

3.9

Chapter Summary

3.10

Key Terms

IaaS	Infrastructure as a Service—virtual machines, storage, networking
SaaS	Software as a Service—complete applications delivered over the internet
Docker	Platform for building, running, and distributing containers
Pod	Smallest deployable unit in Kubernetes; one or more containers
Service	Kubernetes resource providing stable network endpoint for pods
Lambda	AWS serverless computing service for running functions
IaC	Infrastructure as Code—managing infrastructure through code
VPC	Virtual Private Cloud—isolated network within cloud provider

3.11

Review Questions

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.

3.12

Hands-On Exercises

3.12.1 Exercise 11.1: Containerize Your Application

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

3.12.2 Exercise 11.2: Deploy to Kubernetes

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

3.12.3 Exercise 11.3: Serverless Function

- 1.
- 2.
- 3.
- 4.
- 5.

3.12.4 Exercise 11.4: Infrastructure as Code

- 1.
- 2.
- 3.
- 4.

5.

3.12.5 Exercise 11.5: Security Audit

- 1.
- 2.
- 3.
- 4.
- 5.

3.12.6 Exercise 11.6: Cost Analysis

- 1.
- 2.
- 3.
- 4.
- 5.

3.13

Further Reading

-
-
-
-

-
-
-
-

3.14

References

CHAPTER

04

SECURITY

Software Security

Integrating secure design, implementation, verification, and operational practice.

LEARNING OBJECTIVES

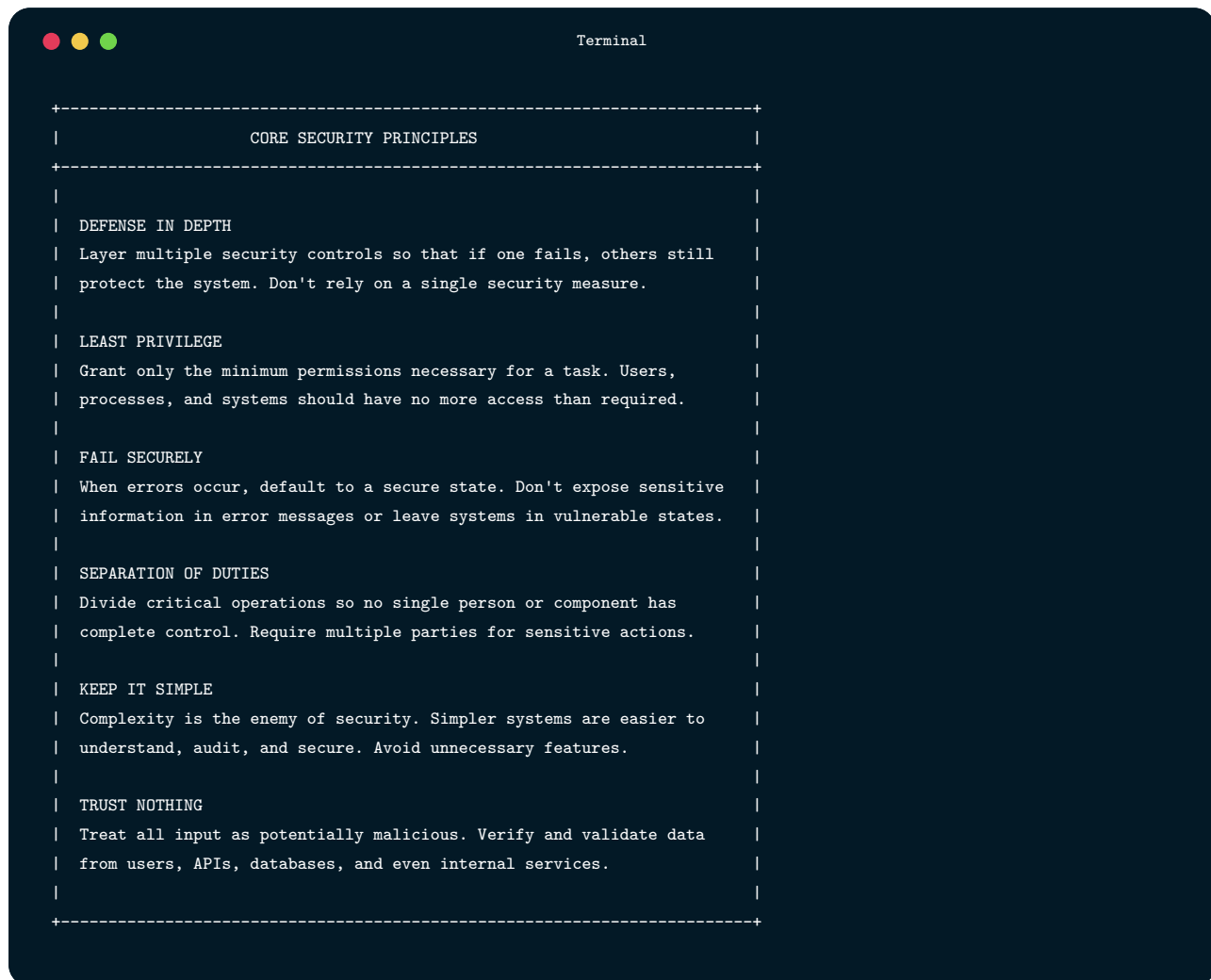
- ✓ By the end of this chapter, you will be able to:
 - ✓ Explain the importance of security as a fundamental software quality attribute
 - ✓ Identify and mitigate the OWASP Top 10 web application vulnerabilities
 - ✓ Implement secure authentication and authorization mechanisms
 - ✓ Apply secure coding practices to prevent common vulnerabilities
 - ✓ Properly validate and sanitize user input to prevent injection attacks
 - ✓ Use encryption appropriately for data at rest and in transit
 - ✓ Configure security headers to protect against client-side attacks
 - ✓ Establish a vulnerability management process for dependencies
 - ✓ Design and execute security testing strategies
 - ✓ Respond effectively to security incidents

4.1

The Imperative of Software Security

4.1.1 The Cost of Security Failures

4.1.2 Security Principles

A terminal window with a dark blue background and white text. The title bar at the top says "Terminal" and has three colored window control buttons (red, yellow, green) on the left. The content is a list of security principles, each preceded by a dashed line and a vertical bar. The principles are: DEFENSE IN DEPTH, LEAST PRIVILEGE, FAIL SECURELY, SEPARATION OF DUTIES, KEEP IT SIMPLE, and TRUST NOTHING. Each principle is followed by a brief explanation. The list is enclosed in a dashed border.

```
+-----+
|               CORE SECURITY PRINCIPLES               |
+-----+
|
|  DEFENSE IN DEPTH
|  Layer multiple security controls so that if one fails, others still
|  protect the system. Don't rely on a single security measure.
|
|  LEAST PRIVILEGE
|  Grant only the minimum permissions necessary for a task. Users,
|  processes, and systems should have no more access than required.
|
|  FAIL SECURELY
|  When errors occur, default to a secure state. Don't expose sensitive
|  information in error messages or leave systems in vulnerable states.
|
|  SEPARATION OF DUTIES
|  Divide critical operations so no single person or component has
|  complete control. Require multiple parties for sensitive actions.
|
|  KEEP IT SIMPLE
|  Complexity is the enemy of security. Simpler systems are easier to
|  understand, audit, and secure. Avoid unnecessary features.
|
|  TRUST NOTHING
|  Treat all input as potentially malicious. Verify and validate data
|  from users, APIs, databases, and even internal services.
|
+-----+
```

4.1.3 The Security Mindset

4.2

OWASP Top 10 Web Application Vulnerabilities



4.2.1 A01: Broken Access Control

Understanding Access Control Failures

Implementing Proper Access Control

```
Terminal

const jwt = require('jsonwebtoken');

const authenticate = async (req, res, next) => {
  try {
    // Extract token from Authorization header
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({
        error: 'Authentication required. Please provide a valid token.'
      });
    }

    const token = authHeader.substring(7); // Remove 'Bearer ' prefix

    // Verify token signature and extract payload
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // Load full user record from database
    // Don't rely solely on token contents - verify user still exists and is active
    const user = await db('users')
      .where('id', decoded.userId)
      .where('is_active', true)
      .first();

    if (!user) {
      return res.status(401).json({
        error: 'User account not found or inactive.'
      });
    }

    // Attach user to request for downstream handlers
    req.user = user;
    next();
  } catch (error) {
```

Terminal -- continued

```
if (error.name === 'TokenExpiredError') {
  return res.status(401).json({ error: 'Token has expired. Please log in again.' });
}
if (error.name === 'JsonWebTokenError') {
  return res.status(401).json({ error: 'Invalid token.' });
}
next(error);
}
};
```

Terminal

```
// Middleware to verify resource ownership
const authorizeOwner = (resourceUserIdField = 'userId') => {
  return async (req, res, next) => {
    const resourceUserId = parseInt(req.params[resourceUserIdField]);

    // Users can access their own resources
    if (req.user.id === resourceUserId) {
      return next();
    }

    // Administrators can access any resource
    if (req.user.role === 'admin') {
      return next();
    }

    // Log unauthorized access attempts for security monitoring
    console.warn('Authorization failure:', {
      attemptedBy: req.user.id,
      attemptedResource: resourceUserId,
      path: req.path,
      timestamp: new Date().toISOString()
    });

    return res.status(403).json({
```

Terminal -- continued

```
      error: 'You do not have permission to access this resource.'
    });
  };
};

// Middleware to require specific roles
const requireRole = (...allowedRoles) => {
  return (req, res, next) => {
    if (!allowedRoles.includes(req.user.role)) {
      console.warn('Role authorization failure:', {
        user: req.user.id,
        userRole: req.user.role,
        requiredRoles: allowedRoles,
        path: req.path
      });

      return res.status(403).json({
        error: 'You do not have sufficient privileges for this action.'
      });
    }
    next();
  };
};
```

Terminal

```
// User can only access their own profile
app.get('/api/users/:userId/profile',
  authenticate, // First, verify who they are
  authorizeOwner('userId'), // Then, verify they can access this resource
  async (req, res) => {
    const user = await db('users')
      .where('id', req.params.userId)
```

Terminal -- continued

```
.select('id', 'name', 'email', 'created_at') // Never return password_hash!
.first();

if (!user) {
  return res.status(404).json({ error: 'User not found' });
}

res.json({ data: user });
}
);

// Only administrators can view all users
app.get('/api/admin/users',
  authenticate,
  requireRole('admin'),
  async (req, res) => {
    const users = await db('users')
      .select('id', 'name', 'email', 'role', 'created_at')
      .orderBy('created_at', 'desc');

    res.json({ data: users });
  }
);
```

Terminal

```
// Creating a task - server controls ownership
app.post('/api/tasks',
  authenticate,
  validate(taskSchema),
  async (req, res) => {
    const task = await db('tasks').insert({
      title: req.body.title,
      description: req.body.description,
      user_id: req.user.id, // Always use authenticated user, never req.body.userId
      status: 'todo',
      created_at: new Date()
    }).returning('*');
  });
```

Terminal -- continued

```
    res.status(201).json({ data: task[0] });  
  }  
);
```

4.2.2 A02: Cryptographic Failures

Understanding Cryptographic Requirements

Password Hashing Done Right



Terminal

```
const bcrypt = require('bcrypt');

// Work factor of 12 means 2^12 = 4096 iterations
// This takes about 250ms on modern hardware
// Increase this as computers get faster
const SALT_ROUNDS = 12;

async function hashPassword(plainPassword) {
  // bcrypt generates a random salt and includes it in the output
  // The result is a 60-character string containing:
  // - Algorithm identifier ($2b$)
  // - Work factor (12)
  // - Salt (22 characters)
  // - Hash (31 characters)
  return bcrypt.hash(plainPassword, SALT_ROUNDS);
}

async function verifyPassword(plainPassword, storedHash) {
  // bcrypt extracts the salt from the stored hash,
  // hashes the provided password with that salt,
  // and compares the results in constant time
  return bcrypt.compare(plainPassword, storedHash);
}
```

Encrypting Sensitive Data

```
Terminal

const crypto = require('crypto');

const ALGORITHM = 'aes-256-gcm';
const IV_LENGTH = 12; // 96 bits recommended for GCM
const AUTH_TAG_LENGTH = 16; // 128 bits

function encrypt(plaintext, key) {
  // The initialization vector (IV) must be unique for each encryption
  // with the same key. GCM mode is catastrophically broken if you
  // reuse an IV with the same key - it can reveal the key itself.
  const iv = crypto.randomBytes(IV_LENGTH);

  // Create cipher using authenticated encryption mode
  const cipher = crypto.createCipheriv(ALGORITHM, key, iv);

  // Encrypt the data
  let encrypted = cipher.update(plaintext, 'utf8', 'hex');
  encrypted += cipher.final('hex');

  // Get the authentication tag - this ensures integrity
  const authTag = cipher.getAuthTag();

  // Return everything needed for decryption
  // IV + AuthTag + Ciphertext
  return iv.toString('hex') + authTag.toString('hex') + encrypted;
}

function decrypt(encryptedData, key) {
  // Extract components from the combined string
  const iv = Buffer.from(encryptedData.slice(0, IV_LENGTH * 2), 'hex');
  const authTag = Buffer.from(
    encryptedData.slice(IV_LENGTH * 2, IV_LENGTH * 2 + AUTH_TAG_LENGTH * 2),
```


Terminal -- continued

```
'hex'
);
const ciphertext = encryptedData.slice(IV_LENGTH * 2 + AUTH_TAG_LENGTH * 2);

// Create decipher and set authentication tag
const decipher = crypto.createDecipheriv(ALGORITHM, key, iv);
decipher.setAuthTag(authTag);

// Decrypt - this will throw if authentication fails
// (i.e., if the ciphertext has been tampered with)
let decrypted = decipher.update(ciphertext, 'hex', 'utf8');
decrypted += decipher.final('utf8');

return decrypted;
}
```

4.2.3 A03: Injection

Understanding SQL Injection

```
Terminal

// VULNERABLE CODE - DO NOT USE
async function checkLogin(email, password) {
  const query = `SELECT * FROM users WHERE email = '${email}' AND password = '${password}'`;
  const result = await db.raw(query);
  return result.rows[0];
}
```

```
Terminal

SELECT * FROM users WHERE email = 'alice@example.com' AND password = 'secretpassword'
```

```
Terminal

SELECT * FROM users WHERE email = '' OR '1'='1' --' AND password = '...'
```

-
-
-

```
Terminal

'; DROP TABLE users; --
```



Terminal

```
' UNION SELECT password_hash FROM users WHERE email = 'admin@example.com' --
```

Preventing SQL Injection



Terminal

```
// SECURE: Using parameterized queries
async function checkLogin(email, password) {
  // The ? placeholders are filled by the driver, which properly escapes values
  const result = await db.raw(
    'SELECT * FROM users WHERE email = ? AND password_hash = ?',
    [email, password]
  );
  return result.rows[0];
}
```



Terminal

```
// Using Knex query builder - automatically parameterized
async function getUserByEmail(email) {
  return db('users')
    .where('email', email) // Knex parameterizes this automatically
    .first();
}

// Complex queries remain safe
async function searchTasks(userId, searchTerm, status) {
  return db('tasks')
```

Terminal -- continued

```
.where('user_id', userId)
.where('title', 'like', `%${searchTerm}%`) // Still parameterized
.modify((query) => {
  if (status) {
    query.where('status', status);
  }
})
.orderBy('created_at', 'desc');
}
```

Command Injection

Terminal

```
// VULNERABLE: User controls part of shell command
app.post('/api/ping', (req, res) => {
  const { host } = req.body;
  exec(`ping -c 1 ${host}`, (error, stdout) => {
    res.send(stdout);
  });
});

// Attack: host = "example.com; cat /etc/passwd"
// Executes: ping -c 1 example.com; cat /etc/passwd
```

Terminal

```
// SECURE: Arguments passed as array, not interpolated into string
const { execFile } = require('child_process');

app.post('/api/ping', (req, res) => {
```

Terminal -- continued

```
const { host } = req.body;

// Validate input format
if (!/^[a-zA-Z0-9.-]+$/ .test(host)) {
  return res.status(400).json({ error: 'Invalid host format' });
}

// execFile doesn't invoke a shell, and arguments are passed separately
execFile('ping', ['-c', '1', host], (error, stdout) => {
  res.send(stdout);
});
});
```

4.2.4 A04: Insecure Design

Design-Level Security Thinking

Secure Design for Password Reset

-
-
-
-
-
-

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.

```
Terminal

const crypto = require('crypto');

// Rate limiting at multiple levels
const requestResetLimiter = rateLimit({
  windowMs: 60 * 60 * 1000, // 1 hour
  max: 3,                    // 3 requests per IP per hour
  message: { error: 'Too many requests. Please try again later.' }
});

const resetTokenLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 5,                  // 5 attempts to use token
```

Terminal -- continued

```
    message: { error: 'Too many attempts. Please request a new reset link.' }  
  });
```

Terminal

```
app.post('/api/auth/forgot-password', requestResetLimiter, async (req, res) => {  
  const { email } = req.body;  
  
  // IMPORTANT: Always return the same response regardless of whether  
  // the email exists. This prevents email enumeration attacks.  
  const genericResponse = {  
    message: 'If an account exists with this email, you will receive a reset link.'  
  };  
  
  const user = await db('users').where('email', email.toLowerCase()).first();  
  
  if (!user) {  
    // Add artificial delay to match timing of successful requests  
    await new Promise(r => setTimeout(r, 100));  
    return res.json(genericResponse);  
  }  
}
```

Terminal

```
// Generate cryptographically secure token  
// 32 bytes = 256 bits of entropy - infeasible to brute force  
const resetToken = crypto.randomBytes(32).toString('hex');  
  
// Store HASH of token, not the token itself  
// If database is compromised, attacker still can't use the hashes  
const tokenHash = crypto.createHash('sha256').update(resetToken).digest('hex');  
  
await db('password_resets').insert({  
  user_id: user.id,  
  token_hash: tokenHash,
```

Terminal -- continued

```

    expires_at: new Date(Date.now() + 60 * 60 * 1000), // 1 hour
    created_at: new Date()
  });

  // Send unhashed token in email
  await sendEmail({
    to: user.email,
    subject: 'Password Reset Request',
    html: `
      <p>Click below to reset your password. This link expires in 1 hour.</p>
      <a href="https://yourapp.com/reset-password?token=${resetToken}">
        Reset Password
      </a>
      <p>If you didn't request this, you can safely ignore this email.</p>
    `
  });

  res.json(genericResponse);
});

```

Terminal

```

app.post('/api/auth/reset-password', resetTokenLimiter, async (req, res) => {
  const { token, newPassword } = req.body;

  // Hash the provided token to compare with stored hash
  const tokenHash = crypto.createHash('sha256').update(token).digest('hex');

  const resetRequest = await db('password_resets')
    .where('token_hash', tokenHash)
    .where('expires_at', '>', new Date())
    .where('used_at', null) // Single-use check
    .first();

  if (!resetRequest) {
    return res.status(400).json({
      error: 'Invalid or expired reset link.'
    });
  }
}

```



```
Terminal

// Validate new password meets requirements
const passwordErrors = validatePasswordStrength(newPassword);
if (passwordErrors.length > 0) {
  return res.status(400).json({ error: passwordErrors[0] });
}

const passwordHash = await bcrypt.hash(newPassword, 12);

// Use transaction to ensure all changes succeed or none do
await db.transaction(async (trx) => {
  // Update password
  await trx('users')
    .where('id', resetRequest.user_id)
    .update({ password_hash: passwordHash });

  // Mark token as used (single-use enforcement)
  await trx('password_resets')
    .where('id', resetRequest.id)
    .update({ used_at: new Date() });

  // Invalidate ALL sessions for this user
  // If attacker had access, they're now locked out
  await trx('sessions')
    .where('user_id', resetRequest.user_id)
    .delete();

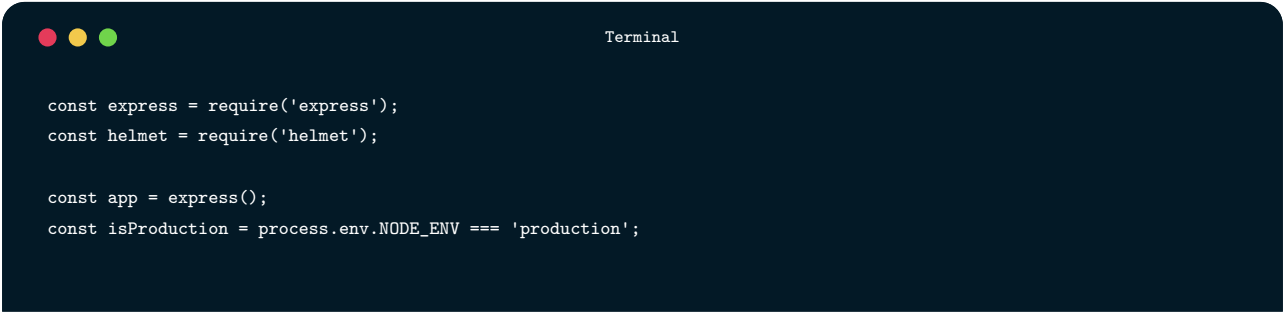
  await trx('refresh_tokens')
    .where('user_id', resetRequest.user_id)
    .update({ revoked_at: new Date() });
});

res.json({ message: 'Password updated successfully. Please log in.' });
});
```

4.2.5 A05: Security Misconfiguration

Common Misconfiguration Patterns

Secure Configuration

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the word "Terminal". The terminal displays four lines of JavaScript code for configuring an Express application with helmet and checking the production environment.

```
const express = require('express');  
const helmet = require('helmet');  
  
const app = express();  
const isProduction = process.env.NODE_ENV === 'production';
```

Terminal -- continued

```
// Helmet sets many security headers with sensible defaults
app.use(helmet());
```

Terminal

```
app.use(helmet({
  // Content Security Policy - controls which resources can load
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"], // Only scripts from our domain
      styleSrc: ["'self'", "unsafe-inline"], // Styles from our domain
      imgSrc: ["'self'", "data:", "https:"], // Images from anywhere over HTTPS
      connectSrc: ["'self'", "https://api.ourapp.com"], // API connections
      fontSrc: ["'self'"],
      objectSrc: ["'none'"], // No Flash, Java applets, etc.
      frameAncestors: ["'none'"], // Can't be embedded in frames
      upgradeInsecureRequests: [], // Upgrade HTTP to HTTPS
    },
  },
  // Force HTTPS for one year, including subdomains
  hsts: {
    maxAge: 31536000,
    includeSubDomains: true,
    preload: true,
  },
}));
```

```
Terminal

// CORS configuration - allow only specific origins
const corsOptions = {
  origin: isProduction
    ? ['https://ourapp.com', 'https://www.ourapp.com']
    : ['http://localhost:3000'],
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization'],
  credentials: true, // Allow cookies
  maxAge: 86400, // Cache preflight for 24 hours
};
app.use(cors(corsOptions));

// Don't reveal technology stack
app.disable('x-powered-by');

// Limit request body size to prevent DoS
app.use(express.json({ limit: '10kb' }));
app.use(express.urlencoded({ extended: true, limit: '10kb' }));
```

```
Terminal

// Error handling middleware
app.use((err, req, res, next) => {
  // Always log full error details for debugging
  console.error('Error:', {
    message: err.message,
    stack: err.stack,
    path: req.path,
    method: req.method,
    ip: req.ip,
    user: req.user?.id
  });

  // Determine what to send to client
  const statusCode = err.statusCode || 500;

  if (isProduction) {
```

Terminal -- continued

```
// In production, never expose internals
const safeMessage = statusCode >= 500
  ? 'An unexpected error occurred' // Generic for server errors
  : err.message; // Client errors are usually safe to show

res.status(statusCode).json({
  error: { message: safeMessage }
});
} else {
  // In development, show everything for debugging
  res.status(statusCode).json({
    error: {
      message: err.message,
      stack: err.stack,
      details: err.details
    }
  });
}
});
```

4.2.6 A06: Vulnerable and Outdated Components

The Scale of the Problem

Managing Dependency Security

```
Terminal

# List all dependencies and their versions
npm list --all

# Output includes the dependency tree:
# taskflow-api@1.0.0
# +- bcrypt@5.1.0
# |   +- @mapbox/node-pre-gyp@1.0.10
# |   |   +- detect-libc@2.0.1
# |   |   +- https-proxy-agent@5.0.1
# |   |   +- ...
```

```
Terminal

# Built-in npm audit
npm audit

# Output shows vulnerabilities by severity:
#                                     Manual Review
#           Critical  High  Moderate  Low
# Dependency    0      2      5      3
```

Terminal -- continued

```
#  
# Run `npm audit fix` to attempt automatic fixes
```

Terminal

```
# .github/workflows/security.yml  
name: Security Scan  
  
on:  
  push:  
    branches: [main, develop]  
  schedule:  
    - cron: '0 0 * * *' # Daily scan catches newly disclosed vulnerabilities  
  
jobs:  
  dependency-scan:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
  
      - name: Run npm audit  
        run: npm audit --audit-level=high  
  
      - name: Run Snyk scan  
        uses: snyk/actions/node@master  
        env:  
          SNYK_TOKEN: ${ secrets.SNYK_TOKEN }  
        with:  
          args: --severity-threshold=high
```



Terminal

```
# See which packages have updates available
npm outdated

# Package      Current  Wanted  Latest
# express      4.17.1   4.17.3   4.18.2
# lodash        4.17.19  4.17.21  4.17.21
# jsonwebtoken  8.5.1    8.5.1    9.0.0

# Update to latest compatible versions (respects semver in package.json)
npm update

# Update major versions (may have breaking changes)
npm install express@latest
```

4.2.7 A07: Identification and Authentication Failures

Password Policies

-
-
-
-
-


```
Terminal

const crypto = require('crypto');
const https = require('https');

async function isPasswordBreached(password) {
  // Hash the password with SHA-1 (required by HaveIBeenPwned API)
  const hash = crypto.createHash('sha1')
    .update(password)
    .digest('hex')
    .toUpperCase();

  // Send only first 5 characters to the API (k-anonymity)
  // This means the API never sees the full hash
  const prefix = hash.substring(0, 5);
  const suffix = hash.substring(5);

  // Query the HaveIBeenPwned API
  const response = await fetch(`https://api.pwnedpasswords.com/range/${prefix}`);
  const text = await response.text();

  // Response contains all hash suffixes with that prefix
  // Check if our suffix is in the list
  const lines = text.split('\n');
  for (const line of lines) {
    const [hashSuffix, count] = line.split(':');
    if (hashSuffix === suffix) {
      return true; // Password has been breached
    }
  }

  return false;
}
```

Brute Force Protection

```
Terminal

const loginAttempts = new Map(); // In production, use Redis for distributed systems

async function checkBruteForce(email, ip) {
  const key = `${email}:${ip}`;
  const attempts = loginAttempts.get(key) || { count: 0, blockedUntil: null };

  // Check if currently blocked
  if (attempts.blockedUntil && attempts.blockedUntil > Date.now()) {
    const waitMinutes = Math.ceil((attempts.blockedUntil - Date.now()) / 60000);
    throw new Error(`Too many attempts. Try again in ${waitMinutes} minutes.`);
  }

  return attempts;
}

async function recordLoginAttempt(email, ip, success) {
  const key = `${email}:${ip}`;

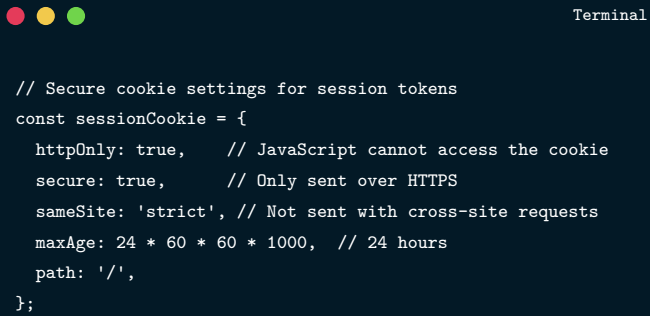
  if (success) {
    // Clear attempts on successful login
    loginAttempts.delete(key);
    return;
  }

  // Increment failed attempts
  const attempts = loginAttempts.get(key) || { count: 0, blockedUntil: null };
  attempts.count++;

  // Progressive lockout: longer blocks for more attempts
  if (attempts.count >= 10) {
    attempts.blockedUntil = Date.now() + 60 * 60 * 1000; // 1 hour
  } else if (attempts.count >= 5) {
    attempts.blockedUntil = Date.now() + 15 * 60 * 1000; // 15 minutes
  } else if (attempts.count >= 3) {
    attempts.blockedUntil = Date.now() + 1 * 60 * 1000; // 1 minute
  }

  loginAttempts.set(key, attempts);
}
```

Secure Session Management



```
// Secure cookie settings for session tokens
const sessionCookie = {
  httpOnly: true,    // JavaScript cannot access the cookie
  secure: true,      // Only sent over HTTPS
  sameSite: 'strict', // Not sent with cross-site requests
  maxAge: 24 * 60 * 60 * 1000, // 24 hours
  path: '/',
};
```

4.2.8 A08: Software and Data Integrity Failures

Supply Chain Security

-
-

-
-
-
-

```
Terminal

// package-lock.json includes integrity hashes
{
  "packages": {
    "node_modules/express": {
      "version": "4.18.2",
      "resolved": "https://registry.npmjs.org/express/-/express-4.18.2.tgz",
      "integrity": "sha512-5/psL6iGPdfQ/1KM1UuielYgv3BUoJfz1aUwU9vHZ+J7gyvwdQXFEBIEIaxeGf0GIcreATNyBExtalisDbuMqQ=="
    }
  }
}
```

Unsafe Deserialization

```
Terminal

// DANGEROUS: Libraries that deserialize with code execution
const nodeSerialize = require('node-serialize');

app.post('/api/data', (req, res) => {
  // This can execute arbitrary code!
  const data = nodeSerialize.unserialize(req.body.payload);
}
```

Terminal -- continued

```
    res.json(data);
  });

  // Attack payload: Functions embedded in serialized data
  // get executed during deserialization
```

Terminal

```
// SAFE: JSON.parse only creates data, never executes code
app.post('/api/data', (req, res) => {
  const data = JSON.parse(req.body.payload);

  // Still validate the structure!
  const validated = dataSchema.validate(data);
  if (validated.error) {
    return res.status(400).json({ error: 'Invalid data format' });
  }

  res.json(validated.value);
});
```

4.2.9 A09: Security Logging and Monitoring Failures

What to Log

How to Log Securely



Terminal

```
const winston = require('winston');

const securityLogger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    winston.format.json()
  ),
  transports: [
    new winston.transports.File({ filename: 'security.log' })
  ]
});

function logSecurityEvent(eventType, userId, details) {
  securityLogger.info({
    eventType,
    userId,
    timestamp: new Date().toISOString(),
    ...details,
    // Never include passwords, tokens, or other secrets!
  });
}
```

Terminal -- continued

```
}

// Usage examples
logSecurityEvent('LOGIN_SUCCESS', user.id, { ip: req.ip });
logSecurityEvent('LOGIN_FAILURE', null, { email: email, ip: req.ip, reason: 'invalid_password' });
logSecurityEvent('AUTHORIZATION_FAILURE', req.user.id, { path: req.path, method: req.method });
logSecurityEvent('RATE_LIMIT_EXCEEDED', req.user?.id, { endpoint: req.path, ip: req.ip });
```

Monitoring and Alerting

-
-
-
-
-

-
-
-
-

4.2.10 A10: Server-Side Request Forgery (SSRF)

Understanding SSRF

-
-
-
-

Preventing SSRF

```
Terminal

const ALLOWED_DOMAINS = ['github.com', 'api.github.com', 'raw.githubusercontent.com'];

function validateUrl(userUrl) {
  const parsed = new URL(userUrl);

  if (!ALLOWED_DOMAINS.includes(parsed.hostname)) {
    throw new Error('Domain not allowed');
  }

  return parsed;
}
```



```
Terminal

async function safeFetch(userUrl) {
  const parsed = new URL(userUrl);

  // Block non-HTTP protocols
  if (!['http:', 'https:'].includes(parsed.protocol)) {
    throw new Error('Only HTTP(S) allowed');
  }

  // Block known internal hostnames
  const blockedHostnames = [
    'localhost', '127.0.0.1', '0.0.0.0',
    '169.254.169.254', // AWS metadata
    'metadata.google.internal', // GCP metadata
    '10.', '172.16.', '192.168.' // Private ranges (check with startsWith)
  ];

  for (const blocked of blockedHostnames) {
    if (parsed.hostname.startsWith(blocked) || parsed.hostname === blocked) {
      throw new Error('Access to internal resources not allowed');
    }
  }

  // Resolve hostname and verify IP isn't internal
  const dns = require('dns').promises;
  const addresses = await dns.resolve4(parsed.hostname);

  for (const ip of addresses) {
    if (isPrivateIP(ip)) {
      throw new Error('Domain resolves to internal IP');
    }
  }

  // Finally safe to fetch
  return fetch(userUrl, {
    timeout: 5000,
    follow: 0 // Don't follow redirects (could redirect to internal)
  });
}
```

4.3

Input Validation and Sanitization

4.3.1 Validation Strategies

```
Terminal

// Only allow alphanumeric characters and limited punctuation
const USERNAME_PATTERN = /^[a-zA-Z0-9_-]{3,30}$/;

if (!USERNAME_PATTERN.test(username)) {
  throw new Error('Username must be 3-30 alphanumeric characters');
}
```

```
Terminal

// WEAK: Block <script> tags
if (input.includes('<script>')) {
  throw new Error('Invalid input');
}

// Bypass: <SCRIPT>, <scr<script>ipt>, <script , etc.
```

```
Terminal

// Convert to expected type, reject if conversion fails
const userId = parseInt(req.params.id, 10);
if (isNaN(userId) || userId <= 0) {
  throw new Error('Invalid user ID');
}
```

```
Terminal

// Age must be reasonable
if (age < 0 || age > 150) {
  throw new Error('Age must be between 0 and 150');
}

// Title has length limits
if (title.length < 1 || title.length > 200) {
  throw new Error('Title must be 1-200 characters');
}
```

4.3.2 Comprehensive Validation with Joi

```
Terminal

const Joi = require('joi');

// Define validation schemas once, use everywhere
const schemas = {
  userRegistration: Joi.object({
    email: Joi.string()
      .email()
      .max(254)
      .required()
      .messages({
        'string.email': 'Please enter a valid email address',
        'any.required': 'Email is required'
      })
  })
}
```

Terminal -- continued

```
password: Joi.string()
  .min(12)
  .max(128)
  .required(),

name: Joi.string()
  .min(1)
  .max(100)
  .pattern(/^[\p{L}\s'-~]+$/u) // Unicode letters, spaces, hyphens, apostrophes
  .required()
}),

taskCreate: Joi.object({
  title: Joi.string().min(1).max(200).required(),
  description: Joi.string().max(10000).allow(''),
  priority: Joi.number().integer().min(0).max(4).default(0),
  dueDate: Joi.date().iso().greater('now').allow(null)
})
};
```

Terminal

```
function validate(schemaName) {
  return (req, res, next) => {
    const schema = schemas[schemaName];
    const { error, value } = schema.validate(req.body, {
      abortEarly: false, // Return ALL errors, not just first
      stripUnknown: true // Remove fields not in schema
    });

    if (error) {
      return res.status(422).json({
        error: 'Validation failed',
        details: error.details.map(d => ({
          field: d.path.join('.'),
          message: d.message
        }))
      });
    }
  }
}
```

Terminal -- continued

```
// Replace body with validated, sanitized version
req.body = value;
next();
};
}

// Apply to routes
app.post('/api/users', validate('userRegistration'), createUser);
app.post('/api/tasks', authenticate, validate('taskCreate'), createTask);
```

4.3.3 Output Encoding

Terminal

```
const he = require('he');

// User input that might contain HTML
const userComment = '<script>alert("xss")</script>Hello!';

// Encode for safe HTML display
const safeComment = he.encode(userComment);
// Result: &lt;script&gt;alert(&quot;xss&quot;)&lt;/script&gt;Hello!

// Browser displays literally: <script>alert("xss")</script>Hello!
// Instead of executing the script
```



Terminal

```
// SAFE: React automatically encodes
<div>{userComment}</div>

// DANGEROUS: Deliberately inserting HTML
<div dangerouslySetInnerHTML={{__html: userComment}} />
```



Terminal

```
const DOMPurify = require('dompurify');
const { JSDOM } = require('jsdom');

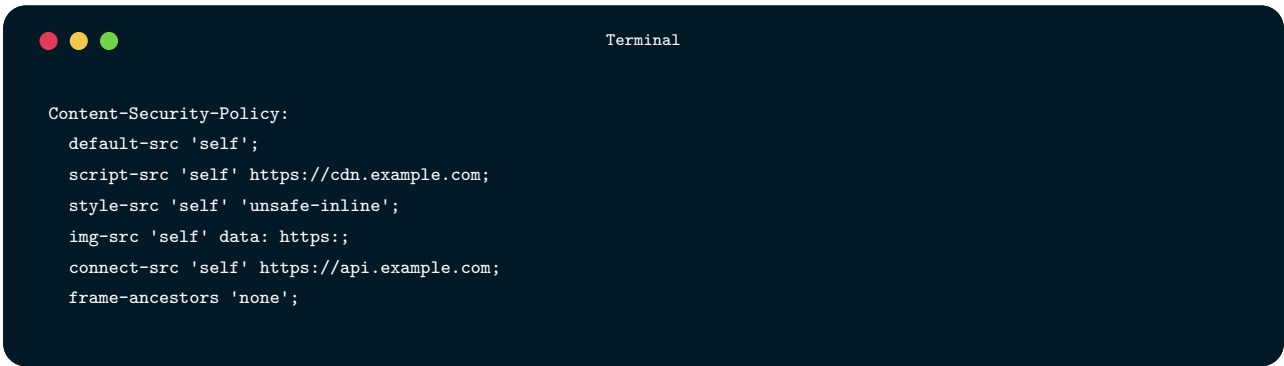
const window = new JSDOM('').window;
const purify = DOMPurify(window);

// Allow only safe tags, remove everything else
const safeHtml = purify.sanitize(userHtml, {
  ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'a', 'p', 'br'],
  ALLOWED_ATTR: ['href', 'title']
});
```

4.4

Security Headers

4.4.1 Content Security Policy

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the word "Terminal". The terminal displays a Content Security Policy configuration.

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' https://cdn.example.com;
  style-src 'self' 'unsafe-inline';
  img-src 'self' data: https:;
  connect-src 'self' https://api.example.com;
  frame-ancestors 'none';
```



Terminal

```
<!-- VIOLATES CSP: Inline event handler -->
<button onclick="handleClick()">Click</button>

<!-- CSP-COMPLIANT: Event listener in separate script -->
<button id="myButton">Click</button>
<script src="/js/handlers.js"></script>
```



Terminal

```
Content-Security-Policy-Report-Only: default-src 'self'; report-uri /csp-report
```

4.4.2 Other Essential Headers



Terminal

```
Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
```



Terminal

```
X-Content-Type-Options: nosniff
```




Terminal

```
X-Frame-Options: DENY
```



Terminal

```
Referrer-Policy: strict-origin-when-cross-origin
```

4.5

Security Testing

4.5.1 Types of Security Testing

4.5.2 Integrating Security Testing into CI/CD

A terminal window with a dark blue background and light gray text. The title bar at the top says "Terminal" and has three colored window control buttons (red, yellow, green) on the left. The text inside the terminal is a GitHub Actions workflow configuration for security testing.

```
# .github/workflows/security.yml
name: Security

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
  schedule:
    - cron: '0 0 * * *' # Daily for new vulnerability discoveries

jobs:
  sast:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Run Semgrep
```

Terminal -- continued

```
    uses: returntocorp/semgrep-action@v1
  with:
    config: p/security-audit p/secrets p/owasp-top-ten

sca:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: npm ci
    - run: npm audit --audit-level=high

security-tests:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: npm ci
    - run: npm run test:security
```

4.5.3 Writing Security Tests

Terminal

```
describe('Authentication Security', () => {
  test('rejects requests without authentication', async () => {
    const response = await request(app)
      .get('/api/tasks')
      .expect(401);
  });

  test('rejects invalid tokens', async () => {
    const response = await request(app)
      .get('/api/tasks')
      .set('Authorization', 'Bearer invalid.token.here')
      .expect(401);
  });

  test('rate limits login attempts', async () => {
    // Make many failed login attempts
    const attempts = Array(10).fill().map(() =>
```

Terminal -- continued

```

    request(app)
      .post('/api/auth/login')
      .send({ email: 'test@test.com', password: 'wrong' })
    );

    const responses = await Promise.all(attempts);
    const rateLimited = responses.filter(r => r.status === 429);

    expect(rateLimited.length).toBeGreaterThan(0);
  });
});

describe('Authorization Security', () => {
  test('users cannot access other users data', async () => {
    const user1Token = await getAuthToken('user1@test.com');
    const user2Id = 2;

    await request(app)
      .get(`/api/users/${user2Id}/profile`)
      .set('Authorization', `Bearer ${user1Token}`)
      .expect(403);
  });

  test('non-admins cannot access admin endpoints', async () => {
    const userToken = await getAuthToken('user@test.com');

    await request(app)
      .get('/api/admin/users')
      .set('Authorization', `Bearer ${userToken}`)
      .expect(403);
  });
});

describe('Input Validation', () => {
  test('SQL injection is prevented', async () => {
    const token = await getAuthToken();

    // Attempt SQL injection
    await request(app)
      .get('/api/tasks')
      .query({ search: "'; DROP TABLE tasks; --" })
      .set('Authorization', `Bearer ${token}`)
      .expect(200);

    // Verify table still exists by making another request
    await request(app)
      .get('/api/tasks')

```

Terminal -- continued

```
.set('Authorization', `Bearer ${token}`)
.expect(200);
});
});
```

4.6

Incident Response

4.6.1 Incident Response Phases

Terminal

```
+-----+
|          INCIDENT RESPONSE PHASES          |
+-----+
|
| 1. PREPARATION                             |
|   Before incidents occur:                  |
|   • Document response procedures            |
|   • Establish communication channels        |
|   • Train team members                     |
|   • Set up monitoring and alerting          |
|   • Maintain contact lists (legal, PR, executives) |
|
| 2. IDENTIFICATION                           |
```

Terminal -- continued

```
| Detecting and confirming an incident: |
|   • Monitor alerts and anomalies |
|   • Assess scope and severity |
|   • Document initial findings |
|   • Classify incident type |
| |
| 3. CONTAINMENT |
|   Limiting damage: |
|   • Short-term: Stop immediate damage |
|   • Long-term: Implement temporary fixes |
|   • Preserve evidence for analysis |
| |
| 4. ERADICATION |
|   Removing the threat: |
|   • Remove attacker access |
|   • Patch vulnerabilities |
|   • Reset compromised credentials |
|   • Verify complete removal |
| |
| 5. RECOVERY |
|   Returning to normal: |
|   • Restore systems to normal operation |
|   • Monitor for signs of persistent compromise |
|   • Gradually return to full service |
| |
| 6. LESSONS LEARNED |
|   Improving for the future: |
|   • Conduct post-incident review |
|   • Document timeline and actions |
|   • Identify improvement opportunities |
|   • Update procedures and controls |
| |
+-----+
```

4.6.2 Containment Actions

4.6.3 Communication

4.7

Chapter Summary

4.8

Key Terms

Term	Definition
SQL Injection	Attack that inserts malicious SQL code through user input

Term	Definition
CSRF	Cross-Site Request Forgery—tricking users into performing unintended actions
IDOR	Insecure Direct Object Reference—accessing objects by manipulating identifiers
JWT	JSON Web Token—compact, self-contained token for authentication
HSTS	HTTP Strict Transport Security—forces HTTPS connections
DAST	Dynamic Application Security Testing—testing running applications
Defense in Depth	Layering multiple security controls so failure of one doesn't compromise security

4.9

Review Questions

- 1.
- 2.

- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.

4.10

Hands-On Exercises

4.10.1 Exercise 12.1: Security Audit

- 1.
- 2.
- 3.
- 4.
- 5.

4.10.2 Exercise 12.2: Implement OWASP Protections

- 1.
- 2.
- 3.
- 4.
- 5.

4.10.3 Exercise 12.3: Security Testing Suite

- 1.
- 2.
- 3.
- 4.
- 5.

4.10.4 Exercise 12.4: Dependency Security Pipeline

- 1.
- 2.
- 3.
- 4.
- 5.

4.10.5 Exercise 12.5: Security Logging Implementation

- 1.
- 2.
- 3.
- 4.
- 5.

4.10.6 Exercise 12.6: Incident Response Plan

- 1.
- 2.
- 3.
- 4.
- 5.

4.11

Further Reading

-
-
-
-
-
-
-

4.12

References

Part III.

Evolution and Professional Practice

CHAPTER 05

EVOLUTION

Software Maintenance and Evolution

Maintaining, refactoring, documenting, and evolving long-lived software systems.

LEARNING OBJECTIVES

- ✓ By the end of this chapter, you will be able to:
 - ✓ Explain why software maintenance constitutes the majority of software lifecycle costs
 - ✓ Identify and categorize different types of technical debt
 - ✓ Apply systematic refactoring techniques to improve code quality
 - ✓ Develop strategies for working with and modernizing legacy systems
 - ✓ Create and maintain effective documentation at multiple levels
 - ✓ Implement semantic versioning and change management practices
 - ✓ Design systems with maintainability as a primary concern
 - ✓ Balance the competing demands of new features, maintenance, and technical debt reduction
 - ✓ Establish metrics and processes for tracking software health over time

5.1

The Reality of Software Maintenance

5.1.1 Types of Software Maintenance

A terminal window with a dark blue background and white text. The title bar says "Terminal". Inside the terminal, there is a table with two columns. The first column lists types of software maintenance, and the second column lists the percentage of maintenance effort for each type. The table is enclosed in a box with dashed lines.

```
+-----+
|          TYPES OF SOFTWARE MAINTENANCE          |
+-----+
| CORRECTIVE MAINTENANCE (20% of maintenance effort) |
| Fixing defects discovered after deployment.         |
| • Bug fixes for incorrect behavior                  |
| • Security vulnerability patches                    |
| • Data corruption repairs                           |
| • Performance issue resolution                      |
| ADAPTIVE MAINTENANCE (25% of maintenance effort)   |
| Modifying software to work in changed environments. |
+-----+
```


Terminal -- continued

```
| • Operating system upgrades |
| • Database version migrations |
| • Third-party API changes |
| • Hardware platform changes |
| • Regulatory compliance updates |
|
| PERFECTIVE MAINTENANCE (50% of maintenance effort) |
| Enhancing software to meet new or changed requirements. |
| • New feature development |
| • Performance improvements |
| • Usability enhancements |
| • Capacity scaling |
|
| PREVENTIVE MAINTENANCE (5% of maintenance effort) |
| Improving maintainability to prevent future problems. |
| • Code refactoring |
| • Documentation updates |
| • Technical debt reduction |
| • Test coverage improvement |
```

```
+-----+
```

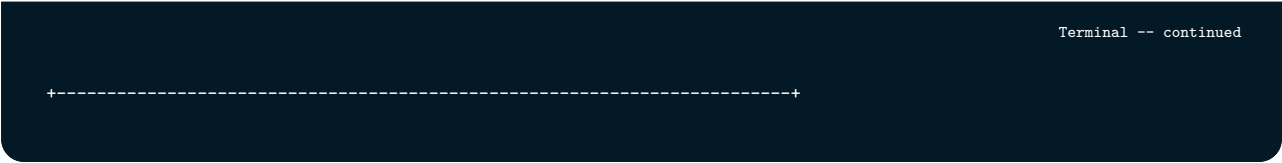
5.1.2 The Maintenance Mindset

5.1.3 Measuring Maintainability

5.2

Technical Debt



A dark-themed terminal window with a dashed line and a plus sign at the end.

```
Terminal -- continued
```

5.2.2 Manifestations of Technical Debt

5.2.3 The Interest Payments

```
Terminal

+-----+
|          TECHNICAL DEBT COMPOUNDING          |
+-----+
|
| Year 1: Initial debt accumulated during rapid development |
| ----- |
|           [] Debt: Low |
| Feature Velocity: 100% |
|
| Year 2: Debt starts affecting productivity |
| ----- |
|           [] Debt: Moderate |
| Feature Velocity: 85% |
|
| Year 3: Significant slowdown, more bugs |
| ----- |
|           [] Debt: High |
| Feature Velocity: 60% |
|
| Year 4: Team spends most time on maintenance |
| ----- |
|           [] Debt: Critical |
| Feature Velocity: 35% |
|
| Year 5: System becomes unmaintainable, rewrite discussions begin |
| ----- |
|           [] Debt: Overwhelming |
| Feature Velocity: 10% |
|
+-----+
```

5.2.4 Managing Technical Debt

```
Terminal

// TODO(tech-debt): This function has O(n^2) complexity due to nested loops.
// Acceptable for current scale (<1000 items) but will need optimization
// if we exceed 10,000 items. Estimated fix: 4 hours.
// Ticket: TECH-234
// Added: 2024-01-15, Author: jsmith
function findDuplicates(items) {
  const duplicates = [];
  for (let i = 0; i < items.length; i++) {
    for (let j = i + 1; j < items.length; j++) {
      if (items[i].id === items[j].id) {
        duplicates.push(items[i]);
      }
    }
  }
  return duplicates;
}
```

-
-
-
-

5.3

Refactoring

5.3.1 Why Refactoring Matters

5.3.2 When to Refactor

5.3.3 Refactoring Techniques

Extract Function

```
Terminal

function processOrder(order) {
  // Validate order items
  if (!order.items || order.items.length === 0) {
    throw new Error('Order must have at least one item');
  }

  for (const item of order.items) {
    if (!item.productId || !item.quantity || item.quantity <= 0) {
      throw new Error('Invalid order item');
    }

    const product = productCatalog.find(p => p.id === item.productId);
    if (!product) {
      throw new Error(`Product ${item.productId} not found`);
    }

    if (product.inventory < item.quantity) {
      throw new Error(`Insufficient inventory for ${product.name}`);
    }
  }

  // Calculate totals
  let subtotal = 0;
  for (const item of order.items) {
    const product = productCatalog.find(p => p.id === item.productId);
    subtotal += product.price * item.quantity;
  }

  const tax = subtotal * 0.08;
  const shipping = subtotal > 100 ? 0 : 10;
  const total = subtotal + tax + shipping;

  // Create order record
  const orderRecord = {
    id: generateOrderId(),
    customerId: order.customerId,
    items: order.items.map(item => ({
      productId: item.productId,
      quantity: item.quantity,
      price: productCatalog.find(p => p.id === item.productId).price
    })),
    subtotal,
    tax,
    shipping,
    total,
    status: 'pending',
    createdAt: new Date()
  }
```

Terminal -- continued

```
};

// Update inventory
for (const item of order.items) {
  const product = productCatalog.find(p => p.id === item.productId);
  product.inventory -= item.quantity;
}

// Save and return
orders.push(orderRecord);
return orderRecord;
}
```

Terminal

```
function processOrder(order) {
  validateOrder(order);

  const pricing = calculateOrderPricing(order.items);
  const orderRecord = createOrderRecord(order, pricing);

  reserveInventory(order.items);
  saveOrder(orderRecord);

  return orderRecord;
}

function validateOrder(order) {
  if (!order.items || order.items.length === 0) {
    throw new Error('Order must have at least one item');
  }

  for (const item of order.items) {
    validateOrderItem(item);
  }
}

function validateOrderItem(item) {
  if (!item.productId || !item.quantity || item.quantity <= 0) {
```

Terminal -- continued

```
    throw new Error('Invalid order item');
  }

  const product = findProduct(item.productId);

  if (product.inventory < item.quantity) {
    throw new Error(`Insufficient inventory for ${product.name}`);
  }
}

function findProduct(productId) {
  const product = productCatalog.find(p => p.id === productId);
  if (!product) {
    throw new Error(`Product ${productId} not found`);
  }
  return product;
}

function calculateOrderPricing(items) {
  const subtotal = items.reduce((sum, item) => {
    const product = findProduct(item.productId);
    return sum + (product.price * item.quantity);
  }, 0);

  const tax = calculateTax(subtotal);
  const shipping = calculateShipping(subtotal);
  const total = subtotal + tax + shipping;

  return { subtotal, tax, shipping, total };
}

function calculateTax(subtotal) {
  return subtotal * 0.08;
}

function calculateShipping(subtotal) {
  return subtotal > 100 ? 0 : 10;
}

function createOrderRecord(order, pricing) {
  return {
    id: generateOrderId(),
    customerId: order.customerId,
    items: order.items.map(item => ({
      productId: item.productId,
      quantity: item.quantity,
      price: findProduct(item.productId).price
```

Terminal -- continued

```
    })),
    ...pricing,
    status: 'pending',
    createdAt: new Date()
  };
}

function reserveInventory(items) {
  for (const item of items) {
    const product = findProduct(item.productId);
    product.inventory -= item.quantity;
  }
}

function saveOrder(orderRecord) {
  orders.push(orderRecord);
}
```

Replace Conditional with Polymorphism

Terminal

```
function calculateShippingCost(shipment) {
  switch (shipment.type) {
    case 'standard':
      return shipment.weight * 0.5 + 5;

    case 'express':
      return shipment.weight * 1.0 + 15;
  }
}
```

Terminal -- continued

```
    case 'overnight':
      return shipment.weight * 2.0 + 25;

    case 'international':
      const baseRate = shipment.weight * 3.0;
      const customsFee = 20;
      const distanceMultiplier = getDistanceMultiplier(shipment.destination);
      return (baseRate + customsFee) * distanceMultiplier;

    default:
      throw new Error(`Unknown shipment type: ${shipment.type}`);
  }
}

function getEstimatedDelivery(shipment) {
  switch (shipment.type) {
    case 'standard':
      return addBusinessDays(new Date(), 5);

    case 'express':
      return addBusinessDays(new Date(), 2);

    case 'overnight':
      return addBusinessDays(new Date(), 1);

    case 'international':
      return addBusinessDays(new Date(), 14);

    default:
      throw new Error(`Unknown shipment type: ${shipment.type}`);
  }
}

// Every function dealing with shipments needs these switch statements
// Adding a new shipment type requires modifying every function
```

```
Terminal

// Base class defines the interface
class ShippingMethod {
  constructor(shipment) {
    this.shipment = shipment;
  }

  calculateCost() {
    throw new Error('Subclass must implement calculateCost');
  }

  getEstimatedDelivery() {
    throw new Error('Subclass must implement getEstimatedDelivery');
  }
}

class StandardShipping extends ShippingMethod {
  calculateCost() {
    return this.shipment.weight * 0.5 + 5;
  }

  getEstimatedDelivery() {
    return addBusinessDays(new Date(), 5);
  }
}

class ExpressShipping extends ShippingMethod {
  calculateCost() {
    return this.shipment.weight * 1.0 + 15;
  }

  getEstimatedDelivery() {
    return addBusinessDays(new Date(), 2);
  }
}

class OvernightShipping extends ShippingMethod {
  calculateCost() {
    return this.shipment.weight * 2.0 + 25;
  }

  getEstimatedDelivery() {
    return addBusinessDays(new Date(), 1);
  }
}

class InternationalShipping extends ShippingMethod {
  calculateCost() {
```

Terminal -- continued

```
const baseRate = this.shipment.weight * 3.0;
const customsFee = 20;
const distanceMultiplier = getDistanceMultiplier(this.shipment.destination);
return (baseRate + customsFee) * distanceMultiplier;
}

getEstimatedDelivery() {
  return addBusinessDays(new Date(), 14);
}
}

// Factory creates the appropriate shipping method
function createShippingMethod(shipment) {
  const methods = {
    'standard': StandardShipping,
    'express': ExpressShipping,
    'overnight': OvernightShipping,
    'international': InternationalShipping
  };

  const MethodClass = methods[shipment.type];
  if (!MethodClass) {
    throw new Error(`Unknown shipment type: ${shipment.type}`);
  }

  return new MethodClass(shipment);
}

// Usage is clean and type-agnostic
function processShipment(shipment) {
  const method = createShippingMethod(shipment);

  return {
    cost: method.calculateCost(),
    estimatedDelivery: method.getEstimatedDelivery()
  };
}
```


Introduce Parameter Object

```
Terminal

function createEvent(title, description, startDate, startTime, endDate, endTime,
                    location, isVirtual, meetingUrl, attendees, reminderMinutes) {
    // Validate dates
    const start = combineDateAndTime(startDate, startTime);
    const end = combineDateAndTime(endDate, endTime);

    if (end <= start) {
        throw new Error('End must be after start');
    }

    // Validate location
    if (isVirtual && !meetingUrl) {
        throw new Error('Virtual events require a meeting URL');
    }

    // ... rest of creation logic
}

function updateEvent(eventId, title, description, startDate, startTime, endDate,
                    endTime, location, isVirtual, meetingUrl, attendees, reminderMinutes) {
    // Same parameters repeated
}

function isEventConflicting(existingEvent, startDate, startTime, endDate, endTime) {
    // Subset of parameters for time checking
}
```

```
Terminal

// Time range as a distinct concept
class TimeRange {
  constructor(start, end) {
    if (!(start instanceof Date) || !(end instanceof Date)) {
      throw new Error('Start and end must be Date objects');
    }
    if (end <= start) {
      throw new Error('End must be after start');
    }

    this.start = start;
    this.end = end;
  }

  get durationMinutes() {
    return (this.end - this.start) / (1000 * 60);
  }

  overlaps(other) {
    return this.start < other.end && other.start < this.end;
  }

  contains(date) {
    return date >= this.start && date <= this.end;
  }
}

// Location as a distinct concept
class EventLocation {
  constructor({ venue, address, isVirtual, meetingUrl }) {
    this.venue = venue;
    this.address = address;
    this.isVirtual = isVirtual;
    this.meetingUrl = meetingUrl;

    this.validate();
  }

  validate() {
    if (this.isVirtual && !this.meetingUrl) {
      throw new Error('Virtual events require a meeting URL');
    }
  }

  get displayString() {
    if (this.isVirtual) {
      return `Virtual: ${this.meetingUrl}`;
    }
  }
}
```

Terminal -- continued

```
    }
    return this.address ? `${this.venue}, ${this.address}` : this.venue;
  }
}

// Event creation with parameter objects
function createEvent({ title, description, timeRange, location, attendees, reminderMinutes }) {
  // Validation is already done in TimeRange and EventLocation constructors

  return {
    id: generateEventId(),
    title,
    description,
    timeRange,
    location,
    attendees: attendees || [],
    reminderMinutes: reminderMinutes || 30,
    createdAt: new Date()
  };
}

// Usage is clearer
const event = createEvent({
  title: 'Team Standup',
  description: 'Daily sync meeting',
  timeRange: new TimeRange(
    new Date('2024-03-15T09:00:00'),
    new Date('2024-03-15T09:15:00')
  ),
  location: new EventLocation({
    isVirtual: true,
    meetingUrl: 'https://zoom.us/j/123456'
  }),
  attendees: ['alice@example.com', 'bob@example.com']
});

// Conflict checking is now clean
function isEventConflicting(existingEvent, proposedTimeRange) {
  return existingEvent.timeRange.overlaps(proposedTimeRange);
}
```

5.3.4 Refactoring Safely

5.4

Working with Legacy Systems

5.4.1 Understanding Legacy Challenges

5.4.2 Strategies for Legacy Evolution



```

Terminal -- continued

|           | New | | Legacy |           |
|           | System | | System |           |
|           | (5%) | | (95%) |           |
|           +-----+ +-----+           |
|
| Phase 2: New system takes over more functionality
|
|           +-----+ +-----+           |
|           | New | | Legacy |           |
|           | System | | System |           |
|           | (60%) | | (40%) |           |
|           +-----+ +-----+           |
|
| Phase 3: Legacy system retired
|
|           +-----+           |
|           | New System |           |
|           | (100%) |           |
|           +-----+           |
|
+-----+

```

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

```

Terminal

// Step 1: Identify the component to replace
// Original code directly uses the legacy payment processor
class OrderService {

```

Terminal -- continued

```

async checkout(order) {
  // Direct dependency on legacy system
  const result = await LegacyPaymentSystem.processPayment({
    amount: order.total,
    cardNumber: order.paymentDetails.cardNumber,
    expiry: order.paymentDetails.expiry
  });
  return result;
}
}

// Step 2: Create an abstraction
interface PaymentProcessor {
  processPayment(amount: number, paymentDetails: PaymentDetails): Promise<PaymentResult>;
}

// Step 3: Wrap legacy system in the abstraction
class LegacyPaymentProcessor implements PaymentProcessor {
  async processPayment(amount, paymentDetails) {
    return LegacyPaymentSystem.processPayment({
      amount,
      cardNumber: paymentDetails.cardNumber,
      expiry: paymentDetails.expiry
    });
  }
}

// Step 4: Update consumers to use abstraction
class OrderService {
  constructor(paymentProcessor: PaymentProcessor) {
    this.paymentProcessor = paymentProcessor;
  }

  async checkout(order) {
    return this.paymentProcessor.processPayment(
      order.total,
      order.paymentDetails
    );
  }
}

// Step 5: Create new implementation
class StripePaymentProcessor implements PaymentProcessor {
  async processPayment(amount, paymentDetails) {
    // New, modern implementation
    return stripe.charges.create({
      amount: amount * 100, // Stripe uses cents

```

Terminal -- continued

```

        currency: 'usd',
        source: paymentDetails.stripeToken
    });
}
}

// Step 6: Gradually transition
class PaymentProcessorFactory {
    static create(merchant) {
        // Feature flag controls which implementation to use
        if (featureFlags.isEnabled('new-payment-system', merchant.id)) {
            return new StripePaymentProcessor();
        }
        return new LegacyPaymentProcessor();
    }
}

```

Terminal

```

// Characterization test process:
// 1. Call the system with various inputs
// 2. Record actual outputs
// 3. Write tests that verify these outputs

describe('LegacyPricingEngine (characterization)', () => {
    // We don't know if these prices are "correct" - we're documenting behavior

    test('basic item pricing', () => {
        const price = LegacyPricingEngine.calculate({
            itemCode: 'ABC123',
            quantity: 1,
            customerType: 'retail'
        });

        // This is what the system currently returns
        // If we change it, we need to consciously decide if the new behavior is better
        expect(price).toBe(29.99);
    });
}

```


Terminal -- continued

```
test('bulk discount calculation', () => {
  const price = LegacyPricingEngine.calculate({
    itemCode: 'ABC123',
    quantity: 100,
    customerType: 'retail'
  });

  // Documents current bulk discount behavior
  expect(price).toBe(2499.15); // Not exactly 100 * 29.99
});

test('wholesale pricing', () => {
  const price = LegacyPricingEngine.calculate({
    itemCode: 'ABC123',
    quantity: 1,
    customerType: 'wholesale'
  });

  // Documents the wholesale discount
  expect(price).toBe(22.49); // 25% discount from retail
});

// Edge cases we discovered through exploration
test('handles negative quantity by returning zero', () => {
  const price = LegacyPricingEngine.calculate({
    itemCode: 'ABC123',
    quantity: -5,
    customerType: 'retail'
  });

  expect(price).toBe(0); // Surprising but this is current behavior
});
});
```

5.4.3 Managing Risk During Legacy Evolution



Terminal

```
async function processWithComparison(input) {
  // Run both systems
  const [legacyResult, newResult] = await Promise.all([
    legacySystem.process(input),
    newSystem.process(input)
  ]);

  // Compare results
  const match = deepEqual(legacyResult, newResult);

  if (!match) {
    // Log for investigation but don't fail
    logger.warn('Result mismatch', {
      input,
      legacyResult,
      newResult
    });

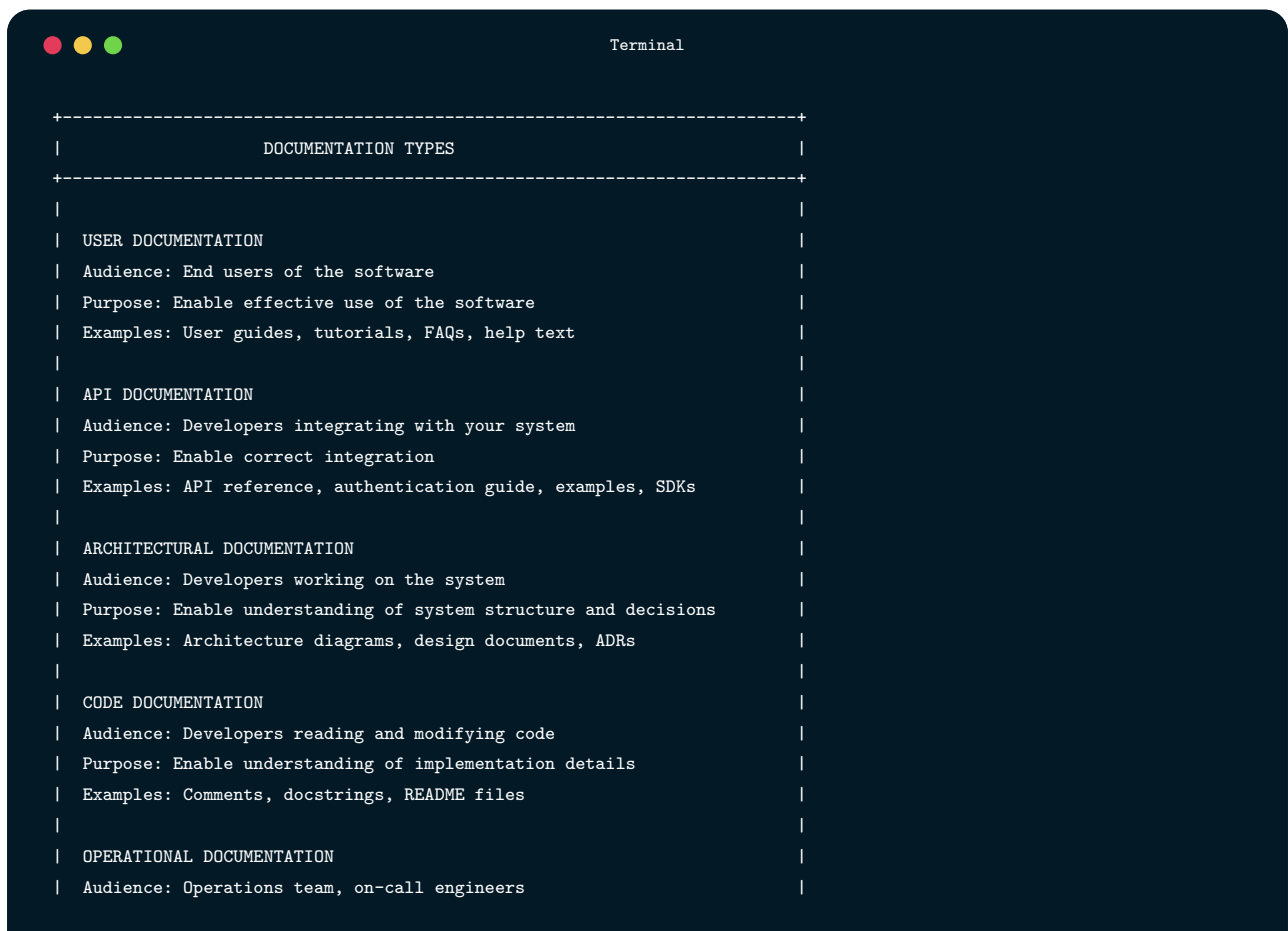
    metrics.increment('result_mismatch');
  }

  // Return legacy result for safety
  return legacyResult;
}
```

5.5

Documentation

5.5.1 Types of Documentation

A terminal window with a dark blue background and white text. The title bar shows three colored circles (red, yellow, green) and the word "Terminal". The content is a table of documentation types, enclosed in a dashed border. The table lists five types: User Documentation, API Documentation, Architectural Documentation, Code Documentation, and Operational Documentation, each with its audience, purpose, and examples.

DOCUMENTATION TYPES	
USER DOCUMENTATION	
Audience: End users of the software	
Purpose: Enable effective use of the software	
Examples: User guides, tutorials, FAQs, help text	
API DOCUMENTATION	
Audience: Developers integrating with your system	
Purpose: Enable correct integration	
Examples: API reference, authentication guide, examples, SDKs	
ARCHITECTURAL DOCUMENTATION	
Audience: Developers working on the system	
Purpose: Enable understanding of system structure and decisions	
Examples: Architecture diagrams, design documents, ADRs	
CODE DOCUMENTATION	
Audience: Developers reading and modifying code	
Purpose: Enable understanding of implementation details	
Examples: Comments, docstrings, README files	
OPERATIONAL DOCUMENTATION	
Audience: Operations team, on-call engineers	

Terminal -- continued

```
| Purpose: Enable running and troubleshooting the system |
| Examples: Runbooks, deployment guides, monitoring guides |
| |
+-----+
```

5.5.2 Architectural Decision Records

Terminal

```
# ADR 0012: Use PostgreSQL as Primary Database

## Status
Accepted (2024-01-15)

## Context
We need to select a primary database for the TaskFlow application. The
application requires:
- Strong consistency for financial transactions
- Complex querying capabilities for reporting
- JSON storage for flexible user preferences
- Full-text search for task descriptions
- Expected scale: 100,000 users, 10 million tasks

We considered: PostgreSQL, MySQL, MongoDB, and CockroachDB.

## Decision
We will use PostgreSQL as our primary database.

## Rationale

**Why PostgreSQL over MySQL:**
- Superior JSON support (JSONB with indexing)
- Better full-text search capabilities
```

Terminal -- continued

```
- More advanced indexing options (GIN, GiST)
- Stronger standards compliance

**Why PostgreSQL over MongoDB:**
- Strong consistency required for task state transitions
- Complex reporting queries span multiple collections
- Team has more SQL experience than MongoDB experience
- PostgreSQL's JSONB provides document flexibility where needed

**Why PostgreSQL over CockroachDB:**
- CockroachDB's distributed architecture is premature for our scale
- PostgreSQL has larger ecosystem and more operational tooling
- We can migrate to CockroachDB later if horizontal scaling is needed

## Consequences

**Positive:**
- Single database handles relational data, JSON, and full-text search
- Strong ecosystem of tools, libraries, and hosting options
- Team familiarity reduces learning curve

**Negative:**
- Single-node PostgreSQL limits horizontal scaling
- Must implement application-level sharding if we exceed single-node capacity
- Some MongoDB-native patterns won't apply

**Risks:**
- If we grow beyond 10M tasks significantly, we may need to migrate to
  distributed database or implement sharding

## Alternatives Considered

See comparison matrix in Appendix A.

## Related Decisions
- ADR 0015: Use read replicas for reporting queries
- ADR 0018: Implement Redis caching layer
```

5.5.3 Code Comments

```
Terminal

// Bad: Restates the obvious
let count = 0; // Initialize count to zero

// Bad: Explains what, not why
// Loop through users
for (const user of users) {
  // Check if user is active
  if (user.isActive) {
    // Increment count
    count++;
  }
}

// Bad: Outdated comment contradicting code
// Send email notification
await sendSmsNotification(user); // Comment says email, code sends SMS
```

```
Terminal

// Good: Explains why, not what
// We process in batches of 100 to avoid overwhelming the email service
// rate limits (max 200 requests/minute). See incident INC-234.
const BATCH_SIZE = 100;

// Good: Documents non-obvious behavior
// Returns null instead of throwing for missing users because
// the caller often doesn't care if the user exists (e.g.,
// permission checks for anonymous users)
function findUser(id) {
  return users.get(id) || null;
}

// Good: Warns about surprising behavior
// WARNING: This function modifies the input array in place for performance.
// Clone before calling if you need to preserve the original.
```

Terminal -- continued

```
function sortByPriority(tasks) {
  return tasks.sort((a, b) => b.priority - a.priority);
}

// Good: Provides context that code can't express
// This calculation matches the formula in Section 4.2 of the
// ISO 4217 currency specification. Don't "simplify" without
// verifying against the spec.
function roundCurrency(amount, currency) {
  const decimals = currencyDecimals[currency] ?? 2;
  const factor = Math.pow(10, decimals);
  return Math.round(amount * factor) / factor;
}

// Good: Explains workaround for external issue
// HACK: The Stripe API sometimes returns duplicate webhook events.
// We deduplicate by tracking processed event IDs for 24 hours.
// Remove this when Stripe fixes the issue (reported: 2024-01-10)
const processedEventIds = new Set();
```

5.5.4 README Files

Terminal

```
# TaskFlow API

A RESTful API for task management with real-time collaboration features.

## Quick Start

```bash
Clone repository
git clone https://github.com/example/taskflow-api
cd taskflow-api
```

Terminal -- continued

```
Install dependencies
npm install

Set up environment
cp .env.example .env
Edit .env with your configuration

Start development server
npm run dev

Run tests
npm test
```

## 5.6

# Requirements

- 
- - 
  -

## 5.7

# Project Structure





Terminal

```
src/
+-- api/ # Route handlers and middleware
+-- services/ # Business logic
+-- repositories/ # Database access
+-- models/ # Data models and validation
+-- utils/ # Shared utilities

tests/
+-- unit/ # Unit tests
+-- integration/ # Integration tests
+-- e2e/ # End-to-end tests
```

5.8

# Configuration



DATABASE_URL	PostgreSQL connection string	Required
JWT_SECRET	Secret for JWT signing	Required

## Configuration Guide

5.9

## API Documentation

---

[API Reference](#)

5.10

## Development

---

### 5.10.1 Running Tests

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the word "Terminal". The terminal displays three lines of code, each preceded by a comment.

```
All tests
npm test

Unit tests only
npm run test:unit

With coverage
npm run test:coverage
```

### 5.10.2 Code Style

### 5.10.3 Commit Messages

#### Conventional Commits

- 
- 
- 

### 5.11

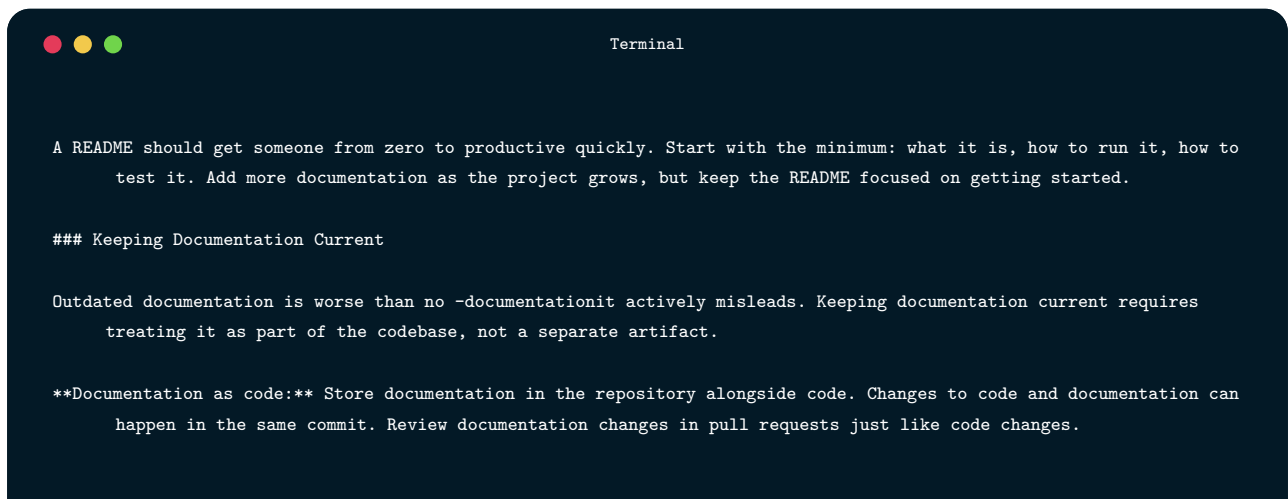
## Contributing

#### CONTRIBUTING.md

### 5.12

## License

#### LICENSE



Terminal -- continued

**\*\*Automate what you can:\*\*** Generate API documentation from code annotations. Generate architecture diagrams from code structure. Automated documentation can't become outdated because it's derived from the source of truth.

**\*\*Test documentation:\*\*** Verify that code examples in documentation actually work. Ensure links aren't broken. Check that described behaviors match actual behaviors.

**\*\*Include documentation in Definition of Done:\*\*** Features aren't complete until documented. This prevents documentation debt from accumulating.

**\*\*Regular review:\*\*** Periodically audit documentation for accuracy. This might be quarterly or when preparing releases. Assign documentation ownership to ensure someone is responsible.

---

## ## Version Management

Software changes constantly, and managing those changes requires careful versioning. Good versioning communicates change impact, enables safe upgrades, and provides rollback capability.

### ### Semantic Versioning

**\*\*Semantic Versioning (SemVer)\*\*** encodes compatibility information in version numbers. A version number MAJOR.MINOR.PATCH communicates the nature of changes:



Terminal

The power of SemVer is in the promises it makes. Users can trust that upgrading from 2.4.0 to 2.5.0 won't break their code. They can automate minor and patch updates with confidence. They know to approach major updates with care and testing.

**\*\*Before 1.0.0\*\*:** The project is in initial development. Anything may change at any time. The public API should not be considered stable. Many projects stay below 1.0.0 forever, which unfortunately makes SemVer less useful.

**\*\*Deprecation before removal\*\*:** SemVer etiquette requires warning users before removing features. Mark features as deprecated in a minor release, give users time to migrate, then remove in the next major release.

```
```javascript
/**
 * @deprecated Use `fetchUsers({ includeInactive: true })` instead.
 * Will be removed in version 3.0.0.
 */
function getAllUsersIncludingInactive() {
  console.warn(
    'getAllUsersIncludingInactive is deprecated. ' +
    'Use fetchUsers({ includeInactive: true }) instead.'
  );
  return fetchUsers({ includeInactive: true });
}
```

5.12.1 Change Management



Terminal

```
# Changelog

## [2.5.0] - 2024-03-15

### Added
- New `batchCreate` method for creating multiple tasks efficiently
- Support for task templates
- Webhook notifications for task status changes

### Changed
- Improved performance of task queries with new index strategy
```

Terminal -- continued

```

- Updated dependencies to latest versions

### Deprecated
- `createMultiple` method - use `batchCreate` instead

### Fixed
- Race condition when updating task status concurrently
- Memory leak in long-running websocket connections

## [2.4.1] - 2024-03-01

### Security
- Fixed XSS vulnerability in task description rendering (CVE-2024-1234)

### Fixed
- Incorrect due date calculation for recurring tasks

## [2.4.0] - 2024-02-15
...

```

Terminal

```

# Migrating from v2 to v3

## Breaking Changes

### Authentication API Changes

The authentication methods have been restructured for consistency.

**Before (v2):**
```javascript
const token = await auth.login(email, password);
const user = await auth.verifyToken(token);

```

Terminal

```

const { token, user } = await auth.authenticate({ email, password });
// Token verification is now automatic in middleware

```

### 5.12.2 Configuration Changes



Terminal

```
const config = require('./config');
```



Terminal

```
const config = require('./config')[process.env.NODE_ENV];
```

migration script

### 5.13

## Deprecated Features Removed

- 
- 



Terminal

```
Release branches allow maintaining multiple versions simultaneously. While you develop version 3, you might need
to release security patches for version 2:
```

Terminal

## ### Database Migrations

Code versioning is relatively simple—you deploy new code, it runs. Data versioning is harder. Databases persist across deployments, and changing schemas requires careful migration.

**\*\*Migration files\*\*** describe schema changes as code:

```
```javascript
// migrations/20240315_001_add_task_priority.js

exports.up = async function(knex) {
  // Add new column with default value
  await knex.schema.alterTable('tasks', table => {
    table.integer('priority').defaultTo(0).nullable();
    table.index('priority'); // Index for sorting by priority
  });

  // Backfill existing tasks based on business rules
  await knex.raw(`
    UPDATE tasks
    SET priority = CASE
      WHEN due_date < NOW() THEN 3      -- Overdue = high priority
      WHEN due_date < NOW() + INTERVAL '1 day' THEN 2 -- Due soon
      ELSE 0                            -- Default
    END
  `);
};

exports.down = async function(knex) {
  await knex.schema.alterTable('tasks', table => {
    table.dropColumn('priority');
  });
};
```
```

- 
- 
- 
-



- 

### 5.13.1 API Versioning



Terminal

```
GET /api/v1/tasks
GET /api/v2/tasks
```



Terminal

```
GET /api/tasks
Accept: application/vnd.taskflow.v2+json
```



Terminal

```
GET /api/tasks?version=2
```

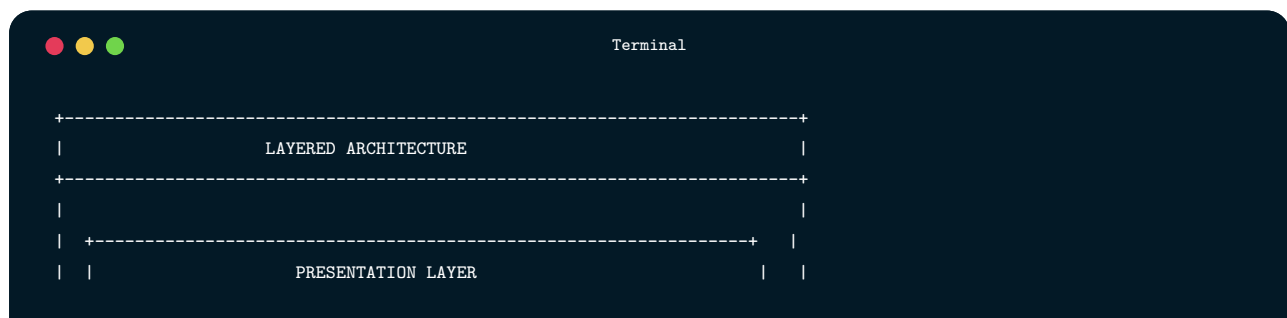
## 5.14

# Designing for Maintainability

---

## 5.14.1 Principles for Maintainable Design

## 5.14.2 Structural Patterns for Maintainability



Terminal -- continued

```

| | Handles HTTP requests, formats responses, manages sessions | |
| | Express routes, controllers, middleware, view rendering | |
| +-----+
| |
| |
| +-----+
	APPLICATION LAYER	
	Orchestrates use cases, coordinates between services	
	Application services, use case handlers, DTOs	
+-----+		
+-----+		
	DOMAIN LAYER	
	Core business logic, rules, and entities	
	Domain models, business rules, domain services	
+-----+		
+-----+		
	INFRASTRUCTURE LAYER	
	External concerns: database, APIs, file system, messaging	
	Repositories, API clients, queue handlers	
+-----+		
Dependencies flow downward. Each layer only knows about the layer		
immediately below it.		
+-----+		

```



Terminal

```

Traditional structure (by technical role)
Finding all code for "tasks" requires looking everywhere
src/
+-- controllers/
| +-- taskController.js
| +-- userController.js
| +-- projectController.js

```

Terminal -- continued

```

+-- services/
| +-- taskService.js
| +-- userService.js
| +-- projectService.js
+-- repositories/
| +-- taskRepository.js
| +-- ...
+-- models/
 +-- ...

Modular structure (by feature)
All task-related code is together
src/
+-- tasks/
| +-- taskController.js
| +-- taskService.js
| +-- taskRepository.js
| +-- taskModel.js
| +-- taskRoutes.js
+-- users/
| +-- userController.js
| +-- ...
+-- projects/
| +-- ...
+-- shared/
 +-- database.js
 +-- auth.js
 +-- errors.js

```

### 5.14.3 Testing for Maintainability

Terminal

```

+-----+
| TESTING PYRAMID |
+-----+

```

Terminal -- continued

```

| |
| |
| /\ |
| / \ |
| / E2E\ Few, slow, test real user flows |
| /-----\ |
| / \ |
| /Integration\ Some, test component integration |
| /-----\ |
| / \ |
| / Unit Tests \ Many, fast, test individual |
| /-----\ \ functions in isolation |
|/ \|
|/-----\
|
| Good test coverage enables confident refactoring.
| Bad tests (testing implementation) make refactoring painful.
|
+-----+

```

Terminal

```

// Bad: Tests implementation details
test('task service calls repository', () => {
 const mockRepo = { create: jest.fn() };
 const service = new TaskService(mockRepo);

 service.createTask({ title: 'Test' });

 // This breaks if we change how TaskService is implemented
 expect(mockRepo.create).toHaveBeenCalled();
});

// Good: Tests observable behavior
test('createTask returns created task with id', async () => {
 const service = new TaskService(realRepository);

 const task = await service.createTask({ title: 'Test' });

 // Tests what matters to callers, not internal implementation
 expect(task.id).toBeDefined();
 expect(task.title).toBe('Test');
 expect(task.status).toBe('pending');
});

```

Terminal -- continued

```
});
```

5.15

## Chapter Summary

---

5.16

## Key Terms

---

|                       |                                                                             |
|-----------------------|-----------------------------------------------------------------------------|
| Technical Debt        | Accumulated cost of shortcuts and deferred work in software                 |
| Legacy System         | Existing system that remains valuable but is difficult to work with         |
| ADR                   | Architectural Decision Record—document capturing reasoning behind decisions |
| Characterization Test | Test that documents actual behavior of existing code                        |
| Cyclomatic Complexity | Metric measuring number of independent paths through code                   |
| Coupling              | Degree of interdependence between modules                                   |

Changelog

Document recording what changed in each version

5.17

## Review Questions

---

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.



## 5.18

# Hands-On Exercises

---

## 5.18.1 Exercise 13.1: Technical Debt Audit

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

## 5.18.2 Exercise 13.2: Refactoring Practice

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

## 5.18.3 Exercise 13.3: Documentation Improvement

- 1.
- 2.
- 3.
- 4.
- 5.

#### 5.18.4 Exercise 13.4: Legacy Code Characterization

- 1.
- 2.
- 3.
- 4.
- 5.

#### 5.18.5 Exercise 13.5: Version Management

- 1.
- 2.
- 3.
- 4.
- 5.

#### 5.18.6 Exercise 13.6: Maintainability Metrics

- 1.
- 2.
- 3.
- 4.
- 5.

#### 5.19

## Further Reading

---

- 

- 

- 

- 

- 

- 

- 

5.20

## References

---

## CHAPTER

## 06

## PROFESSIONAL PRACTICE

# Professional Practice and Ethics

Practicing software engineering responsibly, collaboratively, legally, and ethically.

## LEARNING OBJECTIVES

- ✓ By the end of this chapter, you will be able to:
  - ✓ Articulate the ethical responsibilities that accompany software development
  - ✓ Apply ethical reasoning frameworks to technology decisions
  - ✓ Navigate intellectual property considerations including open source licensing
  - ✓ Communicate effectively with technical and non-technical stakeholders
  - ✓ Build productive working relationships within development teams
  - ✓ Develop strategies for continuous professional growth
  - ✓ Understand the legal and regulatory landscape affecting software
  - ✓ Recognize and respond appropriately to ethical dilemmas in practice

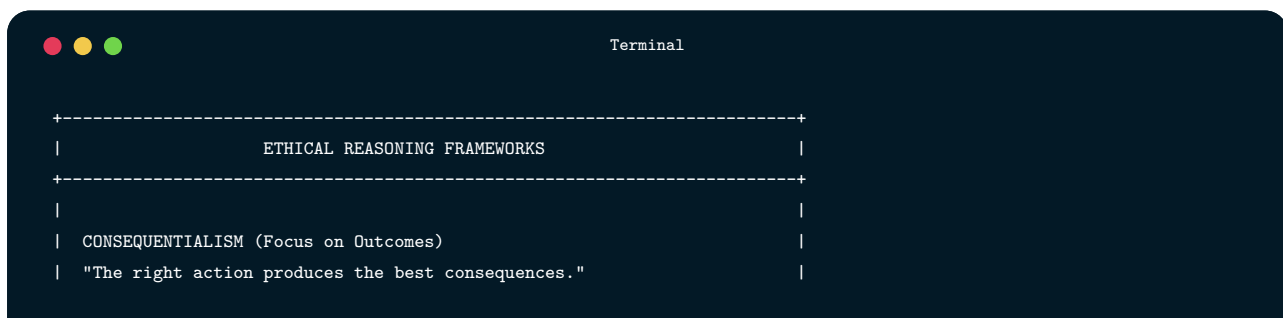
6.1

# The Ethical Dimension of Software Engineering

---

## 6.1.1 Why Ethics Matter in Software

### 6.1.2 Ethical Reasoning Frameworks

A dark-themed terminal window with a title bar containing three colored circles (red, yellow, green) and the word "Terminal". The terminal displays a list of ethical reasoning frameworks enclosed in a dashed border. The text is as follows:

```
+-----+
| ETHICAL REASONING FRAMEWORKS |
+-----+
|
| CONSEQUENTIALISM (Focus on Outcomes) |
| "The right action produces the best consequences." |
|
```

Terminal -- continued

```

|
| Evaluate actions by their results. The action that maximizes good
| outcomes (or minimizes bad ones) is ethically correct. This requires
| predicting -consequenceswhich can be difficult in complex -systems
| and deciding whose welfare counts and how to measure it.
|
| Questions to ask:
| • Who is affected by this decision?
| • What are the likely consequences for each group?
| • Does this maximize overall well-being?
| • Are we considering long-term and indirect effects?
|
+-----+
|
| DEONTOLOGY (Focus on Duties and Rules)
| "Some actions are right or wrong regardless of consequences."
|
| Certain duties and principles must be upheld regardless of outcomes.
| Lying is wrong even if it produces good results. Respecting autonomy
| matters even if paternalism might lead to better outcomes. This
| framework emphasizes rights, duties, and universal principles.
|
| Questions to ask:
| • What duties or obligations apply here?
| • Are we respecting people's rights and autonomy?
| • Could this principle be applied universally?
| • Are we treating people as ends, not merely means?
|
+-----+
|
| VIRTUE ETHICS (Focus on Character)
| "What would a person of good character do?"
|
| Focus on developing good character traits (virtues) rather than
| following rules or calculating outcomes. A virtuous person naturally
| makes good decisions. Relevant virtues include honesty, courage,
| fairness, prudence, and compassion.
|
| Questions to ask:
| • What character traits does this action express?
| • What would someone I admire do in this situation?
| • Does this decision align with who I want to be?
| • Am I acting with integrity?
|
+-----+
|
| CARE ETHICS (Focus on Relationships)

```

Terminal -- continued

```
| "What does caring for those affected require?" |
| | |
| Emphasizes the importance of relationships and caring for others, |
| especially the vulnerable. Ethical decisions maintain and nurture |
| relationships rather than applying abstract principles to isolated |
| individuals. |
| | |
| Questions to ask: |
| • How does this affect our relationships with users/stakeholders? |
| • Who is vulnerable in this situation? |
| • Are we being responsive to others' needs? |
| • What would maintaining trust require? |
| | |
+-----+
```

### 6.1.3 Professional Codes of Ethics



- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.

#### **6.1.4 Applying Ethics in Practice**



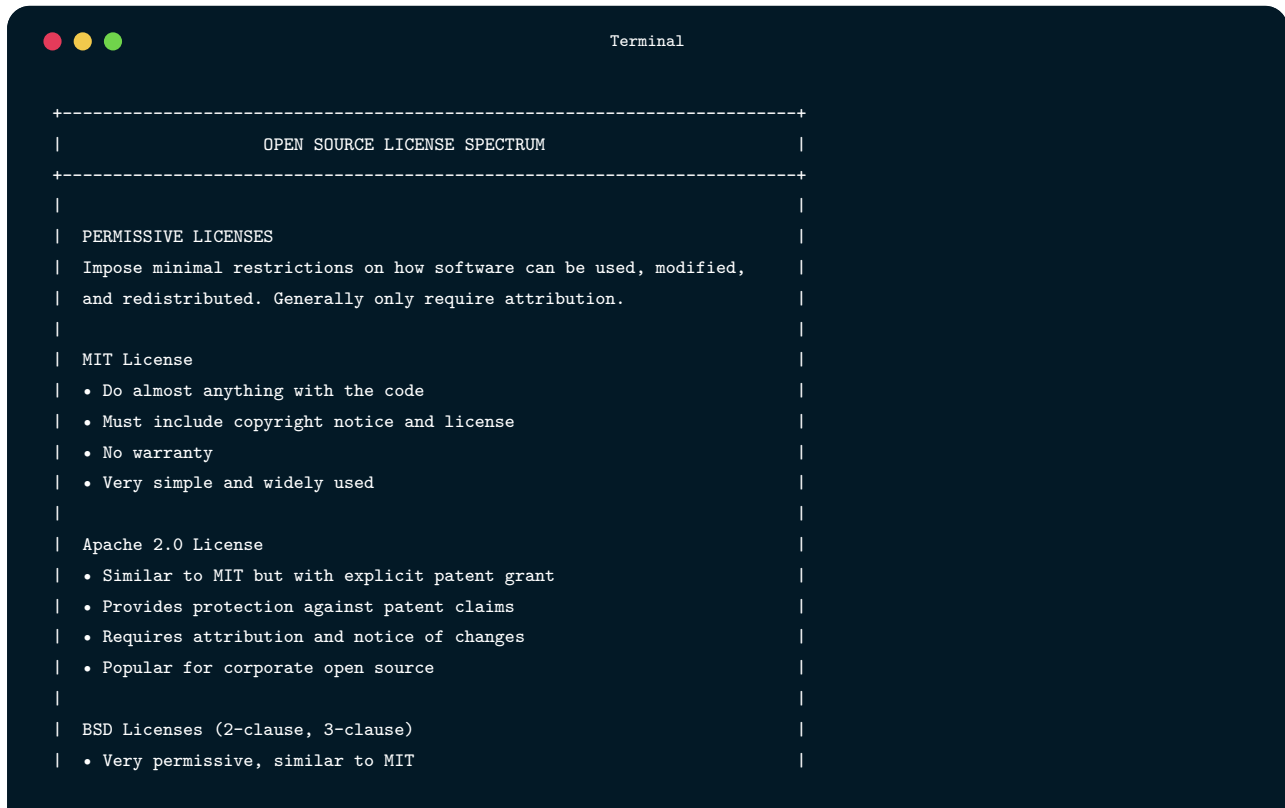
## 6.2

# Intellectual Property and Licensing

---

### 6.2.1 Types of Intellectual Property

## 6.2.2 Open Source Licensing

A terminal window with a dark blue background and white text. The title bar at the top says "Terminal" and has three colored window control buttons (red, yellow, green) on the left. The content of the terminal is a text-based diagram titled "OPEN SOURCE LICENSE SPECTRUM". It is enclosed in a dashed border. The diagram lists "PERMISSIVE LICENSES" and describes them as imposing minimal restrictions. It then lists three specific licenses: MIT License, Apache 2.0 License, and BSD Licenses (2-clause, 3-clause), each with a bulleted list of characteristics.

```
+-----+
| OPEN SOURCE LICENSE SPECTRUM |
+-----+
|
| PERMISSIVE LICENSES |
| Impose minimal restrictions on how software can be used, modified, |
| and redistributed. Generally only require attribution. |
|
| MIT License |
| • Do almost anything with the code |
| • Must include copyright notice and license |
| • No warranty |
| • Very simple and widely used |
|
| Apache 2.0 License |
| • Similar to MIT but with explicit patent grant |
| • Provides protection against patent claims |
| • Requires attribution and notice of changes |
| • Popular for corporate open source |
|
| BSD Licenses (2-clause, 3-clause) |
| • Very permissive, similar to MIT |
|
```

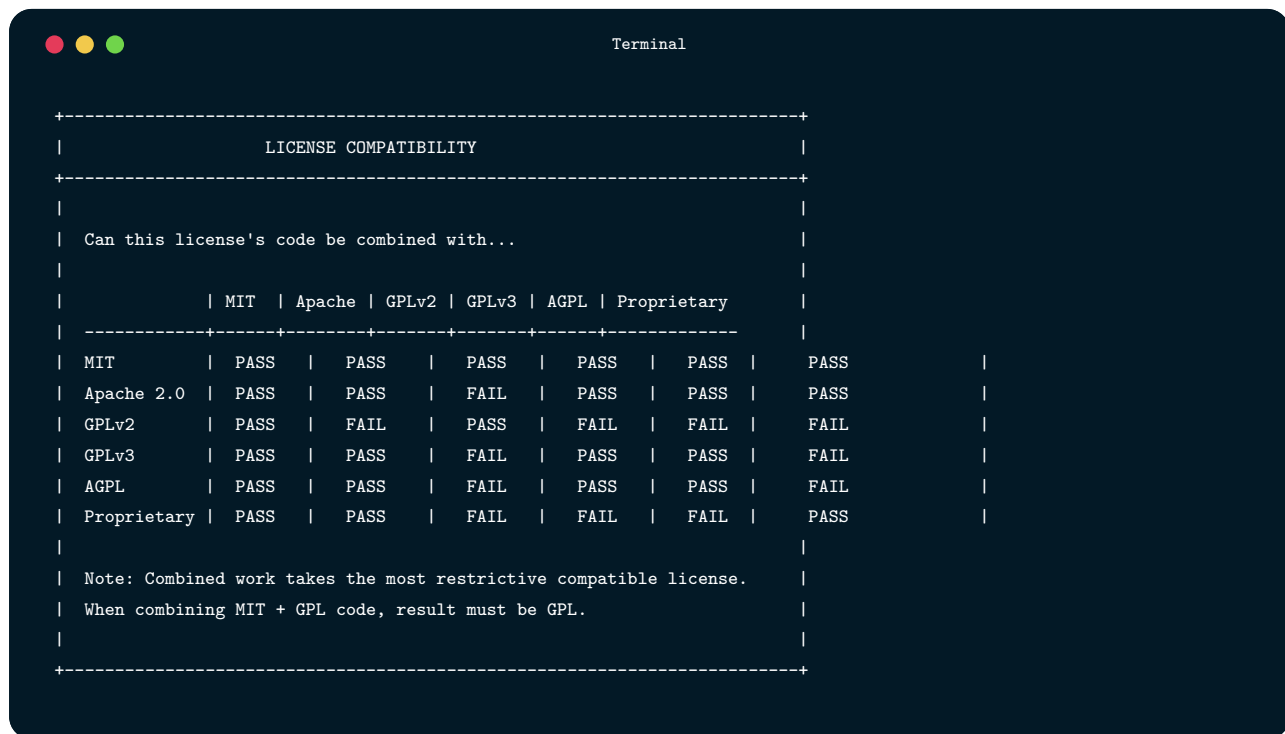
Terminal -- continued

```

| • Various versions with minor differences
| • 3-clause includes non-endorsement clause
|
+-----+
|
| COPYLEFT LICENSES
| Require that derivative works also be open source under the same
| or compatible license. "Viral" because the license propagates.
|
| GNU GPL (v2, v3)
| • Derivative works must be GPL-licensed
| • Source code must be made available
| • "Strong copyleft" - applies to entire combined work
| • GPLv3 addresses patents and "Tivoization"
|
| GNU LGPL
| • "Lesser" GPL - weaker copyleft
| • Can link proprietary code to LGPL libraries
| • The library itself remains LGPL
| • Modifications to the library must be shared
|
| AGPL
| • GPL extended to network use
| • If users interact over network, source must be available
| • Closes "SaaS loophole" in GPL
| • Rarely used due to complexity
|
+-----+
|
| CREATIVE COMMONS (for documentation, assets)
| • CC0 - Public domain dedication
| • CC BY - Attribution required
| • CC BY-SA - Attribution + ShareAlike (copyleft)
| • CC BY-NC - Attribution + NonCommercial
| • Generally not recommended for code
|
+-----+

```

### 6.2.3 License Compatibility and Compliance



A terminal window titled "Terminal" displays a table titled "LICENSE COMPATIBILITY". The table lists various licenses and their compatibility with other licenses. The table is enclosed in a dashed border. Below the table, there are two notes: "Note: Combined work takes the most restrictive compatible license." and "When combining MIT + GPL code, result must be GPL."

|             | MIT  | Apache | GPLv2 | GPLv3 | AGPL | Proprietary |
|-------------|------|--------|-------|-------|------|-------------|
| MIT         | PASS | PASS   | PASS  | PASS  | PASS | PASS        |
| Apache 2.0  | PASS | PASS   | FAIL  | PASS  | PASS | PASS        |
| GPLv2       | PASS | FAIL   | PASS  | FAIL  | FAIL | FAIL        |
| GPLv3       | PASS | PASS   | FAIL  | PASS  | PASS | FAIL        |
| AGPL        | PASS | PASS   | FAIL  | PASS  | PASS | FAIL        |
| Proprietary | PASS | PASS   | FAIL  | FAIL  | FAIL | PASS        |

Note: Combined work takes the most restrictive compatible license.  
When combining MIT + GPL code, result must be GPL.

## 6.2.4 Practical License Management

### 6.2.5 Choosing a License for Your Code

## 6.3

# Team Dynamics and Collaboration

---

### 6.3.1 Effective Development Teams



### 6.3.2 Roles and Responsibilities

| COMMON TEAM ROLES               |                                                                                                                                                                                                             |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PRODUCT OWNER / PRODUCT MANAGER | Defines what to build and why. Represents user needs and business goals. Prioritizes work based on value. Accepts or rejects completed work. Works closely with developers to clarify requirements.         |
| TECH LEAD / ARCHITECT           | Guides technical decisions and system design. Ensures architectural consistency. Mentors other developers. Balances short-term delivery with long-term sustainability. May or may not be a management role. |
| SOFTWARE DEVELOPER / ENGINEER   | Designs, implements, tests, and maintains software. Participates in planning and estimation. Reviews others' code. Collaborates with product, design, and operations to deliver value.                      |
| QA ENGINEER / TEST ENGINEER     | Designs and executes testing strategies. Identifies defects and risks. Develops automated tests. Advocates for quality throughout the development process, not just at the end.                             |
| UX DESIGNER                     | Researches user needs and behaviors. Designs interfaces and interactions. Creates prototypes. Validates designs through user testing. Works with developers to implement designs faithfully.                |
| DEVOPS / SRE                    | Manages infrastructure and deployment pipelines. Monitors production systems. Responds to incidents. Works to improve reliability and developer productivity. Bridges development and operations.           |
| ENGINEERING MANAGER             | Supports team members' growth and career development. Removes                                                                                                                                               |

Terminal -- continued

```
| obstacles. Coordinates with other teams. Responsible for hiring. |
| Sets context and direction without micromanaging. |
| |
| SCRUM MASTER / AGILE COACH |
| Facilitates agile processes. Removes impediments. Protects team |
| from distractions. Helps team improve practices. Serves the team |
| rather than managing them. |
| |
+-----+
```

### 6.3.3 Communication Practices

- 
- 
- 
- 
-

### 6.3.4 Conflict Resolution

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

### 6.3.5 Working with Non-Technical Stakeholders

6.4

# Career Development

---

## 6.4.1 The Learning Imperative

### 6.4.2 Career Paths

### 6.4.3 Building Your Professional Network

#### **6.4.4 Navigating Organizational Dynamics**

#### **6.4.5 Work-Life Balance and Sustainability**

## 6.5

# Legal and Regulatory Considerations

---

### 6.5.1 Privacy Regulations

- 
- 
-



- 
- 
- 
- 
- 
- 

- 
- 
- 
- 
- 
- 

### 6.5.2 Accessibility Requirements

-

- 
- 
- 
- 
- 
- 
- 

### 6.5.3 Industry-Specific Regulations

### 6.5.4 Liability and Professional Responsibility

6.6

## Emerging Ethical Challenges

---

### 6.6.1 Artificial Intelligence Ethics

## 6.6.2 Sustainability and Environmental Impact

- 
- 
- 
- 
- 

## 6.6.3 Security and Dual Use

6.7

# Building an Ethical Practice

---

## 6.7.1 Personal Ethical Practice

## 6.7.2 Organizational Ethical Practice

### 6.7.3 When to Walk Away

- 
- 
- 

- 
- 
- 
- 
- 

## 6.8

# Chapter Summary

---

6.9

## Key Terms

---

|                    |                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------|
| Ethics             | Branch of philosophy concerned with right and wrong conduct                             |
| Deontology         | Ethical theory judging actions by adherence to duties and rules                         |
| Copyright          | Legal protection for original creative works, including software                        |
| Open Source        | Software distributed with license granting use, modification, and redistribution rights |
| Permissive License | Open source license with minimal restrictions (e.g., MIT, Apache)                       |
| WCAG               | Web Content Accessibility Guidelines for making web content accessible                  |
| Technical Debt     | Cost of shortcuts and deferred work that impacts future development                     |

## 6.10

## Review Questions

- 1.
- 2.



3.

4.

5.

6.

7.

8.

9.

10.

## 6.11

# Hands-On Exercises

---

## 6.11.1 Exercise 14.1: Ethical Case Analysis

1.

2.

3.

4.

5.

6.

### 6.11.2 Exercise 14.2: License Audit

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

### 6.11.3 Exercise 14.3: Accessibility Evaluation

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

### 6.11.4 Exercise 14.4: Team Retrospective

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

### 6.11.5 Exercise 14.5: Career Development Plan

- 1.
- 2.
- 3.

- 4.
- 5.
- 6.

### 6.11.6 Exercise 14.6: Ethics in Design

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

### 6.12

## Further Reading

---

- 
- 
- 
- 
-

- 
- 
- 
- 

6.13

## References

---

## CHAPTER

## 07

## INTEGRATION

# Final Project Integration and Course Synthesis

Bringing the lifecycle together in a complete, polished, professional software project.

## LEARNING OBJECTIVES

- ✓ By the end of this chapter, you will be able to:
  - ✓ Integrate concepts from throughout the course into a cohesive software project
  - ✓ Apply systematic approaches to project completion and polish
  - ✓ Conduct comprehensive final testing and quality assurance
  - ✓ Prepare and deliver effective technical presentations and demonstrations
  - ✓ Reflect on learning and identify areas for continued growth
  - ✓ Create professional project documentation and portfolios
  - ✓ Synthesize software engineering principles into a coherent mental framework
  - ✓ Plan next steps for continued professional development

## 7.1

# The Integration Challenge

---

### 7.1.1 From Learning to Doing

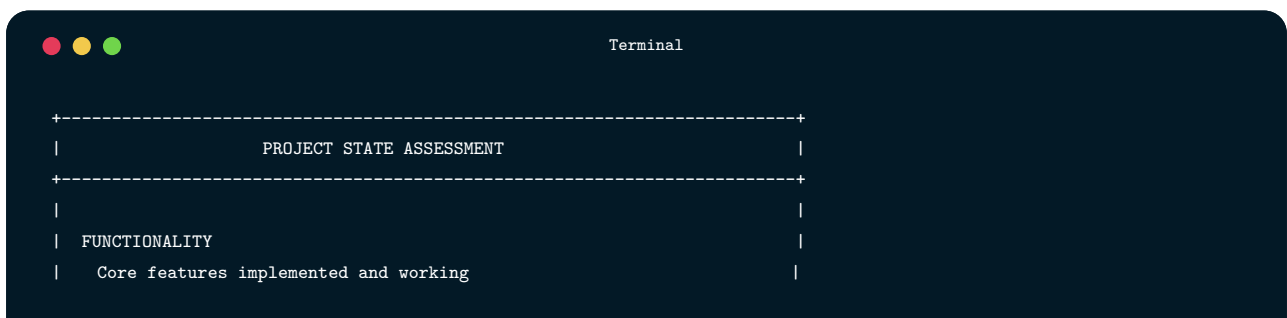
### 7.1.2 Project Integration Principles

## 7.2

## Project Completion Strategies

---

### 7.2.1 Assessing Project State

A terminal window with a dark blue background and white text. The title bar shows three colored circles (red, yellow, green) and the word "Terminal". The output is a table with dashed lines and vertical bars at the corners.

```
+-----+
| PROJECT STATE ASSESSMENT |
+-----+
|
| FUNCTIONALITY |
| Core features implemented and working |
|
```

Terminal -- continued

```
| Edge cases handled appropriately |
| Error states managed gracefully |
| User flows complete from start to finish |
|
| QUALITY |
| Code is readable and maintainable |
| Tests cover critical functionality |
| No known critical bugs |
| Performance is acceptable |
|
| OPERATIONS |
| Application deploys reliably |
| Configuration is externalized |
| Logging enables debugging |
| Monitoring reveals application health |
|
| SECURITY |
| Authentication works correctly |
| Authorization enforced on all endpoints |
| Input validation implemented |
| Sensitive data protected |
|
| DOCUMENTATION |
| README enables getting started |
| API documentation is accurate |
| Architecture decisions documented |
| Known issues acknowledged |
|
| PRESENTATION |
| Demo flow planned |
| Sample data prepared |
| Backup plans for failures |
| Talking points prepared |
|
+-----+
```

## 7.2.2 Prioritization Under Pressure



- 
- 
- 
- 

### 7.2.3 Scope Management

## 7.2.4 The Final Push

### 7.3

## Polish and Refinement

---

### 7.3.1 User Experience Polish



Terminal

```
// Level 1: Crashes or shows technical error
app.post('/api/tasks', async (req, res) => {
 // If req.body.title is undefined, this throws
 const task = await db('tasks').insert({
 title: req.body.title,
 user_id: req.user.id
 });
 res.json(task);
});

// Level 2: Catches error but provides poor feedback
app.post('/api/tasks', async (req, res) => {
 try {
 const task = await db('tasks').insert({
 title: req.body.title,
 user_id: req.user.id
 });
 res.json(task);
 } catch (error) {
 res.status(500).json({ error: 'Something went wrong' });
 }
});

// Level 3: Validates input and provides helpful feedback
app.post('/api/tasks', async (req, res) => {
 // Validate input
 const { title, description, dueDate } = req.body;

 if (!title || title.trim().length === 0) {
 return res.status(400).json({
 error: 'Title is required',
 field: 'title'
 });
 }

 if (title.length > 200) {
```

Terminal -- continued

```
return res.status(400).json({
 error: 'Title must be 200 characters or less',
 field: 'title'
});
}

if (dueDate && new Date(dueDate) < new Date()) {
 return res.status(400).json({
 error: 'Due date cannot be in the past',
 field: 'dueDate'
 });
}

try {
 const task = await db('tasks').insert({
 title: title.trim(),
 description: description?.trim() || null,
 due_date: dueDate || null,
 user_id: req.user.id,
 status: 'pending',
 created_at: new Date()
 }).returning('*');

 res.status(201).json({
 data: task[0],
 message: 'Task created successfully'
 });
} catch (error) {
 console.error('Task creation failed:', error);
 res.status(500).json({
 error: 'Unable to create task. Please try again.'
 });
}
});
```

### 7.3.2 Code Quality Polish

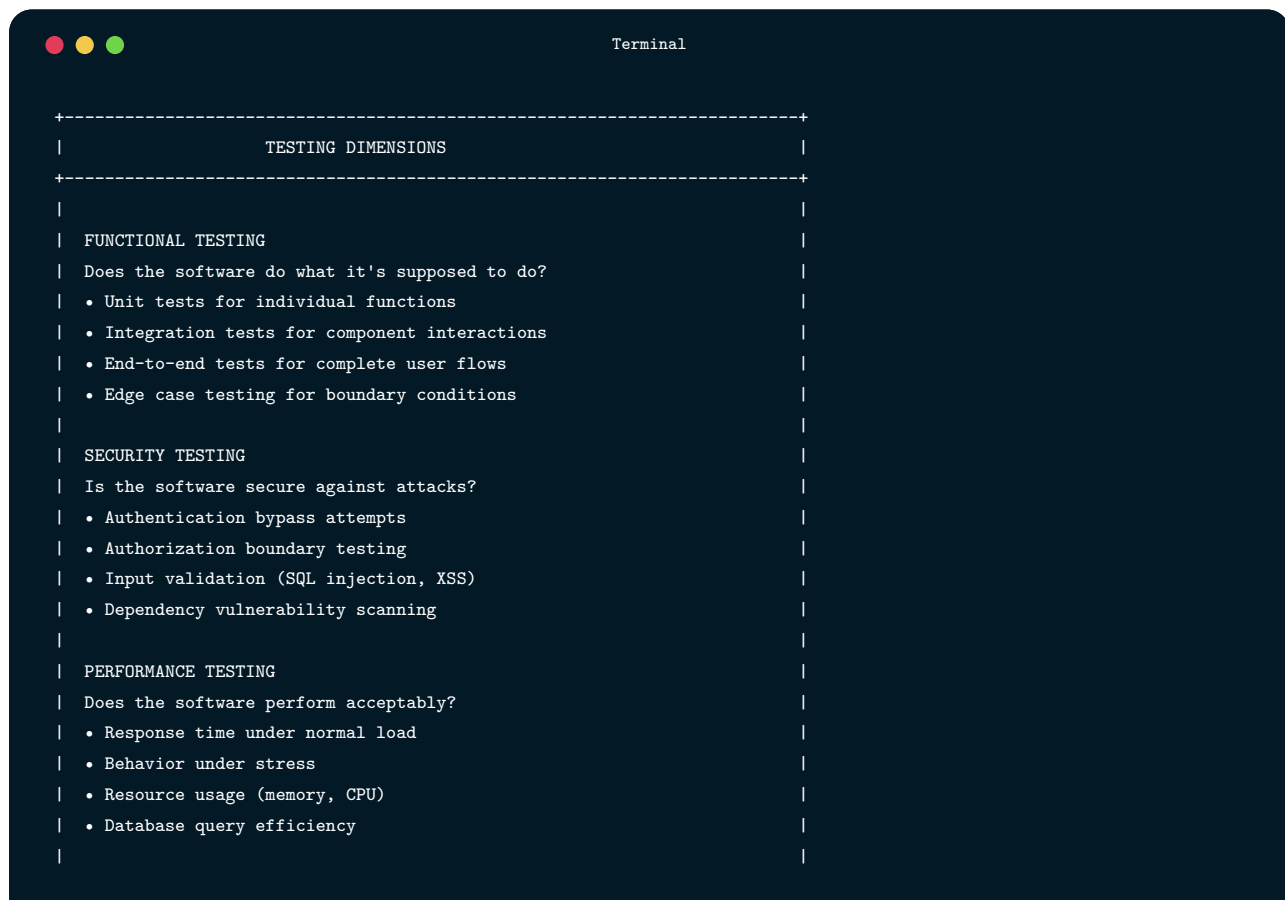
### **7.3.3 Documentation Polish**

### **7.3.4 Operational Polish**

## 7.4

# Comprehensive Testing

## 7.4.1 Testing Strategy

A terminal window with a dark blue background and white text. The title bar at the top says "Terminal" and has three colored window control buttons (red, yellow, green) on the left. The content of the terminal is a structured document for testing dimensions, enclosed in a dashed border. It lists three main categories: Functional Testing, Security Testing, and Performance Testing, each with a guiding question and a bulleted list of specific test types.

```
+-----+
| TESTING DIMENSIONS |
+-----+
|
| FUNCTIONAL TESTING |
| Does the software do what it's supposed to do? |
| • Unit tests for individual functions |
| • Integration tests for component interactions |
| • End-to-end tests for complete user flows |
| • Edge case testing for boundary conditions |
|
| SECURITY TESTING |
| Is the software secure against attacks? |
| • Authentication bypass attempts |
| • Authorization boundary testing |
| • Input validation (SQL injection, XSS) |
| • Dependency vulnerability scanning |
|
| PERFORMANCE TESTING |
| Does the software perform acceptably? |
| • Response time under normal load |
| • Behavior under stress |
| • Resource usage (memory, CPU) |
| • Database query efficiency |
|
```

Terminal -- continued

```
| USABILITY TESTING |
| Can users accomplish their goals? |
| • Task completion testing |
| • Error recovery testing |
| • Accessibility testing |
| • Cross-browser/device testing |
| |
| RELIABILITY TESTING |
| Does the software work consistently? |
| • Repeated operation testing |
| • Failure and recovery testing |
| • Data integrity verification |
| • Concurrent access testing |
| |
+-----+
```

## 7.4.2 Final Testing Checklist

### 7.4.3 Bug Triage

## 7.5

# Preparing Effective Presentations

---

### 7.5.1 Understanding Your Audience



## **7.5.2 Structuring Your Presentation**

## **7.5.3 The Art of the Demo**

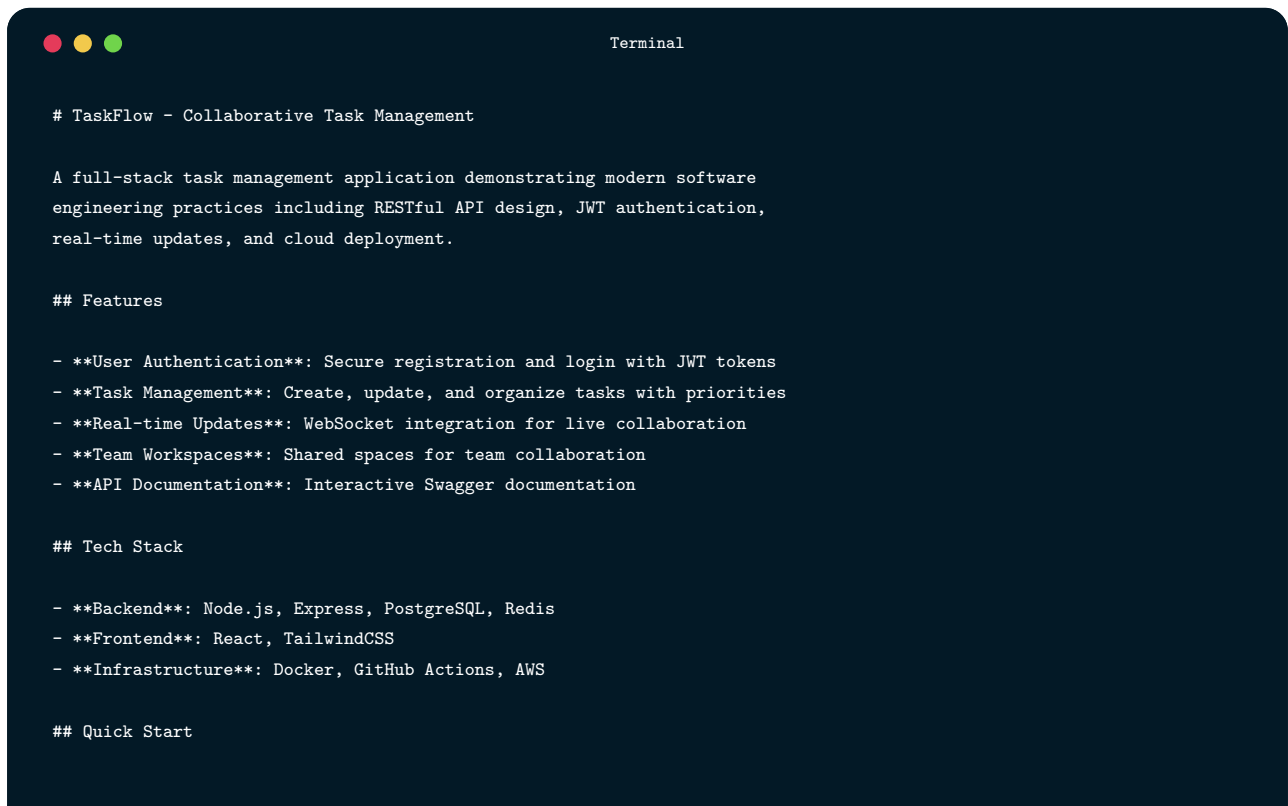
### 7.5.4 Backup Plans

### 7.5.5 Handling Questions

## 7.6

# Documentation for Posterity

## 7.6.1 README Excellence

A terminal window with a dark blue background and white text. The title bar at the top shows three colored circles (red, yellow, green) on the left and the word "Terminal" on the right. The content is a README for a project called "TaskFlow".

```
TaskFlow - Collaborative Task Management

A full-stack task management application demonstrating modern software
engineering practices including RESTful API design, JWT authentication,
real-time updates, and cloud deployment.

Features

- User Authentication: Secure registration and login with JWT tokens
- Task Management: Create, update, and organize tasks with priorities
- Real-time Updates: WebSocket integration for live collaboration
- Team Workspaces: Shared spaces for team collaboration
- API Documentation: Interactive Swagger documentation

Tech Stack

- Backend: Node.js, Express, PostgreSQL, Redis
- Frontend: React, TailwindCSS
- Infrastructure: Docker, GitHub Actions, AWS

Quick Start
```

Terminal -- continued

```
Prerequisites

- Node.js 20+
- PostgreSQL 15+
- Redis 7+

Installation

```bash
# Clone the repository
git clone https://github.com/yourusername/taskflow.git
cd taskflow

# Install dependencies
npm install

# Set up environment variables
cp .env.example .env
# Edit .env with your database credentials

# Run database migrations
npm run db:migrate

# Seed sample data (optional)
npm run db:seed

# Start development server
npm run dev
```

7.6.2 Running Tests

```
Terminal

# Run all tests
npm test

# Run with coverage
npm run test:coverage

# Run specific test suites
npm run test:unit
npm run test:integration
```

7.7

Project Structure

```
Terminal

taskflow/
+-- src/
|   +-- api/           # Express routes and middleware
|   +-- services/      # Business logic
|   +-- repositories/  # Database access
|   +-- models/        # Data models
|   +-- utils/         # Shared utilities
+-- tests/
|   +-- unit/          # Unit tests
|   +-- integration/   # Integration tests
+-- docs/              # Additional documentation
+-- scripts/           # Build and deployment scripts
```

7.8

API Documentation



POST	/api/auth/register	Register new user
GET	/api/tasks	List user's tasks
PUT	/api/tasks/ad	Update task

API Reference

7.9

Architecture



Architecture Documentation

-
-
-
-

7.10

Known Issues

-
- ☐
 - ☐
 - ☐

7.11

Future Improvements

-
- -
 -
 -

7.12

Contributing

CONTRIBUTING.md

7.13

License

LICENSE

7.14

Acknowledgments

-
-
-

```
Terminal

### Architecture Documentation

For projects of any complexity, document the overall architecture:

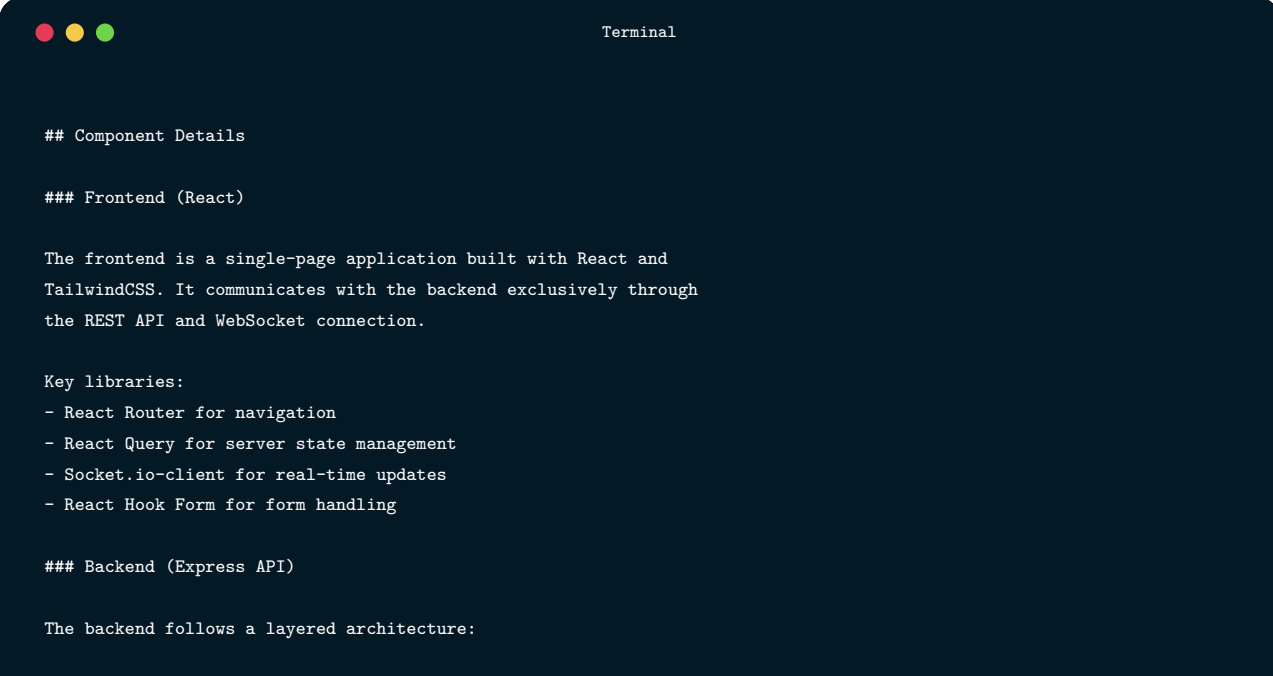
```markdown
TaskFlow Architecture

System Overview

TaskFlow is a collaborative task management system built with a
three-tier architecture: React frontend, Express API backend,
and PostgreSQL database.

Architecture Diagram
```





```
Terminal

Component Details

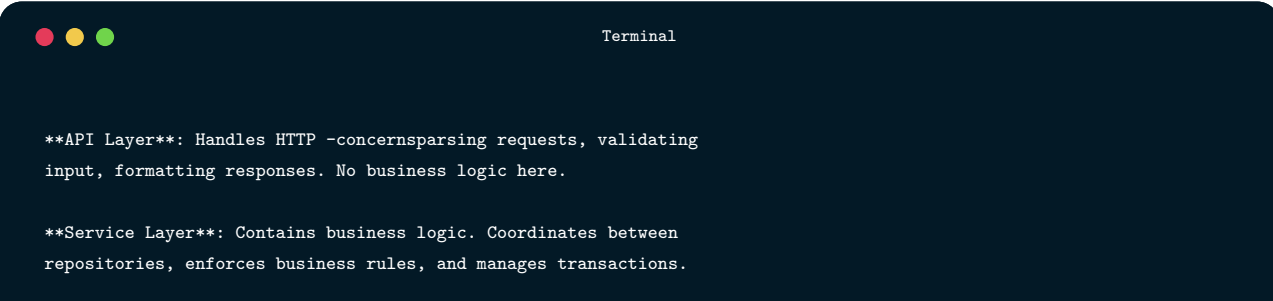
Frontend (React)

The frontend is a single-page application built with React and
TailwindCSS. It communicates with the backend exclusively through
the REST API and WebSocket connection.

Key libraries:
- React Router for navigation
- React Query for server state management
- Socket.io-client for real-time updates
- React Hook Form for form handling

Backend (Express API)

The backend follows a layered architecture:
```



```
Terminal

API Layer: Handles HTTP -concernsparsing requests, validating
input, formatting responses. No business logic here.

Service Layer: Contains business logic. Coordinates between
repositories, enforces business rules, and manages transactions.
```

Terminal -- continued

```
Repository Layer: Abstracts database operations. Services
never write SQL directly; they call repository methods.

Database Schema

```sql
-- Core tables
users (id, email, password_hash, name, created_at)
teams (id, name, owner_id, created_at)
team_members (team_id, user_id, role, joined_at)
tasks (id, title, description, status, priority, due_date,
       assignee_id, team_id, created_by, created_at, updated_at)
```

Database Schema

7.15

Key Design Decisions

7.15.1 ADR 001: JWT for Authentication

7.15.2 ADR 002: Repository Pattern

7.15.3 ADR 003: WebSocket for Real-time Updates

7.16

Security Architecture

Security Documentation

-
-
-
-
-

```
Terminal

### Lessons Learned Document

Documenting lessons learned captures valuable knowledge:

```markdown
TaskFlow: Lessons Learned

What Went Well

Early API Design
Investing time in API design before implementation paid off.
Having clear contracts enabled parallel frontend/backend work.
The few times we changed API contracts caused significant rework,
validating the importance of upfront design.

Continuous Integration
Setting up CI/CD early caught issues quickly. The discipline of
```

Terminal -- continued

```
keeping tests passing prevented accumulation of broken code.
Automated deployment eliminated "works on my machine" issues.

Regular Testing During Development
Writing tests alongside features, rather than after, improved
code quality and caught bugs early. Tests also served as
documentation of expected behavior.

What Could Have Gone Better

Database Schema Changes
We underestimated how often we'd need to modify the schema.
Early migrations were poorly planned, creating technical debt.
Lesson: Spend more time on data modeling upfront; plan for
schema evolution from the start.

Scope Management
We tried to implement too many features, leading to several
being incomplete. A smaller set of polished features would have
been better than many partial features. Lesson: Be ruthless
about scope; better to do fewer things well.

Performance Considerations
Performance testing came too late. We discovered N+1 query
problems near the deadline that required significant refactoring.
Lesson: Include basic performance testing earlier.

Technical Insights

Insight: Caching is Harder Than Expected
We added Redis caching expecting simple performance gains.
Cache invalidation proved -trickystale data bugs were subtle
and hard to reproduce. Caching should be added only when needed,
with careful invalidation strategy.

Insight: WebSocket Reconnection
Initial WebSocket implementation didn't handle disconnection
well. Users would lose real-time updates after network blips
without knowing. Lesson: Design for unreliable connections
from the start.

Insight: Testing Async Code
Async tests were flaky until we understood proper patterns.
Awaiting promises correctly and handling timeouts required
learning. Using proper async/await patterns fixed flakiness.

Recommendations for Future Projects
```

Terminal -- continued

1. **\*\*Start with data model\*\***: Invest in understanding and modeling the domain before coding.
2. **\*\*Deploy continuously\*\***: Get deployment working from day one. Deploying regularly surfaces integration issues early.
3. **\*\*Define "done" clearly\*\***: Agree on acceptance criteria before implementation, not after.
4. **\*\*Track technical debt\*\***: Acknowledge shortcuts and plan to address them. Ignoring debt doesn't make it disappear.
5. **\*\*Leave buffer time\*\***: Everything takes longer than expected. Plan for 80% scope with schedule, not 100%.

## 7.17

## Course Synthesis

---

### 7.17.1 The Software Development Lifecycle Revisited



Terminal

```
+-----+
| SOFTWARE DEVELOPMENT LIFECYCLE |
+-----+
| |
| PLANNING & REQUIREMENTS (Chapters 1-2) |
| |
+-----+
```

Terminal -- continued

```

| Understanding what to build and why
| -----
| • Stakeholder identification and engagement
| • Requirements elicitation and documentation
| • User stories and acceptance criteria
| • Scope definition and prioritization
|
| DESIGN (Chapters 3-5)
| Deciding how to build it
| -----
| • System modeling and UML
| • Architecture patterns and decisions
| • UI/UX design principles
| • API design
|
| IMPLEMENTATION (Chapters 6-8)
| Actually building it
| -----
| • Agile methodologies for organizing work
| • Version control for collaboration
| • Testing for quality assurance
| • Code review and collaboration
|
| DEPLOYMENT (Chapters 9-11)
| Getting it to users
| -----
| • CI/CD pipelines for automation
| • Data management and APIs
| • Cloud services and containerization
| • Infrastructure as code
|
| OPERATION & MAINTENANCE (Chapters 12-14)
| Keeping it running and evolving
| -----
| • Security throughout the lifecycle
| • Technical debt management
| • Refactoring and legacy system evolution
| • Professional practice and ethics
|
+-----+

```

## **7.17.2 Connecting the Concepts**

## **7.17.3 Principles That Transcend Specific Technologies**

#### 7.17.4 What This Volume Didn't Cover

#### 7.18

## Planning Continued Growth

---



### **7.18.1 Immediate Next Steps**

### **7.18.2 Building on Course Foundation**

### **7.18.3 Long-Term Professional Development**

7.19

## Chapter Summary

---

7.20

## Key Terms

---

Integration	Combining separately developed components into a working system
Polish	Attention to detail that distinguishes professional from amateur work
Graceful Degradation	System behavior that maintains partial function when components fail
Portfolio	Collection of work samples demonstrating capabilities
Lessons Learned	Documented reflection on what went well and what could improve

Bug Triage

Process of prioritizing which defects to fix given limited resources

Continuous Integration

Practice of frequently merging and testing code changes

## 7.21

# Review Questions

---

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.

## 7.22

# Hands-On Exercises

---

## 7.22.1 Exercise 15.1: Project State Assessment

- 1.
- 2.
- 3.
- 4.
- 5.

## 7.22.2 Exercise 15.2: Final Testing Sprint

- 1.
- 2.
- 3.
- 4.
- 5.

## 7.22.3 Exercise 15.3: Demo Preparation

- 1.
- 2.
- 3.
- 4.
- 5.

#### 7.22.4 Exercise 15.4: Documentation Sprint

- 1.
- 2.
- 3.
- 4.
- 5.

#### 7.22.5 Exercise 15.5: Course Reflection

- 1.
- 2.
- 3.
- 4.
- 5.

#### 7.22.6 Exercise 15.6: Portfolio Preparation

- 1.
- 2.
- 3.
- 4.
- 5.

### 7.23

# Final Project Checklist

---

### 7.23.1 Functionality

- ☐
- ☐
- ☐
- ☐

### 7.23.2 Code Quality

- ☐
- ☐
- ☐
- ☐

### 7.23.3 Testing

- ☐
- ☐
- ☐
- ☐

### 7.23.4 Security

- ☐
- ☐
- ☐
- ☐
- ☐

### 7.23.5 Documentation

- ☐
- ☐
- ☐
- ☐

### 7.23.6 Operations

- ☐
- ☐
- ☐
- ☐

### 7.23.7 Presentation

- ☐
- ☐
- ☐
- ☐

## 7.24

# Further Reading

---

- 
- 
- 
- 
- 
- 
-



7.25

# Conclusion

---

7.26

# References

---